

Exam : **Associate-Developer-Apache-Spark**

Title : Databricks Certified Associate Developer for Apache Spark 3.0 Exam

Vendor : Databricks

Version : V12.35

NO.1 Which of the following code blocks stores a part of the data in DataFrame itemsDf on executors?

- A. itemsDf.cache().count()
- B. itemsDf.cache(eager=True)
- C. cache(itemsDf)
- D. itemsDf.cache().filter()
- E. itemsDf.rdd.storeCopy()

Answer: A

Explanation

Caching means storing a copy of a partition on an executor, so it can be accessed quicker by subsequent operations, instead of having to be recalculated. cache() is a lazily-evaluated method of the DataFrame. Since count() is an action (while filter() is not), it triggers the caching process. More info: [pyspark.sql.DataFrame.cache](#) - PySpark 3.1.2 documentation, Learning Spark, 2nd Edition, Chapter 7 Static notebook | Dynamic notebook: See test 2

NO.2 The code block shown below should return only the average prediction error (column predError) of a random subset, without replacement, of approximately 15% of rows in DataFrame transactionsDf. Choose the answer that correctly fills the blanks in the code block to accomplish this. transactionsDf.__1__(__2__, __3__).__4__(avg('predError'))

- A. 1. sample
2. True
3. 0.15
4. filter
- B. 1. sample
2. False
3. 0.15
4. select
- C. 1. sample
2. 0.85
3. False
4. select
- D. 1. fraction
2. 0.15
3. True
4. where
- E. 1. fraction
2. False
3. 0.85
4. select

Answer: B

Explanation

Correct code block:

transactionsDf.sample(withReplacement=False, fraction=0.15).select(avg('predError')) You should remember that getting a random subset of rows means sampling. This, in turn should point you to the DataFrame.sample() method. Once you know this, you can look up the correct order of

arguments in the documentation (link below).

Lastly, you have to decide whether to use `filter`, `where` or `select`. `where` is just an alias for `filter()`. `filter()` is not the correct method to use here, since it would only allow you to filter rows based on some condition. However, the question asks to return only the average prediction error. You can control the columns that a query returns with the `select()` method - so this is the correct method to use here.

More info: `pyspark.sql.DataFrame.sample` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 2

NO.3 In which order should the code blocks shown below be run in order to return the number of records that are not empty in column `value` in the DataFrame resulting from an inner join of DataFrame `transactionsDf` and `itemsDf` on columns `productId` and `itemId`, respectively?

1. `.filter(~isNull(col('value')))`
2. `.count()`
3. `transactionsDf.join(itemsDf, col("transactionsDf.productId")==col("itemsDf.itemId"))`
4. `transactionsDf.join(itemsDf, transactionsDf.productId==itemsDf.itemId, how='inner')`
5. `.filter(col('value').isNotNull())`
6. `.sum(col('value'))`

A. 4, 1, 2

B. 3, 1, 6

C. 3, 1, 2

D. 3, 5, 2

E. 4, 6

Answer: A

Explanation

Correct code block:

```
transactionsDf.join(itemsDf, transactionsDf.productId==itemsDf.itemId,
how='inner').filter(~isNull(col('value'))).count()
```

Expressions `col("transactionsDf.productId")` and `col("itemsDf.itemId")` are invalid. `col()` does not accept the name of a DataFrame, only column names.

Static notebook | Dynamic notebook: See test 2

NO.4 Which of the following statements about lazy evaluation is incorrect?

A. Predicate pushdown is a feature resulting from lazy evaluation.

B. Execution is triggered by transformations.

C. Spark will fail a job only during execution, but not during definition.

D. Accumulators do not change the lazy evaluation model of Spark.

E. Lineages allow Spark to coalesce transformations into stages

Answer: B

Explanation

Execution is triggered by transformations.

Correct. Execution is triggered by actions only, not by transformations.

Lineages allow Spark to coalesce transformations into stages.

Incorrect. In Spark, lineage means a recording of transformations. This lineage enables lazy evaluation in Spark.

Predicate pushdown is a feature resulting from lazy evaluation.

Wrong. Predicate pushdown means that, for example, Spark will execute filters as early in the process as possible so that it deals with the least possible amount of data in subsequent transformations, resulting in a performance improvements.

Accumulators do not change the lazy evaluation model of Spark.

Incorrect. In Spark, accumulators are only updated when the query that refers to the is actually executed. In other words, they are not updated if the query is not (yet) executed due to lazy evaluation.

Spark will fail a job only during execution, but not during definition.

Wrong. During definition, due to lazy evaluation, the job is not executed and thus certain errors, for example reading from a non-existing file, cannot be caught. To be caught, the job needs to be executed, for example through an action.

NO.5 Which of the following code blocks returns about 150 randomly selected rows from the 1000-row DataFrame transactionsDf, assuming that any row can appear more than once in the returned DataFrame?

A. transactionsDf.resample(0.15, False, 3142)

B. transactionsDf.sample(0.15, False, 3142)

C. transactionsDf.sample(0.15)

D. transactionsDf.sample(0.85, 8429)

E. transactionsDf.sample(True, 0.15, 8261)

Answer: E

Explanation

Answering this question correctly depends on whether you understand the arguments to the DataFrame.sample() method (link to the documentation below). The arguments are as follows: DataFrame.sample(withReplacement=None, fraction=None, seed=None).

The first argument withReplacement specified whether a row can be drawn from the DataFrame multiple times. By default, this option is disabled in Spark. But we have to enable it here, since the question asks for a row being able to appear more than once. So, we need to pass True for this argument.

About replacement: "Replacement" is easiest explained with the example of removing random items from a box. When you remove those "with replacement" it means that after you have taken an item out of the box, you put it back inside. So, essentially, if you would randomly take 10 items out of a box with 100 items, there is a chance you take the same item twice or more times. "Without replacement" means that you would not put the item back into the box after removing it. So, every time you remove an item from the box, there is one less item in the box and you can never take the same item twice.

The second argument to the withReplacement method is fraction. This refers to the fraction of items that should be returned. In the question we are asked for 150 out of 1000 items - a fraction of 0.15.

The last argument is a random seed. A random seed makes a randomized processed repeatable. This means that if you would re-run the same sample() operation with the same random seed, you would get the same rows returned from the sample() command. There is no behavior around the random seed specified in the question. The varying random seeds are only there to confuse you!

More info: [pyspark.sql.DataFrame.sample - PySpark 3.1.1 documentation](#)

Static notebook | Dynamic notebook: See test 1

NO.6 The code block shown below should return a DataFrame with two columns, itemId and col. In this DataFrame, for each element in column attributes of DataFrame itemDf there should be a separate row in which the column itemId contains the associated itemId from DataFrame itemsDf. The new DataFrame should only contain rows for rows in DataFrame itemsDf in which the column attributes contains the element cozy.

A sample of DataFrame itemsDf is below.

Code block:

```
itemsDf.__1__(__2__).__3__(__4__, __5__(__6__))
```

A. 1. filter

2. array_contains("cozy")

3. select

4. "itemId"

5. explode

6. "attributes"

B. 1. where

2. "array_contains(attributes, 'cozy')"

3. select

4. itemId

5. explode

6. attributes

C. 1. filter

2. "array_contains(attributes, 'cozy')"

3. select

4. "itemId"

5. map

6. "attributes"

D. 1. filter

2. "array_contains(attributes, cozy)"

3. select

4. "itemId"

5. explode

6. "attributes"

E. 1. filter

2. "array_contains(attributes, 'cozy')"

3. select

4. "itemId"

5. explode

6. "attributes"

Answer: E

Explanation

The correct code block is:

`itemsDf.filter("array_contains(attributes, 'cozy)').select("itemId", explode("attributes"))` The key here is understanding how to use `array_contains()`. You can either use it as an expression in a string, or you can import it from `pyspark.sql.functions`. In that case, the following would also work:

itemsDf.filter(array_contains("attributes", "cozy")).select("itemId", explode("attributes")) Static notebook | Dynamic notebook: See test 1 (https://flrs.github.io/spark_practice_tests_code/#1/29.html , https://bit.ly/sparkpracticeexams_import_instructions)

NO.7 Which of the following describes the characteristics of accumulators?

- A.** Accumulators are used to pass around lookup tables across the cluster.
- B.** All accumulators used in a Spark application are listed in the Spark UI.
- C.** Accumulators can be instantiated directly via the accumulator(n) method of the pyspark.RDD module.
- D.** Accumulators are immutable.
- E.** If an action including an accumulator fails during execution and Spark manages to restart the action and complete it successfully, only the successful attempt will be counted in the accumulator.

Answer: E

Explanation

If an action including an accumulator fails during execution and Spark manages to restart the action and complete it successfully, only the successful attempt will be counted in the accumulator.

Correct, when Spark tries to rerun a failed action that includes an accumulator, it will only update the accumulator if the action succeeded.

Accumulators are immutable.

No. Although accumulators behave like write-only variables towards the executors and can only be read by the driver, they are not immutable.

All accumulators used in a Spark application are listed in the Spark UI.

Incorrect. For scala, only named, but not unnamed, accumulators are listed in the Spark UI. For pySpark, no accumulators are listed in the Spark UI - this feature is not yet implemented.

Accumulators are used to pass around lookup tables across the cluster.

Wrong - this is what broadcast variables do.

Accumulators can be instantiated directly via the accumulator(n) method of the pyspark.RDD module.

Wrong, accumulators are instantiated via the accumulator(n) method of the sparkContext, for example: counter

= spark.sparkContext.accumulator(0).

More info: python - In Spark, RDDs are immutable, then how Accumulators are implemented? - Stack Overflow, apache spark - When are accumulators truly reliable? - Stack Overflow, Spark - The Definitive Guide, Chapter 14

NO.8 Which of the following code blocks removes all rows in the 6-column DataFrame transactionsDf that have missing data in at least 3 columns?

- A.** transactionsDf.dropna("any")
- B.** transactionsDf.dropna(thresh=4)
- C.** transactionsDf.dropna("",2)
- D.** transactionsDf.dropna(thresh=2)
- E.** transactionsDf.dropna("",4)

Answer: B

Explanation

```
transactionsDf.dropna(thresh=4)
```

Correct. Note that by only working with the thresh keyword argument, the first how keyword argument is ignored. Also, figuring out which value to set for thresh can be difficult, especially when under pressure in the exam. Here, I recommend you use the notes to create a "simulation" of what different values for thresh would do to a DataFrame. Here is an explanatory image why thresh=4 is the correct answer to the question:

```
transactionsDf.dropna(thresh=2)
```

Almost right. See the comment about thresh for the correct answer above.

```
transactionsDf.dropna("any")
```

No, this would remove all rows that have at least one missing value.

```
transactionsDf.drop.na("",2)
```

No, drop.na is not a proper DataFrame method.

```
transactionsDf.dropna("",4)
```

No, this does not work and will throw an error in Spark because Spark cannot understand the first argument.

More info: `pyspark.sql.DataFrame.dropna` - PySpark 3.1.1 documentation (<https://bit.ly/2QZpiCp>)

Static notebook | Dynamic notebook: See test 1

(https://flrs.github.io/spark_practice_tests_code/#1/20.html ,

https://bit.ly/sparkpracticeexams_import_instructions)

NO.9 The code block displayed below contains an error. The code block should create DataFrame `itemsAttributesDf` which has columns `itemId` and `attribute` and lists every attribute from the `attributes` column in DataFrame `itemsDf` next to the `itemId` of the respective row in `itemsDf`. Find the error.

A sample of DataFrame `itemsDf` is below.

1.	+	-----	+
2.		<u>itemId</u> <u>attributes</u>	supplier
3.	+	-----	+
4.		1 [blue, winter, cozy]	Sports Company Inc.
5.		2 [red, summer, fresh, cooling]	YetiX
6.		3 [green, summer, travel]	Sports Company Inc.
7.	+	-----	+

Code block:

```
itemsAttributesDf = itemsDf.explode("attributes").alias("attribute").select("attribute", "itemId")
```

A. Since `itemId` is the index, it does not need to be an argument to the `select()` method.

B. The `alias()` method needs to be called after the `select()` method.

C. The `explode()` method expects a Column object rather than a string.

D. `explode()` is not a method of DataFrame. `explode()` should be used inside the `select()` method instead.

E. The `split()` method should be used inside the `select()` method instead of the `explode()` method.

Answer: D

Explanation:

The correct code block looks like this:

```
from pyspark.sql.functions import explode

itemsAttributesDf = itemsDf.select("itemId", explode("attributes").alias("attribute"))
```

Then, the first couple of rows of itemAttributesDf look like this:

```
+-----+-----+
|itemId|attribute|
+-----+-----+
|    1|    blue|
|    1|   winter|
|    1|    cozy|
|    2|    red|
+-----+-----+
only showing top 4 rows
```

explode() is not a method of DataFrame. explode() should be used inside the select() method instead. This is correct.

The split() method should be used inside the select() method instead of the explode() method.

No, the split() method is used to split strings into parts. However, column attributes is an array of strings. In this case, the explode() method is appropriate.

Since itemId is the index, it does not need to be an argument to the select() method.

No, itemId still needs to be selected, whether it is used as an index or not.

The explode() method expects a Column object rather than a string.

No, a string works just fine here. This being said, there are some valid alternatives to passing in a string:

```
itemsAttributesDf = itemsDf.select("itemId", explode(col("attributes")).alias("attribute"))

itemsAttributesDf = itemsDf.select("itemId", explode(itemsDf.attributes).alias("attribute"))

itemsAttributesDf = itemsDf.select("itemId", explode(itemsDf['attributes']).alias("attribute"))
```

The alias() method needs to be called after the select() method.

No.

More info: pyspark.sql.functions.explode - PySpark 3.1.1 documentation (<https://bit.ly/2QUZ11J>)

Static notebook | Dynamic notebook: See test 1

(https://flrs.github.io/spark_practice_tests_code/#1/22.html ,

https://bit.ly/sparkpracticeexams_import_instructions)

NO.10 The code block shown below should set the number of partitions that Spark uses when shuffling data for joins or aggregations to 100. Choose the answer that correctly fills the blanks in the code block to accomplish this.

```
spark.sql.shuffle.partitions
```

```
__1___.__2___.__3___.(__4__, 100)
```

- A.** 1. spark
2. conf
3. set
4. "spark.sql.shuffle.partitions"

- B.** 1. pyspark
2. config
3. set
4. spark.shuffle.partitions

- C.** 1. spark
2. conf
3. get
4. "spark.sql.shuffle.partitions"

- D.** 1. pyspark
2. config
3. set
4. "spark.sql.shuffle.partitions"

- E.** 1. spark
2. conf
3. set
4. "spark.sql.aggregate.partitions"

Answer: A

Explanation

Correct code block:

```
spark.conf.set("spark.sql.shuffle.partitions", 100)
```

The conf interface is part of the SparkSession, so you need to call it through spark and not pyspark. To configure spark, you need to use the set method, not the get method. get reads a property, but does not write it. The correct property to achieve what is outlined in the question is spark.sql.shuffle.partitions, which needs to be passed to set as a string. Properties spark.shuffle.partitions and spark.sql.aggregate.partitions do not exist in Spark.

Static notebook | Dynamic notebook: See test 2

NO.11 Which of the following statements about the differences between actions and transformations is correct?

- A.** Actions are evaluated lazily, while transformations are not evaluated lazily.
B. Actions generate RDDs, while transformations do not.
C. Actions do not send results to the driver, while transformations do.
D. Actions can be queued for delayed execution, while transformations can only be processed immediately.
E. Actions can trigger Adaptive Query Execution, while transformation cannot.

Answer: E

Explanation

Actions can trigger Adaptive Query Execution, while transformation cannot.

Correct. Adaptive Query Execution optimizes queries at runtime. Since transformations are evaluated lazily, Spark does not have any runtime information to optimize the query until an action is called. If Adaptive Query Execution is enabled, Spark will then try to optimize the query based on the feedback it gathers while it is evaluating the query.

Actions can be queued for delayed execution, while transformations can only be processed immediately.

No, there is no such concept as "delayed execution" in Spark. Actions cannot be evaluated lazily, meaning that they are executed immediately.

Actions are evaluated lazily, while transformations are not evaluated lazily.

Incorrect, it is the other way around: Transformations are evaluated lazily and actions trigger their evaluation.

Actions generate RDDs, while transformations do not.

No. Transformations change the data and, since RDDs are immutable, generate new RDDs along the way.

Actions produce outputs in Python and data types (integers, lists, text files,...) based on the RDDs, but they do not generate them.

Here is a great tip on how to differentiate actions from transformations: If an operation returns a DataFrame, Dataset, or an RDD, it is a transformation. Otherwise, it is an action.

Actions do not send results to the driver, while transformations do.

No. Actions send results to the driver. Think about running `DataFrame.count()`. The result of this command will return a number to the driver. Transformations, however, do not send results back to the driver. They produce RDDs that remain on the worker nodes.

More info: What is the difference between a transformation and an action in Apache Spark? |

Bartosz Mikulski, How to Speed up SQL Queries with Adaptive Query Execution

NO.12 Which of the following code blocks reads in the two-partition parquet file stored at `filePath`, making sure all columns are included exactly once even though each partition has a different schema?

Schema of first partition:

- 1.root
2. |-- transactionId: integer (nullable = true)
3. |-- predError: integer (nullable = true)
4. |-- value: integer (nullable = true)
5. |-- storeId: integer (nullable = true)
6. |-- productId: integer (nullable = true)
7. |-- f: integer (nullable = true)

Schema of second partition:

- 1.root
2. |-- transactionId: integer (nullable = true)
3. |-- predError: integer (nullable = true)
4. |-- value: integer (nullable = true)
5. |-- storeId: integer (nullable = true)
6. |-- rollId: integer (nullable = true)
7. |-- f: integer (nullable = true)

8. |-- tax_id: integer (nullable = false)

- A. spark.read.parquet(filePath, mergeSchema='y')
- B. spark.read.option("mergeSchema", "true").parquet(filePath)
- C. spark.read.parquet(filePath)
- D. 1.nx = 0

2.for file in dbutils.fs.ls(filePath):

3. if not file.name.endswith(".parquet"):

4. continue

5. df_temp = spark.read.parquet(file.path)

6. if nx == 0:

7. df = df_temp

8. else:

9. df = df.union(df_temp)

10. nx = nx+1

11.df

E. 1.nx = 0

2.for file in dbutils.fs.ls(filePath):

3. if not file.name.endswith(".parquet"):

4. continue

5. df_temp = spark.read.parquet(file.path)

6. if nx == 0:

7. df = df_temp

8. else:

9. df = df.join(df_temp, how="outer")

10. nx = nx+1

11.df

Answer: B

Explanation

This is a very tricky question and involves both knowledge about merging as well as schemas when reading parquet files.

spark.read.option("mergeSchema", "true").parquet(filePath)

Correct. Spark's DataFrameReader's mergeSchema option will work well here, since columns that appear in both partitions have matching data types. Note that mergeSchema would fail if one or more columns with the same name that appear in both partitions would have different data types.

spark.read.parquet(filePath)

Incorrect. While this would read in data from both partitions, only the schema in the parquet file that is read in first would be considered, so some columns that appear only in the second partition (e.g. tax_id) would be lost.

nx = 0

for file in dbutils.fs.ls(filePath):

if not file.name.endswith(".parquet"):

continue

df_temp = spark.read.parquet(file.path)

if nx == 0:

df = df_temp

else:

```
df = df.union(df_temp)
```

```
nx = nx+1
```

```
df
```

Wrong. The key idea of this solution is the `DataFrame.union()` command. While this command merges all data, it requires that both partitions have the exact same number of columns with identical data types.

```
spark.read.parquet(filePath, mergeSchema="y")
```

False. While using the `mergeSchema` option is the correct way to solve this problem and it can even be called with `DataFrameReader.parquet()` as in the code block, it accepts the value `True` as a boolean or string variable. But `'y'` is not a valid option.

```
nx = 0
```

```
for file in dbutils.fs.ls(filePath):
```

```
if not file.name.endswith(".parquet"):
```

```
continue
```

```
df_temp = spark.read.parquet(file.path)
```

```
if nx == 0:
```

```
df = df_temp
```

```
else:
```

```
df = df.join(df_temp, how="outer")
```

```
nx = nx+1
```

```
df
```

No. This provokes a full outer join. While the resulting `DataFrame` will have all columns of both partitions, columns that appear in both partitions will be duplicated - the question says all columns that are included in the partitions should appear exactly once.

More info: [Merging different schemas in Apache Spark](#) | by Thiago Cordon | Data Arena | Medium
Static notebook | Dynamic notebook: See test 3

NO.13 The code block shown below should return a copy of `DataFrame transactionsDf` without columns `value` and `productId` and with an additional column `associateId` that has the value 5. Choose the answer that correctly fills the blanks in the code block to accomplish this.

```
transactionsDf.__1__(__2__, __3__).__4__(__5__, 'value')
```

A. 1. `withColumn`

2. `'associateId'`

3. 5

4. `remove`

5. `'productId'`

B. 1. `withNewColumn`

2. `associateId`

3. `lit(5)`

4. `drop`

5. `productId`

C. 1. `withColumn`

2. `'associateId'`

3. `lit(5)`

4. `drop`

5. `'productId'`

D. 1. withColumnRenamed

2. 'associateId'

3. 5

4. drop

5. 'productId'

E. 1. withColumn

2. col(associateId)

3. lit(5)

4. drop

5. col(productId)

Answer: C

Explanation

Correct code block:

transactionsDf.withColumn('associateId', lit(5)).drop('productId', 'value') For solving this question it is important that you know the lit() function (link to documentation below). This function enables you to add a column of a constant value to a DataFrame.

More info: [pyspark.sql.functions.lit - PySpark 3.1.1 documentation](#)

Static notebook | Dynamic notebook: See test 1

NO.14 The code block shown below should return an exact copy of DataFrame transactionsDf that does not include rows in which values in column storeId have the value 25. Choose the answer that correctly fills the blanks in the code block to accomplish this.

A. transactionsDf.remove(transactionsDf.storeId==25)**B.** transactionsDf.where(transactionsDf.storeId!=25)**C.** transactionsDf.filter(transactionsDf.storeId==25)**D.** transactionsDf.drop(transactionsDf.storeId==25)**E.** transactionsDf.select(transactionsDf.storeId!=25)**Answer: B**

Explanation

transactionsDf.where(transactionsDf.storeId!=25)

Correct. DataFrame.where() is an alias for the DataFrame.filter() method. Using this method, it is straightforward to filter out rows that do not have value 25 in column storeId.

transactionsDf.select(transactionsDf.storeId!=25)

Wrong. The select operator allows you to build DataFrames column-wise, but when using it as shown, it does not filter out rows.

transactionsDf.filter(transactionsDf.storeId==25)

Incorrect. Although the filter expression works for filtering rows, the == in the filtering condition is inappropriate. It should be != instead.

transactionsDf.drop(transactionsDf.storeId==25)

No. DataFrame.drop() is used to remove specific columns, but not rows, from the DataFrame.

transactionsDf.remove(transactionsDf.storeId==25)

False. There is no DataFrame.remove() operator in PySpark.

More info: [pyspark.sql.DataFrame.where - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 3

NO.15 Which of the following statements about stages is correct?

- A.** Different stages in a job may be executed in parallel.
- B.** Stages consist of one or more jobs.
- C.** Stages ephemerally store transactions, before they are committed through actions.
- D.** Tasks in a stage may be executed by multiple machines at the same time.
- E.** Stages may contain multiple actions, narrow, and wide transformations.

Answer: D

Explanation

Tasks in a stage may be executed by multiple machines at the same time.

This is correct. Within a single stage, tasks do not depend on each other. Executors on multiple machines may execute tasks belonging to the same stage on the respective partitions they are holding at the same time.

Different stages in a job may be executed in parallel.

No. Different stages in a job depend on each other and cannot be executed in parallel. The nuance is that every task in a stage may be executed in parallel by multiple machines.

For example, if a job consists of Stage A and Stage B, tasks belonging to those stages may not be executed in parallel. However, tasks from Stage A may be executed on multiple machines at the same time, with each machine running it on a different partition of the same dataset. Then, afterwards, tasks from Stage B may be executed on multiple machines at the same time.

Stages may contain multiple actions, narrow, and wide transformations.

No, stages may not contain multiple wide transformations. Wide transformations mean that shuffling is required. Shuffling typically terminates a stage though, because data needs to be exchanged across the cluster. This data exchange often causes partitions to change and rearrange, making it impossible to perform tasks in parallel on the same dataset.

Stages ephemerally store transactions, before they are committed through actions.

No, this does not make sense. Stages do not "store" any data. Transactions are not "committed" in Spark.

Stages consist of one or more jobs.

No, it is the other way around: Jobs consist of one more stages.

More info: Spark: The Definitive Guide, Chapter 15.

NO.16 Which of the following statements about broadcast variables is correct?

- A.** Broadcast variables are serialized with every single task.
- B.** Broadcast variables are commonly used for tables that do not fit into memory.
- C.** Broadcast variables are immutable.
- D.** Broadcast variables are occasionally dynamically updated on a per-task basis.
- E.** Broadcast variables are local to the worker node and not shared across the cluster.

Answer: C

Explanation

Broadcast variables are local to the worker node and not shared across the cluster.

This is wrong because broadcast variables are meant to be shared across the cluster. As such, they are never just local to the worker node, but available to all worker nodes.

Broadcast variables are commonly used for tables that do not fit into memory.

This is wrong because broadcast variables can only be broadcast because they are small and do fit into memory.

Broadcast variables are serialized with every single task.

This is wrong because they are cached on every machine in the cluster, precisely avoiding to have to be serialized with every single task.

Broadcast variables are occasionally dynamically updated on a per-task basis.

This is wrong because broadcast variables are immutable - they are never updated.

More info: Spark - The Definitive Guide, Chapter 14

NO.17 Which of the following describes Spark's Adaptive Query Execution?

A. Adaptive Query Execution features include dynamically coalescing shuffle partitions, dynamically injecting scan filters, and dynamically optimizing skew joins.

B. Adaptive Query Execution is enabled in Spark by default.

C. Adaptive Query Execution reoptimizes queries at execution points.

D. Adaptive Query Execution features are dynamically switching join strategies and dynamically optimizing skew joins.

E. Adaptive Query Execution applies to all kinds of queries.

Answer: D

Explanation

Adaptive Query Execution features include dynamically coalescing shuffle partitions, dynamically injecting scan filters, and dynamically optimizing skew joins.

This is almost correct. All of these features, except for dynamically injecting scan filters, are part of Adaptive Query Execution. Dynamically injecting scan filters for join operations to limit the amount of data to be considered in a query is part of Dynamic Partition Pruning and not of Adaptive Query Execution.

Adaptive Query Execution reoptimizes queries at execution points.

No, Adaptive Query Execution reoptimizes queries at materialization points.

Adaptive Query Execution is enabled in Spark by default.

No, Adaptive Query Execution is disabled in Spark needs to be enabled through the `spark.sql.adaptive.enabled` property.

Adaptive Query Execution applies to all kinds of queries.

No, Adaptive Query Execution applies only to queries that are not streaming queries and that contain at least one exchange (typically expressed through a join, aggregate, or window operator) or one subquery.

More info: How to Speed up SQL Queries with Adaptive Query Execution, Learning Spark, 2nd Edition, Chapter 12 (<https://bit.ly/3tOh8M1>)

NO.18 The code block shown below should store DataFrame transactionsDf on two different executors, utilizing the executors' memory as much as possible, but not writing anything to disk. Choose the answer that correctly fills the blanks in the code block to accomplish this.

```
1.from pyspark import StorageLevel
```

```
2.transactionsDf.__1__(StorageLevel.__2__).__3__
```

A. 1. cache

2. MEMORY_ONLY_2

3. count()

B. 1. persist

2. DISK_ONLY_2

3. count()

C. 1. persist

2. MEMORY_ONLY_2

3. select()

D. 1. cache

2. DISK_ONLY_2

3. count()

E. 1. persist

2. MEMORY_ONLY_2

3. count()

Answer: E

Explanation

Correct code block:

```
from pyspark import StorageLevel
```

```
transactionsDf.persist(StorageLevel.MEMORY_ONLY_2).count()
```

Only persist takes different storage levels, so any option using cache() cannot be correct. persist() is evaluated lazily, so an action needs to follow this command. select() is not an action, but count() is - so all options using select() are incorrect.

Finally, the question states that "the executors' memory should be utilized as much as possible, but not writing anything to disk". This points to a MEMORY_ONLY storage level. In this storage level, partitions that do not fit into memory will be recomputed when they are needed, instead of being written to disk, as with the storage option MEMORY_AND_DISK. Since the data need to be duplicated across two executors, _2 needs to be appended to the storage level.

Static notebook | Dynamic notebook: See test 2

NO.19 Which of the following statements about storage levels is incorrect?

A. The cache operator on DataFrames is evaluated like a transformation.

B. In client mode, DataFrames cached with the MEMORY_ONLY_2 level will not be stored in the edge node's memory.

C. Caching can be undone using the DataFrame.unpersist() operator.

D. MEMORY_AND_DISK replicates cached DataFrames both on memory and disk.

E. DISK_ONLY will not use the worker node's memory.

Answer: D

Explanation

MEMORY_AND_DISK replicates cached DataFrames both on memory and disk.

Correct, this statement is wrong. Spark prioritizes storage in memory, and will only store data on disk that does not fit into memory.

DISK_ONLY will not use the worker node's memory.

Wrong, this statement is correct. DISK_ONLY keeps data only on the worker node's disk, but not in memory.

In client mode, DataFrames cached with the MEMORY_ONLY_2 level will not be stored in the edge node's memory.

Wrong, this statement is correct. In fact, Spark does not have a provision to cache DataFrames in the driver (which sits on the edge node in client mode). Spark caches DataFrames in the executors' memory.

Caching can be undone using the `DataFrame.unpersist()` operator.

Wrong, this statement is correct. Caching, as achieved via the `DataFrame.cache()` or `DataFrame.persist()` operators can be undone using the `DataFrame.unpersist()` operator. This operator will remove all of its parts from the executors' memory and disk.

The cache operator on DataFrames is evaluated like a transformation.

Wrong, this statement is correct. `DataFrame.cache()` is evaluated like a transformation: Through lazy evaluation. This means that after calling `DataFrame.cache()` the command will not have any effect until you call a subsequent action, like `DataFrame.cache().count()`.

More info: [pyspark.sql.DataFrame.unpersist - PySpark 3.1.2 documentation](#)

NO.20 Which of the following code blocks produces the following output, given DataFrame `transactionsDf`?

Output:

1.root

2. |-- transactionId: integer (nullable = true)

3. |-- predError: integer (nullable = true)

4. |-- value: integer (nullable = true)

5. |-- storeId: integer (nullable = true)

6. |-- productId: integer (nullable = true)

7. |-- f: integer (nullable = true)

DataFrame `transactionsDf`:

1.+-----+-----+-----+-----+-----+-----+

2.| transactionId | predError | value | storeId | productId | f |

3.+-----+-----+-----+-----+-----+-----+

4.| 1 | 3 | 4 | 25 | 1 | null |

5.| 2 | 6 | 7 | 2 | 2 | null |

6.| 3 | 3 | null | 25 | 3 | null |

7.+-----+-----+-----+-----+-----+-----+

A. `transactionsDf.schema.print()`

B. `transactionsDf.rdd.printSchema()`

C. `transactionsDf.rdd.formatSchema()`

D. `transactionsDf.printSchema()`

E. `print(transactionsDf.schema)`

Answer: D

Explanation

The output is the typical output of a `DataFrame.printSchema()` call. The DataFrame's RDD representation does not have a `printSchema` or `formatSchema` method (find available methods in the RDD documentation linked below). The output of `print(transactionsDf.schema)` is this:

`StructType(List(StructField(transactionId,IntegerType,true),StructField(predError,IntegerType,true),StructField`

`(value,IntegerType,true),StructField(storeId,IntegerType,true),StructField(productId,IntegerType,true),StructField(f,IntegerType,true))` It includes the same information as the nicely formatted original output, but is not nicely formatted itself. Lastly, the DataFrame's schema attribute does not have a `print()` method.

More info:

- [pyspark.RDD: pyspark.RDD - PySpark 3.1.2 documentation](#)
- [DataFrame.printSchema\(\): pyspark.sql.DataFrame.printSchema - PySpark 3.1.2 documentation](#)

c notebook | Dynamic notebook: See test 2

NO.21 The code block displayed below contains one or more errors. The code block should load parquet files at location filePath into a DataFrame, only loading those files that have been modified before

2029-03-20 05:44:46. Spark should enforce a schema according to the schema shown below. Find the error.

Schema:

- 1.root
2. |-- itemId: integer (nullable = true)
3. |-- attributes: array (nullable = true)
4. | |-- element: string (containsNull = true)
5. |-- supplier: string (nullable = true)

Code block:

- 1.schema = StructType([
2. StructType("itemId", IntegerType(), True),
3. StructType("attributes", ArrayType(StringType(), True), True),
4. StructType("supplier", StringType(), True)
- 5.])
- 6.
- 7.spark.read.options("modifiedBefore", "2029-03-20T05:44:46").schema(schema).load(filePath)

A. The attributes array is specified incorrectly, Spark cannot identify the file format, and the syntax of the call to Spark's DataFrameReader is incorrect.

B. Columns in the schema definition use the wrong object type and the syntax of the call to Spark's DataFrameReader is incorrect.

C. The data type of the schema is incompatible with the schema() operator and the modification date threshold is specified incorrectly.

D. Columns in the schema definition use the wrong object type, the modification date threshold is specified incorrectly, and Spark cannot identify the file format.

E. Columns in the schema are unable to handle empty values and the modification date threshold is specified incorrectly.

Answer: D

Explanation

Correct code block:

```
schema = StructType([
    StructField("itemId", IntegerType(), True),
    StructField("attributes", ArrayType(StringType(), True), True),
    StructField("supplier", StringType(), True)
])
```

spark.read.options(modifiedBefore="2029-03-20T05:44:46").schema(schema).parquet(filePath) This question is more difficult than what you would encounter in the exam. In the exam, for this question type, only one error needs to be identified and not "one or multiple" as in the question.

Columns in the schema definition use the wrong object type, the modification date threshold is specified incorrectly, and Spark cannot identify the file format.

Correct! Columns in the schema definition should use the StructField type. Building a schema from

`pyspark.sql.types`, as here using classes like `StructType` and `StructField`, is one of multiple ways of expressing a schema in Spark. A `StructType` always contains a list of `StructFields` (see documentation linked below). So, nesting `StructType` and `StructType` as shown in the question is wrong.

The modification date threshold should be specified by a keyword argument like `options(modifiedBefore="2029-03-20T05:44:46")` and not two consecutive non-keyword arguments as in the original code block (see documentation linked below).

Spark cannot identify the file format correctly, because either it has to be specified by using the `DataFrameReader.format()`, as an argument to `DataFrameReader.load()`, or directly by calling, for example, `DataFrameReader.parquet()`.

Columns in the schema are unable to handle empty values and the modification date threshold is specified incorrectly.

No. If `StructField` would be used for the columns instead of `StructType` (see above), the third argument specified whether the column is nullable. The original schema shows that columns should be nullable and this is specified correctly by the third argument being `True` in the schema in the code block.

It is correct, however, that the modification date threshold is specified incorrectly (see above).

The attributes array is specified incorrectly, Spark cannot identify the file format, and the syntax of the call to Spark's `DataFrameReader` is incorrect.

Wrong. The attributes array is specified correctly, following the syntax for `ArrayType` (see linked documentation below). That Spark cannot identify the file format is correct, see correct answer above. In addition, the `DataFrameReader` is called correctly through the `SparkSession` `spark`. Columns in the schema definition use the wrong object type and the syntax of the call to Spark's `DataFrameReader` is incorrect.

Incorrect, the object types in the schema definition are correct and syntax of the call to Spark's `DataFrameReader` is correct.

The data type of the schema is incompatible with the `schema()` operator and the modification date threshold is specified incorrectly.

False. The data type of the schema is `StructType` and an accepted data type for the `DataFrameReader.schema()` method. It is correct however that the modification date threshold is specified incorrectly (see correct answer above).

NO.22 Which of the following statements about Spark's execution hierarchy is correct?

- A.** In Spark's execution hierarchy, a job may reach over multiple stage boundaries.
- B.** In Spark's execution hierarchy, manifests are one layer above jobs.
- C.** In Spark's execution hierarchy, a stage comprises multiple jobs.
- D.** In Spark's execution hierarchy, executors are the smallest unit.
- E.** In Spark's execution hierarchy, tasks are one layer above slots.

Answer: A

Explanation

In Spark's execution hierarchy, a job may reach over multiple stage boundaries.

Correct. A job is a sequence of stages, and thus may reach over multiple stage boundaries.

In Spark's execution hierarchy, tasks are one layer above slots.

Incorrect. Slots are not a part of the execution hierarchy. Tasks are the lowest layer.

In Spark's execution hierarchy, a stage comprises multiple jobs.

No. It is the other way around - a job consists of one or multiple stages.

In Spark's execution hierarchy, executors are the smallest unit.

False. Executors are not a part of the execution hierarchy. Tasks are the smallest unit!

In Spark's execution hierarchy, manifests are one layer above jobs.

Wrong. Manifests are not a part of the Spark ecosystem.

NO.23 The code block displayed below contains multiple errors. The code block should remove column transactionDate from DataFrame transactionsDf and add a column transactionTimestamp in which dates that are expressed as strings in column transactionDate of DataFrame transactionsDf are converted into unix timestamps. Find the errors.

Sample of DataFrame transactionsDf:

```
1.+-----+-----+----+-----+-----+----+-----+
2.| transactionId | predError | value | storeId | productId | f | transactionDate |
3.+-----+-----+----+-----+-----+----+-----+
4.| 1 | 3 | 4 | 25 | 1 | null | 2020-04-26 15:35 |
5.| 2 | 6 | 7 | 2 | 2 | null | 2020-04-13 22:01 |
6.| 3 | 3 | null | 25 | 3 | null | 2020-04-02 10:53 |
7.+-----+-----+----+-----+-----+----+-----+ Code block:
```

```
1.transactionsDf = transactionsDf.drop("transactionDate")
2.transactionsDf["transactionTimestamp"] = unix_timestamp("transactionDate", "yyyy-MM-dd")
```

A. Column transactionDate should be dropped after transactionTimestamp has been written. The string indicating the date format should be adjusted. The withColumn operator should be used instead of the existing column assignment. Operator to_unixtime() should be used instead of unix_timestamp().

B. Column transactionDate should be dropped after transactionTimestamp has been written. The withColumn operator should be used instead of the existing column assignment. Column transactionDate should be wrapped in a col() operator.

C. Column transactionDate should be wrapped in a col() operator.

D. The string indicating the date format should be adjusted. The withColumnReplaced operator should be used instead of the drop and assign pattern in the code block to replace column transactionDate with the new column transactionTimestamp.

E. Column transactionDate should be dropped after transactionTimestamp has been written. The string indicating the date format should be adjusted. The withColumn operator should be used instead of the existing column assignment.

Answer: E

Explanation

This question requires a lot of thinking to get right. For solving it, you may take advantage of the digital notepad that is provided to you during the test. You have probably seen that the code block includes multiple errors. In the test, you are usually confronted with a code block that only contains a single error. However, since you are practicing here, this challenging multi-error question will make it easier for you to deal with single-error questions in the real exam.

You can clearly see that column transactionDate should be dropped only after transactionTimestamp has been written. This is because to generate column transactionTimestamp, Spark needs to read the values from column transactionDate.

Values in column transactionDate in the original transactionsDf DataFrame look like 2020-04-26 15:35. So, to convert those correctly, you would have to pass yyyy-MM-dd HH:mm. In other words: The string indicating the date format should be adjusted.

While you might be tempted to change `unix_timestamp()` to `to_unixtime()` (in line with the `from_unixtime()` operator), this function does not exist in Spark. `unix_timestamp()` is the correct operator to use here.

Also, there is no `DataFrame.withColumnReplaced()` operator. A similar operator that exists is `DataFrame.withColumnRenamed()`.

Whether you use `col()` or not is irrelevant with `unix_timestamp()` - the command is fine with both.

Finally, you cannot assign a column like `transactionsDf["columnName"] = ...` in Spark. This is Pandas syntax (Pandas is a popular Python package for data analysis), but it is not supported in Spark.

So, you need to use Spark's `DataFrame.withColumn()` syntax instead.

More info: `pyspark.sql.functions.unix_timestamp` - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 3

NO.24 Which of the following code blocks returns a DataFrame showing the mean value of column "value" of DataFrame `transactionsDf`, grouped by its column `storeId`?

- A. `transactionsDf.groupBy(col(storeId).avg())`
- B. `transactionsDf.groupBy("storeId").avg(col("value"))`
- C. `transactionsDf.groupBy("storeId").agg(avg("value"))`
- D. `transactionsDf.groupBy("storeId").agg(average("value"))`
- E. `transactionsDf.groupBy("value").average()`

Answer: C

Explanation

This question tests your knowledge about how to use the `groupBy` and `agg` pattern in Spark. Using the documentation, you can find out that there is no `average()` method in `pyspark.sql.functions`.

Static notebook | Dynamic notebook: See test 2

NO.25 Which of the following code blocks returns a copy of DataFrame `itemsDf` where the column `supplier` has been renamed to `manufacturer`?

- A. `itemsDf.withColumn(["supplier", "manufacturer"])`
- B. `itemsDf.withColumn("supplier").alias("manufacturer")`
- C. `itemsDf.withColumnRenamed("supplier", "manufacturer")`
- D. `itemsDf.withColumnRenamed(col("manufacturer"), col("supplier"))`
- E. `itemsDf.withColumnsRenamed("supplier", "manufacturer")`

Answer: C

Explanation

`itemsDf.withColumnRenamed("supplier", "manufacturer")`

Correct! This uses the relatively trivial DataFrame method `withColumnRenamed` for renaming column `supplier` to column `manufacturer`.

Note that the question asks for "a copy of DataFrame `itemsDf`". This may be confusing if you are not familiar with Spark yet. RDDs (Resilient Distributed Datasets) are the foundation of Spark DataFrames and are immutable. As such, DataFrames are immutable, too. Any command that changes anything in the DataFrame therefore necessarily returns a copy, or a new version, of it that has the changes applied.

`itemsDf.withColumnsRenamed("supplier", "manufacturer")`

Incorrect. Spark's DataFrame API does not have a `withColumnsRenamed()` method.

`itemsDf.withColumnRenamed(col("manufacturer"), col("supplier"))`

No. Watch out - although the `col()` method works for many methods of the DataFrame API, `withColumnRenamed` is not one of them. As outlined in the documentation linked below, `withColumnRenamed` expects strings.

```
itemsDf.withColumn(["supplier", "manufacturer"])
```

Wrong. While `DataFrame.withColumn()` exists in Spark, it has a different purpose than renaming columns.

`withColumn` is typically used to add columns to DataFrames, taking the name of the new column as a first, and a Column as a second argument. Learn more via the documentation that is linked below.

```
itemsDf.withColumn("supplier").alias("manufacturer")
```

No. While `DataFrame.withColumn()` exists, it requires 2 arguments. Furthermore, the `alias()` method on DataFrames would not help the cause of renaming a column much. `DataFrame.alias()` can be useful in addressing the input of join statements. However, this is far outside of the scope of this question. If you are curious nevertheless, check out the link below.

More info: `pyspark.sql.DataFrame.withColumnRenamed` - PySpark 3.1.1 documentation,

`pyspark.sql.DataFrame.withColumn` - PySpark 3.1.1 documentation, and `pyspark.sql.DataFrame.alias`

- PySpark 3.1.2 documentation (<https://bit.ly/3aSB5tm> , <https://bit.ly/2Tv4rbE> ,

<https://bit.ly/2RbhBd2>) Static notebook | Dynamic notebook: See test 1

(https://flrs.github.io/spark_practice_tests_code/#1/31.html ,

https://bit.ly/sparkpracticeexams_import_instructions)

NO.26 Which of the following code blocks stores DataFrame `itemsDf` in executor memory and, if insufficient memory is available, serializes it and saves it to disk?

- A. `itemsDf.persist(StorageLevel.MEMORY_ONLY)`
- B. `itemsDf.cache(StorageLevel.MEMORY_AND_DISK)`
- C. `itemsDf.store()`
- D. `itemsDf.cache()`
- E. `itemsDf.write.option('destination', 'memory').save()`

Answer: D

Explanation

The key to solving this question is knowing (or reading in the documentation) that, by default, `cache()` stores values to memory and writes any partitions for which there is insufficient memory to disk.

`persist()` can achieve the exact same behavior, however not with the `StorageLevel.MEMORY_ONLY` option listed here. It is also worth noting that `cache()` does not have any arguments.

If you have troubles finding the storage level information in the documentation, please also see this student Q&A thread that sheds some light here.

Static notebook | Dynamic notebook: See test 2

NO.27 Which of the following code blocks applies the boolean-returning Python function `evaluateTestSuccess` to column `storeId` of DataFrame `transactionsDf` as a user-defined function?

- A. 1.`from pyspark.sql import types as T`
2.`evaluateTestSuccessUDF = udf(evaluateTestSuccess, T.BooleanType())`
3.`transactionsDf.withColumn("result", evaluateTestSuccessUDF(col("storeId")))`
- B. 1.`evaluateTestSuccessUDF = udf(evaluateTestSuccess)`
2.`transactionsDf.withColumn("result", evaluateTestSuccessUDF(storeId))`
- C. 1.`from pyspark.sql import types as T`

```

2.evaluateTestSuccessUDF = udf(evaluateTestSuccess, T.IntegerType())
3.transactionsDf.withColumn("result", evaluateTestSuccess(col("storeId")))
D. 1.evaluateTestSuccessUDF = udf(evaluateTestSuccess)
2.transactionsDf.withColumn("result", evaluateTestSuccessUDF(col("storeId")))
E. 1.from pyspark.sql import types as T
2.evaluateTestSuccessUDF = udf(evaluateTestSuccess, T.BooleanType())
3.transactionsDf.withColumn("result", evaluateTestSuccess(col("storeId")))

```

Answer: A

Explanation

Recognizing that the UDF specification requires a return type (unless it is a string, which is the default) is important for solving this question. In addition, you should make sure that the generated UDF (evaluateTestSuccessUDF) and not the Python function (evaluateTestSuccess) is applied to column storeId.

More info: [pyspark.sql.functions.udf - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 2

NO.28 Which of the following code blocks reads in the JSON file stored at filePath, enforcing the schema expressed in JSON format in variable json_schema, shown in the code block below?

Code block:

```

1.json_schema = """
2.{ "type": "struct",
3.  "fields": [
4.    {
5.      "name": "itemId",
6.      "type": "integer",
7.      "nullable": true,
8.      "metadata": {}
9.    },
10.   {
11.     "name": "supplier",
12.     "type": "string",
13.     "nullable": true,
14.     "metadata": {}
15.   }
16. ]
17.}
18. """

```

```

A. spark.read.json(filePath, schema=json_schema)
B. spark.read.schema(json_schema).json(filePath)
1.schema = StructType.fromJson(json.loads(json_schema))
2.spark.read.json(filePath, schema=schema)
C. spark.read.json(filePath, schema=schema_of_json(json_schema))
D. spark.read.json(filePath, schema=spark.read.json(json_schema))

```

Answer: B

Explanation

Spark provides a way to digest JSON-formatted strings as schema. However, it is not trivial to use. Although slightly above exam difficulty, this question is beneficial to your exam preparation, since it helps you to familiarize yourself with the concept of enforcing schemas on data you are reading in - a topic within the scope of the exam.

The first answer that jumps out here is the one that uses `spark.read.schema` instead of `spark.read.json`. Looking at the documentation of `spark.read.schema` (linked below), we notice that the operator expects types `pyspark.sql.types.StructType` or `str` as its first argument. While variable `json_schema` is a string, the documentation states that the `str` should be "a DDL-formatted string (For example `col0 INT, col1 DOUBLE`)". Variable `json_schema` does not contain a string in this type of format, so this answer option must be wrong.

With four potentially correct answers to go, we now look at the schema parameter of `spark.read.json()` (documentation linked below). Here, too, the schema parameter expects an input of type `pyspark.sql.types.StructType` or "a DDL-formatted string (For example `col0 INT, col1 DOUBLE`)". We already know that `json_schema` does not follow this format, so we should focus on how we can transform `json_schema` into `pyspark.sql.types.StructType`. Hereby, we also eliminate the option where `schema=json_schema`.

The option that includes `schema=spark.read.json(json_schema)` is also a wrong pick, since `spark.read.json` returns a `DataFrame`, and not a `pyspark.sql.types.StructType` type.

Ruling out the option which includes `schema_of_json(json_schema)` is rather difficult. The operator's documentation (linked below) states that it "[p]arses a JSON string and infers its schema in DDL format". This use case is slightly different from the case at hand: `json_schema` already is a schema definition, it does not make sense to "infer" a schema from it. In the documentation you can see an example use case which helps you understand the difference better. Here, you pass string `'{a: 1}'` to `schema_of_json()` and the method infers a DDL-format schema `STRUCT<a: BIGINT>` from it.

In our case, we may end up with the output schema of `schema_of_json()` describing the schema of the JSON schema, instead of using the schema itself. This is not the right answer option.

Now you may consider looking at the `StructType.fromJson()` method. It returns a variable of type `StructType` - exactly the type which the schema parameter of `spark.read.json` expects.

Although we could have looked at the correct answer option earlier, this explanation is kept as exhaustive as necessary to teach you how to systematically eliminate wrong answer options.

More info:

- `pyspark.sql.DataFrameReader.schema` - PySpark 3.1.2 documentation
- `pyspark.sql.DataFrameReader.json` - PySpark 3.1.2 documentation
- `pyspark.sql.functions.schema_of_json` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

NO.29 Which of the following statements about garbage collection in Spark is incorrect?

- A.** Garbage collection information can be accessed in the Spark UI's stage detail view.
- B.** Optimizing garbage collection performance in Spark may limit caching ability.
- C.** Manually persisting RDDs in Spark prevents them from being garbage collected.
- D.** In Spark, using the G1 garbage collector is an alternative to using the default Parallel garbage collector.
- E.** Serialized caching is a strategy to increase the performance of garbage collection.

Answer: C

Explanation

Manually persisting RDDs in Spark prevents them from being garbage collected.

This statement is incorrect, and thus the correct answer to the question. Spark's garbage collector will remove even persisted objects, albeit in an "LRU" fashion. LRU stands for least recently used. So, during a garbage collection run, the objects that were used the longest time ago will be garbage collected first.

See the linked StackOverflow post below for more information.

Serialized caching is a strategy to increase the performance of garbage collection.

This statement is correct. The more Java objects Spark needs to collect during garbage collection, the longer it takes. Storing a collection of many Java objects, such as a DataFrame with a complex schema, through serialization as a single byte array thus increases performance. This means that garbage collection takes less time on a serialized DataFrame than an unserialized DataFrame.

Optimizing garbage collection performance in Spark may limit caching ability.

This statement is correct. A full garbage collection run slows down a Spark application. When taking about

"tuning" garbage collection, we mean reducing the amount or duration of these slowdowns.

A full garbage collection run is triggered when the Old generation of the Java heap space is almost full. (If you are unfamiliar with this concept, check out the link to the Garbage Collection Tuning docs below.) Thus, one measure to avoid triggering a garbage collection run is to prevent the Old generation share of the heap space to be almost full.

To achieve this, one may decrease its size. Objects with sizes greater than the Old generation space will then be discarded instead of cached (stored) in the space and helping it to be "almost full".

This will decrease the number of full garbage collection runs, increasing overall performance.

Inevitably, however, objects will need to be recomputed when they are needed. So, this mechanism only works when a Spark application needs to reuse cached data as little as possible.

Garbage collection information can be accessed in the Spark UI's stage detail view.

This statement is correct. The task table in the Spark UI's stage detail view has a "GC Time" column, indicating the garbage collection time needed per task.

In Spark, using the G1 garbage collector is an alternative to using the default Parallel garbage collector.

This statement is correct. The G1 garbage collector, also known as garbage first garbage collector, is an alternative to the default Parallel garbage collector.

While the default Parallel garbage collector divides the heap into a few static regions, the G1 garbage collector divides the heap into many small regions that are created dynamically. The G1 garbage collector has certain advantages over the Parallel garbage collector which improve performance particularly for Spark workloads that require high throughput and low latency.

The G1 garbage collector is not enabled by default, and you need to explicitly pass an argument to Spark to enable it. For more information about the two garbage collectors, check out the Databricks article linked below.

NO.30 Which of the following options describes the responsibility of the executors in Spark?

- A.** The executors accept jobs from the driver, analyze those jobs, and return results to the driver.
- B.** The executors accept tasks from the driver, execute those tasks, and return results to the cluster manager.
- C.** The executors accept tasks from the driver, execute those tasks, and return results to the driver.
- D.** The executors accept tasks from the cluster manager, execute those tasks, and return results to the driver.
- E.** The executors accept jobs from the driver, plan those jobs, and return results to the cluster

manager.

Answer: C

Explanation

More info: Running Spark: an overview of Spark's runtime architecture - Manning
(<https://bit.ly/2RPmJn9>)

NO.31 Which of the following code blocks returns a single-column DataFrame of all entries in Python list throughputRates which contains only float-type values ?

- A. `spark.createDataFrame((throughputRates), FloatType)`
- B. `spark.createDataFrame(throughputRates, FloatType)`
- C. `spark.DataFrame(throughputRates, FloatType)`
- D. `spark.createDataFrame(throughputRates)`
- E. `spark.createDataFrame(throughputRates, FloatType())`

Answer: E

Explanation

`spark.createDataFrame(throughputRates, FloatType())`

Correct! `spark.createDataFrame` is the correct operator to use here and the type `FloatType()` which is passed in for the command's schema argument is correctly instantiated using the parentheses.

Remember that it is essential in PySpark to instantiate types when passing them to `SparkSession.createDataFrame`. And, in Databricks, `spark` returns a `SparkSession` object.

`spark.createDataFrame((throughputRates), FloatType)`

No. While packing `throughputRates` in parentheses does not do anything to the execution of this command, not instantiating the `FloatType` with parentheses as in the previous answer will make this command fail.

`spark.createDataFrame(throughputRates, FloatType)`

Incorrect. Given that it does not matter whether you pass `throughputRates` in parentheses or not, see the explanation of the previous answer for further insights.

`spark.DataFrame(throughputRates, FloatType)`

Wrong. There is no `SparkSession.DataFrame()` method in Spark.

`spark.createDataFrame(throughputRates)`

False. Avoiding the schema argument will have PySpark try to infer the schema. However, as you can see in the documentation (linked below), the inference will only work if you pass in an "RDD of either Row, namedtuple, or dict" for data (the first argument to `createDataFrame`). But since you are passing a Python list, Spark's schema inference will fail.

More info: `pyspark.sql.Session.createDataFrame` - PySpark 3.1.2 documentation Static notebook
| Dynamic notebook: See test 3

NO.32 Which of the following code blocks returns a one-column DataFrame for which every row contains an array of all integer numbers from 0 up to and including the number given in column `predError` of DataFrame `transactionsDf`, and null if `predError` is null?

Sample of DataFrame `transactionsDf`:

1. +-----+-----+-----+-----+-----+-----+
2. | transactionId | predError | value | storeId | productId | f |
3. +-----+-----+-----+-----+-----+-----+
4. | 1 | 3 | 4 | 25 | 1 | null |

5. | 2 | 6 | 7 | 2 | 2 | null |
 6. | 3 | 3 | null | 25 | 3 | null |
 7. | 4 | null | null | 3 | 2 | null |
 8. | 5 | null | null | null | 2 | null |
 9. | 6 | 3 | 2 | 25 | 2 | null |
 10. +-----+-----+-----+-----+-----+

A. 1. def count_to_target(target):

2. if target is None:

3. return

4.

5. result = [range(target)]

6. return result

7.

8. count_to_target_udf = udf(count_to_target, ArrayType(IntegerType))

9.

10. transactionsDf.select(count_to_target_udf(col('predError')))

B. 1. def count_to_target(target):

2. if target is None:

3. return

4.

5. result = list(range(target))

6. return result

7.

8. transactionsDf.select(count_to_target(col('predError')))

C. 1. def count_to_target(target):

2. if target is None:

3. return

4.

5. result = list(range(target))

6. return result

7.

8. count_to_target_udf = udf(count_to_target, ArrayType(IntegerType()))

9.

10. transactionsDf.select(count_to_target_udf('predError'))

(Correct)

D. 1. def count_to_target(target):

2. result = list(range(target))

3. return result

4.

5. count_to_target_udf = udf(count_to_target, ArrayType(IntegerType()))

6.

7. df = transactionsDf.select(count_to_target_udf('predError'))

E. 1. def count_to_target(target):

2. if target is None:

3. return

4.

```

5. result = list(range(target))
6. return result
7.
8.count_to_target_udf = udf(count_to_target)
9.
10.transactionsDf.select(count_to_target_udf('predError'))

```

Answer: C

Explanation

Correct code block:

```

def count_to_target(target):
    if target is None:
        return
    result = list(range(target))
    return result
count_to_target_udf = udf(count_to_target, ArrayType(IntegerType()))
transactionsDf.select(count_to_target_udf('predError'))

```

Output of correct code block:

```

+-----+
|count_to_target(predError)|
+-----+
| [0, 1, 2]|
| [0, 1, 2, 3, 4, 5]|
| [0, 1, 2]|
| null|
| null|
| [0, 1, 2]|
+-----+

```

This question is not exactly easy. You need to be familiar with the syntax around UDFs (user-defined functions). Specifically, in this question it is important to pass the correct types to the udf method - returning an array of a specific type rather than just a single type means you need to think harder about type implications than usual.

Remember that in Spark, you always pass types in an instantiated way like `ArrayType(IntegerType())`, not like `ArrayType(IntegerType)`. The parentheses () are the key here - make sure you do not forget those.

You should also pay attention that you actually pass the UDF `count_to_target_udf`, and not the Python method `count_to_target` to the `select()` operator.

Finally, null values are always a tricky case with UDFs. So, take care that the code can handle them correctly.

More info: [How to Turn Python Functions into PySpark Functions \(UDF\)](#) - Chang Hsin Lee - Committing my thoughts to words.

Static notebook | Dynamic notebook: See test 3

NO.33 Which of the following is not a feature of Adaptive Query Execution?

- A.** Replace a sort merge join with a broadcast join, where appropriate.
- B.** Coalesce partitions to accelerate data processing.
- C.** Split skewed partitions into smaller partitions to avoid differences in partition processing time.

- D.** Reroute a query in case of an executor failure.
- E.** Collect runtime statistics during query execution.

Answer: D

Explanation

Reroute a query in case of an executor failure.

Correct. Although this feature exists in Spark, it is not a feature of Adaptive Query Execution. The cluster manager keeps track of executors and will work together with the driver to launch an executor and assign the workload of the failed executor to it (see also link below).

Replace a sort merge join with a broadcast join, where appropriate.

No, this is a feature of Adaptive Query Execution.

Coalesce partitions to accelerate data processing.

Wrong, Adaptive Query Execution does this.

Collect runtime statistics during query execution.

Incorrect, Adaptive Query Execution (AQE) collects these statistics to adjust query plans. This feedback loop is an essential part of accelerating queries via AQE.

Split skewed partitions into smaller partitions to avoid differences in partition processing time.

No, this is indeed a feature of Adaptive Query Execution. Find more information in the Databricks blog post linked below.

More info: Learning Spark, 2nd Edition, Chapter 12, On which way does RDD of spark finish fault-tolerance?

- Stack Overflow, How to Speed up SQL Queries with Adaptive Query Execution

NO.34 The code block displayed below contains an error. The code block is intended to join DataFrame itemsDf with the larger DataFrame transactionsDf on column itemId. Find the error.
Code block:

```
transactionsDf.join(itemsDf, "itemId", how="broadcast")
```

- A.** The syntax is wrong, how= should be removed from the code block.
- B.** The join method should be replaced by the broadcast method.
- C.** Spark will only perform the broadcast operation if this behavior has been enabled on the Spark cluster.
- D.** The larger DataFrame transactionsDf is being broadcasted, rather than the smaller DataFrame itemsDf.
- E.** broadcast is not a valid join type.

Answer: E

Explanation

broadcast is not a valid join type.

Correct! The code block should read `transactionsDf.join(broadcast(itemsDf), "itemId")`. This would imply an inner join (this is the default in `DataFrame.join()`), but since the join type is not given in the question, this would be a valid choice.

The larger DataFrame transactionsDf is being broadcasted, rather than the smaller DataFrame itemsDf.

This option does not apply here, since the syntax around broadcasting is incorrect.

Spark will only perform the broadcast operation if this behavior has been enabled on the Spark cluster.

No, it is enabled by default, since the `spark.sql.autoBroadcastJoinThreshold` property is set to 10 MB

by default. If that property would be set to -1, then broadcast joining would be disabled.

More info: Performance Tuning - Spark 3.1.1 Documentation (<https://bit.ly/3gCz34r>) The join method should be replaced by the broadcast method.

No, DataFrame has no broadcast() method.

The syntax is wrong, how= should be removed from the code block.

No, having the keyword argument how= is totally acceptable.

NO.35 The code block shown below should return a one-column DataFrame where the column storeId is converted to string type. Choose the answer that correctly fills the blanks in the code block to accomplish this.

transactionsDf.__1__(__2__.__3__(__4__))

A. 1. select

2. col("storeId")

3. cast

4. StringType

B. 1. select

2. col("storeId")

3. as

4. StringType

C. 1. cast

2. "storeId"

3. as

4. StringType()

D. 1. select

2. col("storeId")

3. cast

4. StringType()

E. 1. select

2. storeId

3. cast

4. StringType()

Answer: D

Explanation

Correct code block:

```
transactionsDf.select(col("storeId").cast(StringType()))
```

Solving this question involves understanding that, when using types from the pyspark.sql.types such as StringType, these types need to be instantiated when using them in Spark, or, in simple words, they need to be followed by parentheses like so: StringType(). You could also use .cast("string") instead, but that option is not given here.

More info: pyspark.sql.Column.cast - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 2

NO.36 Which of the following code blocks returns the number of unique values in column storeId of DataFrame transactionsDf?

A. transactionsDf.select("storeId").dropDuplicates().count()

- B. `transactionsDf.select(count("storeId")).dropDuplicates()`
- C. `transactionsDf.select(distinct("storeId")).count()`
- D. `transactionsDf.dropDuplicates().agg(count("storeId"))`
- E. `transactionsDf.distinct().select("storeId").count()`

Answer: A

Explanation

`transactionsDf.select("storeId").dropDuplicates().count()`

Correct! After dropping all duplicates from column `storeId`, the remaining rows get counted, representing the number of unique values in the column.

`transactionsDf.select(count("storeId")).dropDuplicates()`

No. `transactionsDf.select(count("storeId"))` just returns a single-row DataFrame showing the number of non-null rows. `dropDuplicates()` does not have any effect in this context.

`transactionsDf.dropDuplicates().agg(count("storeId"))`

Incorrect. While `transactionsDf.dropDuplicates()` removes duplicate rows from `transactionsDf`, it does not do so taking only column `storeId` into consideration, but eliminates full row duplicates instead.

`transactionsDf.distinct().select("storeId").count()`

Wrong. `transactionsDf.distinct()` identifies unique rows across all columns, but not only unique rows with respect to column `storeId`. This may leave duplicate values in the column, making the count not represent the number of unique values in that column.

`transactionsDf.select(distinct("storeId")).count()`

False. There is no `distinct` method in `pyspark.sql.functions`.

NO.37 Which of the following statements about data skew is incorrect?

- A. Spark will not automatically optimize skew joins by default.
- B. Broadcast joins are a viable way to increase join performance for skewed data over sort-merge joins.
- C. In skewed DataFrames, the largest and the smallest partition consume very different amounts of memory.
- D. To mitigate skew, Spark automatically disregards null values in keys when joining.
- E. Salting can resolve data skew.

Answer: D

Explanation

To mitigate skew, Spark automatically disregards null values in keys when joining.

This statement is incorrect, and thus the correct answer to the question. Joining keys that contain null values is of particular concern with regard to data skew.

In real-world applications, a table may contain a great number of records that do not have a value assigned to the column used as a join key. During the join, the data is at risk of being heavily skewed. This is because all records with a null-value join key are then evaluated as a single large partition, standing in stark contrast to the potentially diverse key values (and therefore small partitions) of the non-null-key records.

Spark specifically does not handle this automatically. However, there are several strategies to mitigate this problem like discarding null values temporarily, only to merge them back later (see last link below).

In skewed DataFrames, the largest and the smallest partition consume very different amounts of memory.

This statement is correct. In fact, having very different partition sizes is the very definition of skew. Skew can degrade Spark performance because the largest partition occupies a single executor for a long time. This blocks a Spark job and is an inefficient use of resources, since other executors that processed smaller partitions need to idle until the large partition is processed.

Salting can resolve data skew.

This statement is correct. The purpose of salting is to provide Spark with an opportunity to repartition data into partitions of similar size, based on a salted partitioning key.

A salted partitioning key typically is a column that consists of uniformly distributed random numbers. The number of unique entries in the partitioning key column should match the number of your desired number of partitions. After repartitioning by the salted key, all partitions should have roughly the same size.

Spark does not automatically optimize skew joins by default.

This statement is correct. Automatic skew join optimization is a feature of Adaptive Query Execution (AQE).

By default, AQE is disabled in Spark. To enable it, Spark's `spark.sql.adaptive.enabled` configuration option needs to be set to true instead of leaving it at the default false.

To automatically optimize skew joins, Spark's `spark.sql.adaptive.skewJoin.enabled` options also needs to be set to true, which it is by default.

When skew join optimization is enabled, Spark recognizes skew joins and optimizes them by splitting the bigger partitions into smaller partitions which leads to performance increases.

Broadcast joins are a viable way to increase join performance for skewed data over sort-merge joins.

This statement is correct. Broadcast joins can indeed help increase join performance for skewed data, under some conditions. One of the DataFrames to be joined needs to be small enough to fit into each executor's memory, along a partition from the other DataFrame. If this is the case, a broadcast join increases join performance over a sort-merge join.

The reason is that a sort-merge join with skewed data involves excessive shuffling. During shuffling, data is sent around the cluster, ultimately slowing down the Spark application. For skewed data, the amount of data, and thus the slowdown, is particularly big.

Broadcast joins, however, help reduce shuffling data. The smaller table is directly stored on all executors, eliminating a great amount of network traffic, ultimately increasing join performance relative to the sort-merge join.

It is worth noting that for optimizing skew join behavior it may make sense to manually adjust Spark's `spark.sql.autoBroadcastJoinThreshold` configuration property if the smaller DataFrame is bigger than the 10 MB set by default.

More info:

- Performance Tuning - Spark 3.0.0 Documentation
- Data Skew and Garbage Collection to Improve Spark Performance
- Section 1.2 - Joins on Skewed Data * GitBook

NO.38 The code block displayed below contains an error. The code block should return a copy of DataFrame `transactionsDf` where the name of column `transactionId` has been changed to `transactionNumber`. Find the error.

Code block:

```
transactionsDf.withColumn("transactionNumber", "transactionId")
```

A. The arguments to the `withColumn` method need to be reordered.

B. The arguments to the `withColumn` method need to be reordered and the `copy()` operator should

be appended to the code block to ensure a copy is returned.

C. The copy() operator should be appended to the code block to ensure a copy is returned.

D. Each column name needs to be wrapped in the col() method and method withColumn should be replaced by method withColumnRenamed.

E. The method withColumn should be replaced by method withColumnRenamed and the arguments to the method need to be reordered.

Answer: E

Explanation

Correct code block:

```
transactionsDf.withColumnRenamed("transactionId", "transactionNumber")
```

Note that in Spark, a copy is returned by default. So, there is no need to append copy() to the code block.

More info: [pyspark.sql.DataFrame.withColumnRenamed](#) - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 2

NO.39 Which of the following code blocks returns a DataFrame with a single column in which all items in column attributes of DataFrame itemsDf are listed that contain the letter i?

Sample of DataFrame itemsDf:

```
1. +-----+-----+-----+-----+
2. |itemId|itemName |attributes |supplier |
3. +-----+-----+-----+-----+
4. |1 |Thick Coat for Walking in the Snow|[blue, winter, cozy] |Sports Company Inc. |
5. |2 |Elegant Outdoors Summer Dress |[red, summer, fresh, cooling]|YetiX |
6. |3 |Outdoors Backpack |[green, summer, travel] |Sports Company Inc. |
7. +-----+-----+-----+-----+
```

A.

```
itemsDf.select(explode("attributes").alias("attributes_exploded")).filter(attributes_exploded.contains("i"))
```

B.

```
itemsDf.explode(attributes).alias("attributes_exploded").filter(col("attributes_exploded").contains("i"))
```

C. itemsDf.select(explode("attributes")).filter("attributes_exploded".contains("i"))

D.

```
itemsDf.select(explode("attributes").alias("attributes_exploded")).filter(col("attributes_exploded").contains("i"))
```

E.

```
itemsDf.select(col("attributes").explode().alias("attributes_exploded")).filter(col("attributes_exploded").contains("i"))
```

Answer: D

Explanation

Result of correct code block:

```
+-----+
|attributes_exploded|
+-----+
| winter |
```

```
| cooling|
```

```
+-----+
```

To solve this question, you need to know about `explode()`. This operation helps you to split up arrays into single rows. If you did not have a chance to familiarize yourself with this method yet, find more examples in the documentation (link below).

Note that `explode()` is a method made available through `pyspark.sql.functions` - it is not available as a method of a `DataFrame` or a `Column`, as written in some of the answer options.

More info: `pyspark.sql.functions.explode` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 2

NO.40 The code block shown below should return the number of columns in the CSV file stored at location `filePath`.

From the CSV file, only lines should be read that do not start with a `#` character. Choose the answer that correctly fills the blanks in the code block to accomplish this.

Code block:

```
__1__(__2__.__3__.csv(filePath, __4__).__5__)
```

A. 1. size

2. spark

3. read()

4. escape='#'

5. columns

B. 1. DataFrame

2. spark

3. read()

4. escape='#'

5. shape[0]

C. 1. len

2. pyspark

3. DataFrameReader

4. comment='#'

5. columns

D. 1. size

2. pyspark

3. DataFrameReader

4. comment='#'

5. columns

E. 1. len

2. spark

3. read

4. comment='#'

5. columns

Answer: E

Explanation

Correct code block:

```
len(spark.read.csv(filePath, comment='#').columns)
```

This is a challenging question with difficulties in an unusual context: The boundary between

DataFrame and the DataFrameReader. It is unlikely that a question of this difficulty level appears in the exam. However, solving it helps you get more comfortable with the DataFrameReader, a subject you will likely have to deal with in the exam.

Before dealing with the inner parentheses, it is easier to figure out the outer parentheses, gaps 1 and 5. Given the code block, the object in gap 5 would have to be evaluated by the object in gap 1, returning the number of columns in the read-in CSV. One answer option includes DataFrame in gap 1 and shape[0] in gap 2. Since DataFrame cannot be used to evaluate shape[0], we can discard this answer option.

Other answer options include size in gap 1. size() is not a built-in Python command, so if we use it, it would have to come from somewhere else. pyspark.sql.functions includes a size() method, but this method only returns the length of an array or map stored within a column (documentation linked below).

So, using a size() method is not an option here. This leaves us with two potentially valid answers. We have to pick between gaps 2 and 3 being spark.read or pyspark.DataFrameReader. Looking at the documentation (linked below), the DataFrameReader is actually a child class of pyspark.sql, which means that we cannot import it using pyspark.DataFrameReader. Moreover, spark.read makes sense because on Databricks, spark references current Spark session (pyspark.sql.Session) and spark.read therefore returns a DataFrameReader (also see documentation below). Finally, there is only one correct answer option remaining.

More info:

- pyspark.sql.functions.size - PySpark 3.1.2 documentation
- pyspark.sql.DataFrameReader.csv - PySpark 3.1.2 documentation
- pyspark.sql.Session.read - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

NO.41 Which of the following code blocks prints out in how many rows the expression Inc. appears in the string-type column supplier of DataFrame itemsDf?

A. 1.counter = 0

2.

3.for index, row in itemsDf.iterrows():

4. if 'Inc.' in row['supplier']:

5. counter = counter + 1

6.

7.print(counter)

B. 1.counter = 0

2.

3.def count(x):

4. if 'Inc.' in x['supplier']:

5. counter = counter + 1

6.

7.itemsDf.foreach(count)

8.print(counter)

C. print(itemsDf.foreach(lambda x: 'Inc.' in x))

D. print(itemsDf.foreach(lambda x: 'Inc.' in x).sum())

E. 1.accum=sc.accumulator(0)

2.

```

3.def check_if_inc_in_supplier(row):
4. if 'Inc.' in row['supplier']:
5. accum.add(1)
6.
7.itemsDf.foreach(check_if_inc_in_supplier)
8.print(accum.value)

```

Answer: E

Explanation

Correct code block:

```

accum=sc.accumulator(0)
def check_if_inc_in_supplier(row):
if 'Inc.' in row['supplier']:
accum.add(1)
itemsDf.foreach(check_if_inc_in_supplier)
print(accum.value)

```

To answer this question correctly, you need to know both about the DataFrame.foreach() method and accumulators.

When Spark runs the code, it executes it on the executors. The executors do not have any information about variables outside of their scope. This is why simply using a Python variable counter, like in the two examples that start with counter = 0, will not work. You need to tell the executors explicitly that counter is a special shared variable, an Accumulator, which is managed by the driver and can be accessed by all executors for the purpose of adding to it.

If you have used Pandas in the past, you might be familiar with the iterrows() command. Notice that there is no such command in PySpark.

The two examples that start with print do not work, since DataFrame.foreach() does not have a return value.

More info: [pyspark.sql.DataFrame.foreach - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 3

NO.42 Which of the following code blocks returns a copy of DataFrame transactionsDf where the column storeId has been converted to string type?

- A.** transactionsDf.withColumn("storeId", convert("storeId", "string"))
- B.** transactionsDf.withColumn("storeId", col("storeId", "string"))
- C.** transactionsDf.withColumn("storeId", col("storeId").convert("string"))
- D.** transactionsDf.withColumn("storeId", col("storeId").cast("string"))
- E.** transactionsDf.withColumn("storeId", convert("storeId").as("string"))

Answer: D

Explanation

This question asks for your knowledge about the cast syntax. cast is a method of the Column class. It is worth noting that one could also convert a column type using the Column.astype() method, which is just an alias for cast.

Find more info in the documentation linked below.

More info: [pyspark.sql.Column.cast - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 2

NO.43 Which of the following code blocks returns a single row from DataFrame transactionsDf?

Full DataFrame transactionsDf:

```

1. +-----+-----+----+-----+-----+----+
2. |transactionId|predError|value|storeId|productId| f|
3. +-----+-----+----+-----+-----+----+
4. | 1| 3| 4| 25| 1|null|
5. | 2| 6| 7| 2| 2|null|
6. | 3| 3| null| 25| 3|null|
7. | 4| null| null| 3| 2|null|
8. | 5| null| null| null| 2|null|
9. | 6| 3| 2| 25| 2|null|
10. +-----+-----+----+-----+-----+----+

```

- A.** transactionsDf.where(col("storeId").between(3,25))
- B.** transactionsDf.filter((col("storeId")!=25) | (col("productId")==2))
- C.** transactionsDf.filter(col("storeId")==25).select("predError","storeId").distinct()
- D.** transactionsDf.select("productId", "storeId").where("storeId == 2 OR storeId != 25")
- E.** transactionsDf.where(col("value").isNull()).select("productId", "storeId").distinct()

Answer: C

Explanation

Output of correct code block:

```

+-----+-----+
|predError|storeId|
+-----+-----+
| 3| 25|
+-----+-----+

```

This question is difficult because it requires you to understand different kinds of commands and operators. All answers are valid Spark syntax, but just one expression returns a single-row DataFrame.

For reference, here is what the incorrect answers return:

transactionsDf.filter((col("storeId")!=25) | (col("productId")==2)) returns

```

+-----+-----+----+-----+-----+----+
|transactionId|predError|value|storeId|productId| f|
+-----+-----+----+-----+-----+----+
| 2| 6| 7| 2| 2|null|
| 4| null| null| 3| 2|null|
| 5| null| null| null| 2|null|
| 6| 3| 2| 25| 2|null|

```

transactionsDf.where(col("storeId").between(3,25)) returns

```

+-----+-----+----+-----+-----+----+
|transactionId|predError|value|storeId|productId| f|
+-----+-----+----+-----+-----+----+
| 1| 3| 4| 25| 1|null|
| 3| 3| null| 25| 3|null|
| 4| null| null| 3| 2|null|
| 6| 3| 2| 25| 2|null|

```

transactionsDf.where(col("value").isNull()).select("productId", "storeId").distinct() returns

```
+-----+-----+
|productId|storeId|
+-----+-----+
| 3| 25|
| 2| 3|
| 2| null|
+-----+-----+
```

transactionsDf.select("productId", "storeId").where("storeId == 2 OR storeId != 25") returns

```
+-----+-----+
|productId|storeId|
+-----+-----+
| 2| 2|
| 2| 3|
+-----+-----+
```

Static notebook | Dynamic notebook: See test 2

NO.44 The code block shown below should write DataFrame transactionsDf as a parquet file to path storeDir, using brotli compression and replacing any previously existing file. Choose the answer that correctly fills the blanks in the code block to accomplish this.

transactionsDf.__1__.format("parquet").__2__(__3__).option(__4__, "brotli").__5__(storeDir)

- A.** 1. save
2. mode
3. "ignore"
4. "compression"
5. path

- B.** 1. store
2. with
3. "replacement"
4. "compression"
5. path

- C.** 1. write
2. mode
3. "overwrite"
4. "compression"
5. save

(Correct)

- D.** 1. save
2. mode
3. "replace"
4. "compression"
5. path

- E.** 1. write
2. mode
3. "overwrite"
4. compression

5. parquet

Answer: D

Explanation

Correct code block:

```
transactionsDf.write.format("parquet").mode("overwrite").option("compression",
"snappy").save(storeDir)
```

Solving this question requires you to know how to access the `DataFrameWriter` (link below) from the `DataFrame` API - through `DataFrame.write`.

Another nuance here is about knowing the different modes available for writing parquet files that determine Spark's behavior when dealing with existing files. These, together with the compression options are explained in the `DataFrameWriter.parquet` documentation linked below.

Finally, bracket `__5__` poses a certain challenge. You need to know which command you can use to pass down the file path to the `DataFrameWriter`. Both `save` and `parquet` are valid options here.

More info:

- `DataFrame.write`: `pyspark.sql.DataFrame.write` - PySpark 3.1.1 documentation
 - `DataFrameWriter.parquet`: `pyspark.sql.DataFrameWriter.parquet` - PySpark 3.1.1 documentation
- n Static notebook | Dynamic notebook: See test 1

NO.45 Which of the following code blocks displays various aggregated statistics of all columns in `DataFrame transactionsDf`, including the standard deviation and minimum of values in each column?

- A.** `transactionsDf.summary()`
- B.** `transactionsDf.agg("count", "mean", "stddev", "25%", "50%", "75%", "min")`
- C.** `transactionsDf.summary("count", "mean", "stddev", "25%", "50%", "75%", "max").show()`
- D.** `transactionsDf.agg("count", "mean", "stddev", "25%", "50%", "75%", "min").show()`
- E.** `transactionsDf.summary().show()`

Answer: E

Explanation

The `DataFrame.summary()` command is very practical for quickly calculating statistics of a `DataFrame`. You need to call `.show()` to display the results of the calculation. By default, the command calculates various statistics (see documentation linked below), including standard deviation and minimum. Note that the answer that lists many options in the `summary()` parentheses does not include the minimum, which is asked for in the question.

Answer options that include `agg()` do not work here as shown, since `DataFrame.agg()` expects more complex, column-specific instructions on how to aggregate values.

More info:

- `pyspark.sql.DataFrame.summary` - PySpark 3.1.2 documentation
- `pyspark.sql.DataFrame.agg` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

NO.46 Which of the following code blocks returns a single-column `DataFrame` showing the number of words in column `supplier` of `DataFrame itemsDf`?

Sample of `DataFrame itemsDf`:

- ```
1. +-----+-----+-----+
2. |itemId|attributes|supplier|
3. +-----+-----+-----+
4. |1|[blue, winter, cozy]|Sports Company Inc.|
5. |2|[red, summer, fresh, cooling]|YetiX|
```

6. | 3 | [green, summer, travel] | Sports Company Inc. |

7. +-----+-----+-----+-----+

- A. itemsDf.split("supplier", " ").count()
- B. itemsDf.split("supplier", " ").size()
- C. itemsDf.select(word\_count("supplier"))
- D. spark.select(size(split(col(supplier), " ")))
- E. itemsDf.select(size(split("supplier", " ")))

**Answer:** E

Explanation

Output of correct code block:

```
+-----+
|size(split(supplier, , -1))|
+-----+
| 3|
| 1|
| 3|
+-----+
```

This question shows a typical use case for the split command: Splitting a string into words. An additional difficulty is that you are asked to count the words. Although it is tempting to use the count method here, the size method (as in: size of an array) is actually the correct one to use. Familiarize yourself with the split and the size methods using the linked documentation below.

More info:

Split method: [pyspark.sql.functions.split - PySpark 3.1.2 documentation](#) Size method:

[pyspark.sql.functions.size - PySpark 3.1.2 documentation](#) Static notebook | Dynamic notebook: See test 2

**NO.47** Which of the following code blocks returns only rows from DataFrame transactionsDf in which values in column productId are unique?

- A. transactionsDf.distinct("productId")
- B. transactionsDf.dropDuplicates(subset=["productId"])
- C. transactionsDf.drop\_duplicates(subset="productId")
- D. transactionsDf.unique("productId")
- E. transactionsDf.dropDuplicates(subset="productId")

**Answer:** B

Explanation

Although the question suggests using a method called unique() here, that method does not actually exist in PySpark. In PySpark, it is called distinct(). But then, this method is not the right one to use here, since with distinct() we could filter out unique values in a specific column.

However, we want to return the entire rows here. So the trick is to use dropDuplicates with the subset keyword parameter. In the documentation for dropDuplicates, the examples show that subset should be used with a list. And this is exactly the key to solving this question: The productId column needs to be fed into the subset argument in a list, even though it is just a single column.

More info: [pyspark.sql.DataFrame.dropDuplicates - PySpark 3.1.1 documentation](#) Static notebook | Dynamic notebook: See test 1

**NO.48** Which of the following code blocks returns a 2-column DataFrame that shows the distinct values in column productId and the number of rows with that productId in DataFrame transactionsDf?

- A. transactionsDf.count("productId").distinct()
- B. transactionsDf.groupBy("productId").agg(col("value").count())
- C. transactionsDf.count("productId")
- D. transactionsDf.groupBy("productId").count()
- E. transactionsDf.groupBy("productId").select(count("value"))

**Answer:** D

Explanation

transactionsDf.groupBy("productId").count()

Correct. This code block first groups DataFrame transactionsDf by column productId and then counts the rows in each group.

transactionsDf.groupBy("productId").select(count("value"))

Incorrect. You cannot call select on a GroupedData object (the output of a groupBy) statement.

transactionsDf.count("productId")

No. DataFrame.count() does not take any arguments.

transactionsDf.count("productId").distinct()

Wrong. Since DataFrame.count() does not take any arguments, this option cannot be right.

transactionsDf.groupBy("productId").agg(col("value").count())

False. A Column object, as returned by col("value"), does not have a count() method. You can see all available methods for Column object linked in the Spark documentation below.

More info: [pyspark.sql.DataFrame.count - PySpark 3.1.2 documentation](#), [pyspark.sql.Column - PySpark](#)

3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.49** Which of the following is the idea behind dynamic partition pruning in Spark?

- A. Dynamic partition pruning is intended to skip over the data you do not need in the results of a query.
- B. Dynamic partition pruning concatenates columns of similar data types to optimize join performance.
- C. Dynamic partition pruning performs wide transformations on disk instead of in memory.
- D. Dynamic partition pruning reoptimizes physical plans based on data types and broadcast variables.
- E. Dynamic partition pruning reoptimizes query plans based on runtime statistics collected during query execution.

**Answer:** A

Explanation

Dynamic partition pruning reoptimizes query plans based on runtime statistics collected during query execution.

No - this is what adaptive query execution does, but not dynamic partition pruning.

Dynamic partition pruning concatenates columns of similar data types to optimize join performance.

Wrong, this answer does not make sense, especially related to dynamic partition pruning.

Dynamic partition pruning reoptimizes physical plans based on data types and broadcast variables.

It is true that dynamic partition pruning works in joins using broadcast variables. This actually

happens in both the logical optimization and the physical planning stage. However, data types do not play a role for the reoptimization.

Dynamic partition pruning performs wide transformations on disk instead of in memory.

This answer does not make sense. Dynamic partition pruning is meant to accelerate Spark - performing any transformation involving disk instead of memory resources would decelerate Spark and certainly achieve the opposite effect of what dynamic partition pruning is intended for.

**NO.50** The code block shown below should show information about the data type that column storeId of DataFrame transactionsDf contains. Choose the answer that correctly fills the blanks in the code block to accomplish this.

Code block:

```
transactionsDf.__1__(__2__).__3__
```

**A.** 1. select  
2. "storeId"  
3. print\_schema()

**B.** 1. limit  
2. 1

3. columns

**C.** 1. select  
2. "storeId"  
3. printSchema()

**D.** 1. limit  
2. "storeId"  
3. printSchema()

**E.** 1. select  
2. storeId  
3. dtypes

**Answer:** B

Explanation

Correct code block:

```
transactionsDf.select("storeId").printSchema()
```

The difficulty of this question is that it is hard to solve with the stepwise first-to-last-gap approach that has worked well for similar questions, since the answer options are so different from one another. Instead, you might want to eliminate answers by looking for patterns of frequently wrong answers.

A first pattern that you may recognize by now is that column names are not expressed in quotes. For this reason, the answer that includes storeId should be eliminated.

By now, you may have understood that the DataFrame.limit() is useful for returning a specified amount of rows. It has nothing to do with specific columns. For this reason, the answer that resolves to limit("storeId") can be eliminated.

Given that we are interested in information about the data type, you should question whether the answer that resolves to limit(1).columns provides you with this information. While

DataFrame.columns is a valid call, it will only report back column names, but not column types. So, you can eliminate this option.

The two remaining options either use the printSchema() or print\_schema() command. You may remember that DataFrame.printSchema() is the only valid command of the two. The select("storeId")

part just returns the storeId column of transactionsDf - this works here, since we are only interested in that column's type anyways.

More info: `pyspark.sql.DataFrame.printSchema` - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 3

**NO.51** The code block shown below should return a DataFrame with all columns of DataFrame transactionsDf, but only maximum 2 rows in which column productId has at least the value 2. Choose the answer that correctly fills the blanks in the code block to accomplish this.

transactionsDf.\_\_1\_\_(\_\_2\_\_).\_\_3\_\_

- A.** 1. where  
2. "productId" > 2  
3. max(2)
- B.** 1. where  
2. transactionsDf[productId] >= 2  
3. limit(2)
- C.** 1. filter  
2. productId > 2  
3. max(2)
- D.** 1. filter  
2. col("productId") >= 2  
3. limit(2)
- E.** 1. where  
2. productId >= 2  
3. limit(2)

**Answer:** D

Explanation

Correct code block:

```
transactionsDf.filter(col("productId") >= 2).limit(2)
```

The filter and where operators in gap 1 are just aliases of one another, so you cannot use them to pick the right answer.

The column definition in gap 2 is more helpful. The DataFrame.filter() method takes an argument of type Column or str. From all possible answers, only the one including col("productId") >= 2 fits this profile, since it returns a Column type.

The answer option using "productId" > 2 is invalid, since Spark does not understand that "productId" refers to column productId. The answer option using transactionsDf[productId] >= 2 is wrong because you cannot refer to a column using square bracket notation in Spark (if you are coming from Python using Pandas, this is something to watch out for). In all other options, productId is being referred to as a Python variable, so they are relatively easy to eliminate.

Also note that the question asks for the value in column productId being at least 2. This translates to a

"greater or equal" sign (>= 2), but not a "greater" sign (> 2).

Another thing worth noting is that there is no DataFrame.max() method. If you picked any option including this, you may be confusing it with the pyspark.sql.functions.max method. The correct method to limit the amount of rows is the DataFrame.limit() method.

More info:

- `pyspark.sql.DataFrame.filter` - PySpark 3.1.2 documentation

- `pyspark.sql.DataFrame.limit` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.52** Which of the following code blocks returns DataFrame `transactionsDf` sorted in descending order by column `predError`, showing missing values last?

- A. `transactionsDf.sort(asc_nulls_last("predError"))`
- B. `transactionsDf.orderBy("predError").desc_nulls_last()`
- C. `transactionsDf.sort("predError", ascending=False)`
- D. `transactionsDf.desc_nulls_last("predError")`
- E. `transactionsDf.orderBy("predError").asc_nulls_last()`

**Answer:** C

Explanation

`transactionsDf.sort("predError", ascending=False)`

Correct! When using `DataFrame.sort()` and setting `ascending=False`, the DataFrame will be sorted by the specified column in descending order, putting all missing values last. An alternative, although not listed as an answer here, would be `transactionsDf.sort(desc_nulls_last("predError"))`.

`transactionsDf.sort(asc_nulls_last("predError"))`

Incorrect. While this is valid syntax, the DataFrame will be sorted on column `predError` in ascending order and not in descending order, putting missing values last.

`transactionsDf.desc_nulls_last("predError")`

Wrong, this is invalid syntax. There is no method `DataFrame.desc_nulls_last()` in the Spark API. There is a Spark function `desc_nulls_last()` however (link see below).

`transactionsDf.orderBy("predError").desc_nulls_last()`

No. While `transactionsDf.orderBy("predError")` is correct syntax (although it sorts the DataFrame by column `predError` in ascending order) and returns a DataFrame, there is no method `DataFrame.desc_nulls_last()` in the Spark API. There is a Spark function `desc_nulls_last()` however (link see below).

`transactionsDf.orderBy("predError").asc_nulls_last()`

Incorrect. There is no method `DataFrame.asc_nulls_last()` in the Spark API (see above).

More info: `pyspark.sql.functions.desc_nulls_last` - PySpark 3.1.2 documentation and

`pyspark.sql.DataFrame.sort` - PySpark 3.1.2 documentation (<https://bit.ly/3g1Jtbl> ,

<https://bit.ly/2R90NCS>) Static notebook | Dynamic notebook: See test 1

([https://flrs.github.io/spark\\_practice\\_tests\\_code/#1/32.html](https://flrs.github.io/spark_practice_tests_code/#1/32.html) ,

[https://bit.ly/sparkpracticeexams\\_import\\_instructions](https://bit.ly/sparkpracticeexams_import_instructions))

**NO.53** The code block displayed below contains an error. The code block is intended to perform an outer join of DataFrames `transactionsDf` and `itemsDf` on columns `productId` and `itemId`, respectively. Find the error.

Code block:

`transactionsDf.join(itemsDf, [itemsDf.itemId, transactionsDf.productId], "outer")`

- A. The "outer" argument should be eliminated, since "outer" is the default join type.
- B. The join type needs to be appended to the join() operator, like `join().outer()` instead of listing it as the last argument inside the join() call.
- C. The term `[itemsDf.itemId, transactionsDf.productId]` should be replaced by `itemsDf.itemId == transactionsDf.productId`.

**D.** The term `[itemsDf.itemId, transactionsDf.productId]` should be replaced by `itemsDf.col("itemId") == transactionsDf.col("productId")`.

**E.** The "outer" argument should be eliminated from the call and join should be replaced by `joinOuter`.

**Answer:** C

Explanation

Correct code block:

`transactionsDf.join(itemsDf, itemsDf.itemId == transactionsDf.productId, "outer")` Static notebook | Dynamic notebook: See test 1 ([https://flrs.github.io/spark\\_practice\\_tests\\_code/#1/33.html](https://flrs.github.io/spark_practice_tests_code/#1/33.html) , [https://bit.ly/sparkpracticeexams\\_import\\_instructions](https://bit.ly/sparkpracticeexams_import_instructions))

**NO.54** Which of the following code blocks immediately removes the previously cached DataFrame `transactionsDf` from memory and disk?

**A.** `array_remove(transactionsDf, "*")`

**B.** `transactionsDf.unpersist()`

(Correct)

**C.** `del transactionsDf`

**D.** `transactionsDf.clearCache()`

**E.** `transactionsDf.persist()`

**Answer:** B

Explanation

`transactionsDf.unpersist()`

Correct. The `DataFrame.unpersist()` command does exactly what the question asks for - it removes all cached parts of the DataFrame from memory and disk.

`del transactionsDf`

False. While this option can help remove the DataFrame from memory and disk, it does not do so immediately. The reason is that this command just notifies the Python garbage collector that the `transactionsDf` now may be deleted from memory. However, the garbage collector does not do so immediately and, if you wanted it to run immediately, would need to be specifically triggered to do so. Find more information linked below.

`array_remove(transactionsDf, "*")`

Incorrect. The `array_remove` method from `pyspark.sql.functions` is used for removing elements from arrays in columns that match a specific condition. Also, the first argument would be a column, and not a DataFrame as shown in the code block.

`transactionsDf.persist()`

No. This code block does exactly the opposite of what is asked for: It caches (writes) DataFrame `transactionsDf` to memory and disk. Note that even though you do not pass in a specific storage level here, Spark will use the default storage level (`MEMORY_AND_DISK`).

`transactionsDf.clearCache()`

Wrong. Spark's DataFrame does not have a `clearCache()` method.

More info: `pyspark.sql.DataFrame.unpersist` - PySpark 3.1.2 documentation, python - How to delete a n RDD in PySpark for the purpose of releasing resources? - Stack Overflow Static notebook | Dynamic notebook: See test 3

**NO.55** The code block shown below should return a single-column DataFrame with a column named `consonant_ct` that, for each row, shows the number of consonants in column `itemName` of

DataFrame itemsDf. Choose the answer that correctly fills the blanks in the code block to accomplish this.

DataFrame itemsDf:

```
1. +-----+-----+-----+-----+
2. |itemId|itemName |attributes |supplier |
3. +-----+-----+-----+-----+
4. |1 |Thick Coat for Walking in the Snow|[blue, winter, cozy] |Sports Company Inc. |
5. |2 |Elegant Outdoors Summer Dress |[red, summer, fresh, cooling]]YetiX |
6. |3 |Outdoors Backpack |[green, summer, travel] |Sports Company Inc. |
7. +-----+-----+-----+-----+ Code block:
itemsDf.select(__1__(__2__(__3__(__4__),"a|e|i|o|u|\s", "")).__5__("consonant_ct"))
```

- A.** 1. length  
2. regexp\_extract  
3. upper  
4. col("itemName")  
5. as
- B.** 1. size  
2. regexp\_replace  
3. lower  
4. "itemName"  
5. alias
- C.** 1. lower  
2. regexp\_replace  
3. length  
4. "itemName"  
5. alias
- D.** 1. length  
2. regexp\_replace  
3. lower  
4. col("itemName")  
5. alias
- E.** 1. size  
2. regexp\_extract  
3. lower  
4. col("itemName")  
5. alias

**Answer:** D

Explanation

Correct code block:

```
itemsDf.select(length(regexp_replace(lower(col("itemName")), "a|e|i|o|u|\s", ""))).alias("consonant_ct"))
```

Returned DataFrame:

```
+-----+
|consonant_ct|
+-----+
| 19|
```

| 16|

| 10|

+-----+

This question tries to make you think about the string functions Spark provides and in which order they should be applied. Arguably the most difficult part, the regular expression "a|e|i|o|u|\s", is not a numbered blank. However, if you are not familiar with the string functions, it may be a good idea to review those before the exam.

The size operator and the length operator can easily be confused. size works on arrays, while length works on strings. Luckily, this is something you can read up about in the documentation.

The code block works by first converting all uppercase letters in column itemName into lowercase (the lower() part). Then, it replaces all vowels by "nothing" - an empty character "" (the regexp\_replace() part). Now, only lowercase characters without spaces are included in the DataFrame. Then, per row, the length operator counts these remaining characters. Note that column itemName in itemsDf does not include any numbers or other characters, so we do not need to make any provisions for these. Finally, by using the alias() operator, we rename the resulting column to consonant\_ct.

More info:

- lower: pyspark.sql.functions.lower - PySpark 3.1.2 documentation
- regexp\_replace: pyspark.sql.functions.regexp\_replace - PySpark 3.1.2 documentation
- length: pyspark.sql.functions.length - PySpark 3.1.2 documentation
- alias: pyspark.sql.Column.alias - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 2

**NO.56** Which of the following is a characteristic of the cluster manager?

- A.** Each cluster manager works on a single partition of data.
- B.** The cluster manager receives input from the driver through the SparkContext.
- C.** The cluster manager does not exist in standalone mode.
- D.** The cluster manager transforms jobs into DAGs.
- E.** In client mode, the cluster manager runs on the edge node.

**Answer:** B

Explanation

The cluster manager receives input from the driver through the SparkContext.

Correct. In order for the driver to contact the cluster manager, the driver launches a SparkContext.

The driver then asks the cluster manager for resources to launch executors.

In client mode, the cluster manager runs on the edge node.

No. In client mode, the cluster manager is independent of the edge node and runs in the cluster.

The cluster manager does not exist in standalone mode.

Wrong, the cluster manager exists even in standalone mode. Remember, standalone mode is an easy means to deploy Spark across a whole cluster, with some limitations. For example, in standalone mode, no other frameworks can run in parallel with Spark. The cluster manager is part of Spark in standalone deployments however and helps launch and maintain resources across the cluster.

The cluster manager transforms jobs into DAGs.

No, transforming jobs into DAGs is the task of the Spark driver.

Each cluster manager works on a single partition of data.

No. Cluster managers do not work on partitions directly. Their job is to coordinate cluster resources so that they can be requested by and allocated to Spark drivers.

More info: Introduction to Core Spark Concepts \* BigData

**NO.57** The code block displayed below contains an error. The code block should return DataFrame transactionsDf, but with the column storeId renamed to storeNumber. Find the error.

Code block:

```
transactionsDf.withColumn("storeNumber", "storeId")
```

- A.** Instead of withColumn, the withColumnRenamed method should be used.
- B.** Arguments "storeNumber" and "storeId" each need to be wrapped in a col() operator.
- C.** Argument "storeId" should be the first and argument "storeNumber" should be the second argument to the withColumn method.
- D.** The withColumn operator should be replaced with the copyDataFrame operator.
- E.** Instead of withColumn, the withColumnRenamed method should be used and argument "storeId" should be the first and argument "storeNumber" should be the second argument to that method.

**Answer:** E

Explanation

Correct code block:

```
transactionsDf.withColumnRenamed("storeId", "storeNumber")
```

More info: [pyspark.sql.DataFrame.withColumnRenamed - PySpark 3.1.1 documentation](#) Static notebook | Dynamic notebook: See test 1

**NO.58** Which of the following code blocks performs an inner join of DataFrames transactionsDf and itemsDf on columns productId and itemId, respectively, excluding columns value and storeId from DataFrame transactionsDf and column attributes from DataFrame itemsDf?

- A.** transactionsDf.drop('value', 'storeId').join(itemsDf.select('attributes'), transactionsDf.productId==itemsDf.itemId)
- B.** 1.transactionsDf.createOrReplaceTempView('transactionsDf')  
2.itemsDf.createOrReplaceTempView('itemsDf')  
3.  
4.spark.sql("SELECT -value, -storeId FROM transactionsDf INNER JOIN itemsDf ON productId==itemId").drop("attributes")
- C.** transactionsDf.drop("value", "storeId").join(itemsDf.drop("attributes"), "transactionsDf.productId==itemsDf.itemId")
- D.** 1.transactionsDf \  
2. .drop(col('value'), col('storeId')) \  
3. .join(itemsDf.drop(col('attributes')), col('productId')==col('itemId'))
- E.** 1.transactionsDf.createOrReplaceTempView('transactionsDf')  
2.itemsDf.createOrReplaceTempView('itemsDf')  
3.  
4.statement = ""  
5.SELECT \* FROM transactionsDf  
6.INNER JOIN itemsDf  
7.ON transactionsDf.productId==itemsDf.itemId  
8.""  
9.spark.sql(statement).drop("value", "storeId", "attributes")

**Answer:** E

### Explanation

This question offers you a wide variety of answers for a seemingly simple question. However, this variety reflects the variety of ways that one can express a join in PySpark. You need to understand some SQL syntax to get to the correct answer here.

```
transactionsDf.createOrReplaceTempView('transactionsDf')
itemsDf.createOrReplaceTempView('itemsDf')
statement = """
SELECT * FROM transactionsDf
INNER JOIN itemsDf
ON transactionsDf.productId==itemsDf.itemId
"""
```

```
spark.sql(statement).drop("value", "storeId", "attributes")
```

Correct - this answer uses SQL correctly to perform the inner join and afterwards drops the unwanted columns. This is totally fine. If you are unfamiliar with the triple-quote `"""` in Python: This allows you to express strings as multiple lines.

```
transactionsDf \
drop(col('value'), col('storeId')) \
join(itemsDf.drop(col('attributes')), col('productId')==col('itemId'))
```

No, this answer option is a trap, since `DataFrame.drop()` does not accept a list of Column objects. You could use `transactionsDf.drop('value', 'storeId')` instead.

```
transactionsDf.drop("value", "storeId").join(itemsDf.drop("attributes"),
"transactionsDf.productId==itemsDf.itemId")
```

Incorrect - Spark does not evaluate `"transactionsDf.productId==itemsDf.itemId"` as a valid join expression.

This would work if it would not be a string.

```
transactionsDf.drop('value', 'storeId').join(itemsDf.select('attributes'),
transactionsDf.productId==itemsDf.itemId) Wrong, this statement incorrectly uses itemsDf.select
instead of itemsDf.drop.
```

```
transactionsDf.createOrReplaceTempView('transactionsDf')
itemsDf.createOrReplaceTempView('itemsDf')
```

```
spark.sql("SELECT -value, -storeId FROM transactionsDf INNER JOIN itemsDf ON
productId==itemId").drop("attributes") No, here the SQL expression syntax is incorrect. Simply
specifying -columnName does not drop a column.
```

More info: [pyspark.sql.DataFrame.join](#) - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.59** Which of the following code blocks reorders the values inside the arrays in column attributes of DataFrame itemsDf from last to first one in the alphabet?

1. +-----+-----+-----+-----+
2. |itemId|attributes |supplier |
3. +-----+-----+-----+-----+
4. |1 |[blue, winter, cozy] |Sports Company Inc. |
5. |2 |[red, summer, fresh, cooling]|YetiX |
6. |3 |[green, summer, travel] |Sports Company Inc. |
7. +-----+-----+-----+-----+

**A.** `itemsDf.withColumn('attributes', sort_array(col('attributes').desc()))`

- B. `itemsDf.withColumn('attributes', sort_array(desc('attributes')))`
- C. `itemsDf.withColumn('attributes', sort(col('attributes'), asc=False))`
- D. `itemsDf.withColumn("attributes", sort_array("attributes", asc=False))`
- E. `itemsDf.select(sort_array("attributes"))`

**Answer:** D

Explanation

Output of correct code block:

```
+-----+-----+-----+
|itemId|attributes |supplier |
+-----+-----+-----+
1	[winter, cozy, blue]	Sports Company Inc.
2	[summer, red, fresh, cooling]	YetiX
3	[travel, summer, green]	Sports Company Inc.
+-----+-----+-----+
```

It can be confusing to differentiate between the different sorting functions in PySpark. In this case, a particularity about `sort_array` has to be considered: The sort direction is given by the second argument, not by the `desc` method. Luckily, this is documented in the documentation ([link below](#)). Also, for solving this question you need to understand the difference between `sort` and `sort_array`. With `sort`, you cannot sort values in arrays. Also, `sort` is a method of `DataFrame`, while `sort_array` is a method of `pyspark.sql.functions`.

More info: [pyspark.sql.functions.sort\\_array](#) - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 2

**NO.60** Which of the following describes characteristics of the Dataset API?

- A. The Dataset API does not support unstructured data.
- B. In Python, the Dataset API mainly resembles Pandas' DataFrame API.
- C. In Python, the Dataset API's schema is constructed via type hints.
- D. The Dataset API is available in Scala, but it is not available in Python.
- E. The Dataset API does not provide compile-time type safety.

**Answer:** D

Explanation

The Dataset API is available in Scala, but it is not available in Python.

Correct. The Dataset API uses fixed typing and is typically used for object-oriented programming. It is available when Spark is used with the Scala programming language, but not for Python. In Python, you use the DataFrame API, which is based on the Dataset API.

The Dataset API does not provide compile-time type safety.

No - in fact, depending on the use case, the type safety that the Dataset API provides is an advantage.

The Dataset API does not support unstructured data.

Wrong, the Dataset API supports structured and unstructured data.

In Python, the Dataset API's schema is constructed via type hints.

No, this is not applicable since the Dataset API is not available in Python.

In Python, the Dataset API mainly resembles Pandas' DataFrame API.

The Dataset API does not exist in Python, only in Scala and Java.

**NO.61** Which of the following describes a valid concern about partitioning?

- A.** A shuffle operation returns 200 partitions if not explicitly set.
- B.** Decreasing the number of partitions reduces the overall runtime of narrow transformations if there are more executors available than partitions.
- C.** No data is exchanged between executors when `coalesce()` is run.
- D.** Short partition processing times are indicative of low skew.
- E.** The `coalesce()` method should be used to increase the number of partitions.

**Answer:** A

Explanation

A shuffle operation returns 200 partitions if not explicitly set.

Correct. 200 is the default value for the Spark property `spark.sql.shuffle.partitions`. This property determines how many partitions Spark uses when shuffling data for joins or aggregations.

The `coalesce()` method should be used to increase the number of partitions.

Incorrect. The `coalesce()` method can only be used to decrease the number of partitions.

Decreasing the number of partitions reduces the overall runtime of narrow transformations if there are more executors available than partitions.

No. For narrow transformations, fewer partitions usually result in a longer overall runtime, if more executors are available than partitions.

A narrow transformation does not include a shuffle, so no data need to be exchanged between executors.

Shuffles are expensive and can be a bottleneck for executing Spark workloads.

Narrow transformations, however, are executed on a per-partition basis, blocking one executor per partition.

So, it matters how many executors are available to perform work in parallel relative to the number of partitions. If the number of executors is greater than the number of partitions, then some executors are idle while other process the partitions. On the flip side, if the number of executors is smaller than the number of partitions, the entire operation can only be finished after some executors have processed multiple partitions, one after the other. To minimize the overall runtime, one would want to have the number of partitions equal to the number of executors (but not more).

So, for the scenario at hand, increasing the number of partitions reduces the overall runtime of narrow transformations if there are more executors available than partitions.

No data is exchanged between executors when `coalesce()` is run.

No. While `coalesce()` avoids a full shuffle, it may still cause a partial shuffle, resulting in data exchange between executors.

Short partition processing times are indicative of low skew.

Incorrect. Data skew means that data is distributed unevenly over the partitions of a dataset. Low skew therefore means that data is distributed evenly.

Partition processing time, the time that executors take to process partitions, can be indicative of skew if some executors take a long time to process a partition, but others do not. However, a short processing time is not per se indicative a low skew: It may simply be short because the partition is small.

A situation indicative of low skew may be when all executors finish processing their partitions in the same timeframe. High skew may be indicated by some executors taking much longer to finish their partitions than others. But the answer does not make any comparison - so by itself it does not provide enough information to make any assessment about skew.

More info: Spark Repartition & Coalesce - Explained and Performance Tuning - Spark 3.1.

## 2 Documentation

**NO.62** Which of the following code blocks adds a column `predErrorSqrt` to DataFrame `transactionsDf` that is the square root of column `predError`?

- A. `transactionsDf.withColumn("predErrorSqrt", sqrt(predError))`
- B. `transactionsDf.select(sqrt(predError))`
- C. `transactionsDf.withColumn("predErrorSqrt", col("predError").sqrt())`
- D. `transactionsDf.withColumn("predErrorSqrt", sqrt(col("predError")))`
- E. `transactionsDf.select(sqrt("predError"))`

**Answer:** D

Explanation

`transactionsDf.withColumn("predErrorSqrt", sqrt(col("predError")))`

Correct. The `DataFrame.withColumn()` operator is used to add a new column to a DataFrame. It takes two arguments: The name of the new column (here: `predErrorSqrt`) and a Column expression as the new column. In PySpark, a Column expression means referring to a column using the `col("predError")` command or by other means, for example by `transactionsDf.predError`, or even just using the column name as a string, `"predError"`.

The question asks for the square root. `sqrt()` is a function in `pyspark.sql.functions` and calculates the square root. It takes a value or a Column as an input. Here it is the `predError` column of DataFrame `transactionsDf` expressed through `col("predError")`.

`transactionsDf.withColumn("predErrorSqrt", sqrt(predError))`

Incorrect. In this expression, `sqrt(predError)` is incorrect syntax. You cannot refer to `predError` in this way - to Spark it looks as if you are trying to refer to the non-existent Python variable `predError`. You could pass `transactionsDf.predError`, `col("predError")` (as in the correct solution), or even just `"predError"` instead.

`transactionsDf.select(sqrt(predError))`

Wrong. Here, the explanation just above this one about how to refer to `predError` applies.

`transactionsDf.select(sqrt("predError"))`

No. While this is correct syntax, it will return a single-column DataFrame only containing a column showing the square root of column `predError`. However, the question asks for a column to be added to the original DataFrame `transactionsDf`.

`transactionsDf.withColumn("predErrorSqrt", col("predError").sqrt())`

No. The issue with this statement is that column `col("predError")` has no `sqrt()` method. `sqrt()` is a member of `pyspark.sql.functions`, but not of `pyspark.sql.Column`.

More info: `pyspark.sql.DataFrame.withColumn` - PySpark 3.1.2 documentation and

`pyspark.sql.functions.sqrt` - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 2

**NO.63** Which of the following code blocks sorts DataFrame `transactionsDf` both by column `storeId` in ascending and by column `productId` in descending order, in this priority?

- A. `transactionsDf.sort("storeId", asc("productId"))`
- B. `transactionsDf.sort(col(storeId)).desc(col(productId))`
- C. `transactionsDf.order_by(col(storeId), desc(col(productId)))`
- D. `transactionsDf.sort("storeId", desc("productId"))`
- E. `transactionsDf.sort("storeId").sort(desc("productId"))`

**Answer: D**

## Explanation

In this question it is important to realize that you are asked to sort transactionDf by two columns.

This means that the sorting of the second column depends on the sorting of the first column.

So, any option that sorts the entire DataFrame (through chaining sort statements) will not work. The two columns need to be channeled through the same call to sort().

Also, order\_by is not a valid DataFrame API method.

More info: `pyspark.sql.DataFrame.sort` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 2

**NO.64** The code block displayed below contains an error. The code block should merge the rows of DataFrames transactionsDfMonday and transactionsDfTuesday into a new DataFrame, matching column names and inserting null values where column names do not appear in both DataFrames. Find the error.

Sample of DataFrame transactionsDfMonday:

```
1.+-----+-----+----+-----+-----+----+
2.|transactionId|predError|value|storeId|productId| f|
3.+-----+-----+----+-----+-----+----+
4.| 5| null| null| null| 2|null|
5.| 6| 3| 2| 25| 2|null|
```

Sample of DataFrame transactionsDfTuesday:

```
1.+-----+-----+-----+-----+
2.|storeId|transactionId|productId|value|
3.+-----+-----+-----+-----+
4.| 25| 1| 1| 4|
5.| 2| 2| 2| 7|
6.| 3| 4| 2| null|
7.| null| 5| 2| null|
8.+-----+-----+-----+-----+
```

Code block:

```
sc.union([transactionsDfMonday, transactionsDfTuesday])
```

- A.** The DataFrames' RDDs need to be passed into the `sc.union` method instead of the DataFrame variable names.
- B.** Instead of `union`, the `concat` method should be used, making sure to not use its default arguments.
- C.** Instead of the Spark context, `transactionDfMonday` should be called with the `join` method instead of the `union` method, making sure to use its default arguments.
- D.** Instead of the Spark context, `transactionDfMonday` should be called with the `union` method.
- E.** Instead of the Spark context, `transactionDfMonday` should be called with the `unionByName` method instead of the `union` method, making sure to not use its default arguments.

**Answer: E**

## Explanation

Correct code block:

```
transactionsDfMonday.unionByName(transactionsDfTuesday, True)
```

Output of correct code block:

```

+-----+-----+-----+-----+-----+-----+
|transactionId|predError|value|storeId|productId| f|
+-----+-----+-----+-----+-----+-----+
5	null	null	null	2	null
6	3	2	25	2	null
1	null	4	25	1	null
2	null	7	2	2	null
4	null	null	3	2	null
5	null	null	null	2	null
+-----+-----+-----+-----+-----+

```

For solving this question, you should be aware of the difference between the `DataFrame.union()` and `DataFrame.unionByName()` methods. The first one matches columns independent of their names, just by their order. The second one matches columns by their name (which is asked for in the question). It also has a useful optional argument, `allowMissingColumns`. This allows you to merge DataFrames that have different columns - just like in this example.

`sc` stands for `SparkContext` and is automatically provided when executing code on Databricks. While `sc.union()` allows you to join RDDs, it is not the right choice for joining DataFrames. A hint away from `sc.union()` is given where the question talks about joining "into a new DataFrame".

`concat` is a method in `pyspark.sql.functions`. It is great for consolidating values from different columns, but has no place when trying to join rows of multiple DataFrames.

Finally, the `join` method is a contender here. However, the default join defined for that method is an inner join which does not get us closer to the goal to match the two DataFrames as instructed, especially given that with the default arguments we cannot define a join condition.

More info:

- `pyspark.sql.DataFrame.unionByName` - PySpark 3.1.2 documentation
- `pyspark.SparkContext.union` - PySpark 3.1.2 documentation
- `pyspark.sql.functions.concat` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.65** In which order should the code blocks shown below be run in order to read a JSON file from location `jsonPath` into a DataFrame and return only the rows that do not have value 3 in column `productId`?

1. `importedDf.createOrReplaceTempView("importedDf")`
2. `spark.sql("SELECT * FROM importedDf WHERE productId != 3")`
3. `spark.sql("FILTER * FROM importedDf WHERE productId != 3")`
4. `importedDf = spark.read.option("format", "json").path(jsonPath)`
5. `importedDf = spark.read.json(jsonPath)`

**A.** 4, 1, 2

**B.** 5, 1, 3

**C.** 5, 2

**D.** 4, 1, 3

**E.** 5, 1, 2

**Answer:** E

Explanation

Correct code block:

```
importedDf = spark.read.json(jsonPath)
```

```
importedDf.createOrReplaceTempView("importedDf")
```

```
spark.sql("SELECT * FROM importedDf WHERE productId != 3")
```

Option 5 is the only correct way listed of reading in a JSON in PySpark. The option("format", "json") is not the correct way to tell Spark's DataFrameReader that you want to read a JSON file. You would do this through format("json") instead. Also, you can communicate the specific path of the JSON file to the DataFrameReader using the load() method, not the path() method.

In order to use a SQL command through the SparkSession spark, you first need to create a temporary view through DataFrame.createOrReplaceTempView().

The SQL statement should start with the SELECT operator. The FILTER operator SQL provides is not the correct one to use here.

Static notebook | Dynamic notebook: See test 2

**NO.66** Which of the following code blocks reduces a DataFrame from 12 to 6 partitions and performs a full shuffle?

- A. DataFrame.repartition(12)
- B. DataFrame.coalesce(6).shuffle()
- C. DataFrame.coalesce(6)
- D. DataFrame.coalesce(6, shuffle=True)
- E. DataFrame.repartition(6)

**Answer:** E

Explanation

```
DataFrame.repartition(6)
```

Correct. repartition() always triggers a full shuffle (different from coalesce()).

```
DataFrame.repartition(12)
```

No, this would just leave the DataFrame with 12 partitions and not 6.

```
DataFrame.coalesce(6)
```

coalesce does not perform a full shuffle of the data. Whenever you see "full shuffle", you know that you are not dealing with coalesce(). While coalesce() can perform a partial shuffle when required, it will try to minimize shuffle operations, so the amount of data that is sent between executors. Here, 12 partitions can easily be repartitioned to be 6 partitions simply by stitching every two partitions into one.

DataFrame.coalesce(6, shuffle=True) and DataFrame.coalesce(6).shuffle() These statements are not valid Spark API syntax.

More info: Spark Repartition & Coalesce - Explained and Repartition vs Coalesce in Apache Spark - Rock the JVM Blog

**NO.67** Which of the following describes a difference between Spark's cluster and client execution modes?

- A. In cluster mode, the cluster manager resides on a worker node, while it resides on an edge node in client mode.
- B. In cluster mode, executor processes run on worker nodes, while they run on gateway nodes in client mode.
- C. In cluster mode, the driver resides on a worker node, while it resides on an edge node in client mode.
- D. In cluster mode, a gateway machine hosts the driver, while it is co-located with the executor in

client mode.

**E.** In cluster mode, the Spark driver is not co-located with the cluster manager, while it is co-located in client mode.

**Answer:** C

Explanation

In cluster mode, the driver resides on a worker node, while it resides on an edge node in client mode. Correct. The idea of Spark's client mode is that workloads can be executed from an edge node, also known as gateway machine, from outside the cluster. The most common way to execute Spark however is in cluster mode, where the driver resides on a worker node.

In practice, in client mode, there are tight constraints about the data transfer speed relative to the data transfer speed between worker nodes in the cluster. Also, any job in that is executed in client mode will fail if the edge node fails. For these reasons, client mode is usually not used in a production environment.

In cluster mode, the cluster manager resides on a worker node, while it resides on an edge node in client execution mode.

No. In both execution modes, the cluster manager may reside on a worker node, but it does not reside on an edge node in client mode.

In cluster mode, executor processes run on worker nodes, while they run on gateway nodes in client mode.

This is incorrect. Only the driver runs on gateway nodes (also known as "edge nodes") in client mode, but not the executor processes.

In cluster mode, the Spark driver is not co-located with the cluster manager, while it is co-located in client mode.

No, in client mode, the Spark driver is not co-located with the driver. The whole point of client mode is that the driver is outside the cluster and not associated with the resource that manages the cluster (the machine that runs the cluster manager).

In cluster mode, a gateway machine hosts the driver, while it is co-located with the executor in client mode.

No, it is exactly the opposite: There are no gateway machines in cluster mode, but in client mode, they host the driver.

**NO.68** Which of the following code blocks creates a new one-column, two-row DataFrame dfDates with column date of type timestamp?

- A.** 1.dfDates = spark.createDataFrame([["23/01/2022 11:28:12","24/01/2022 10:58:34"], ["date"])
- 2.dfDates = dfDates.withColumn("date", to\_timestamp("dd/MM/yyyy HH:mm:ss", "date"))
- B.** 1.dfDates = spark.createDataFrame([("23/01/2022 11:28:12"),("24/01/2022 10:58:34"),], ["date"])
- 2.dfDates = dfDates.withColumnRenamed("date", to\_timestamp("date", "yyyy-MM-dd HH:mm:ss"))
- C.** 1.dfDates = spark.createDataFrame([("23/01/2022 11:28:12"),("24/01/2022 10:58:34"),], ["date"])
- 2.dfDates = dfDates.withColumn("date", to\_timestamp("date", "dd/MM/yyyy HH:mm:ss"))
- D.** 1.dfDates = spark.createDataFrame([["23/01/2022 11:28:12","24/01/2022 10:58:34"], ["date"])
- 2.dfDates = dfDates.withColumnRenamed("date", to\_datetime("date", "yyyy-MM-dd HH:mm:ss"))
- E.** 1.dfDates = spark.createDataFrame([("23/01/2022 11:28:12"),("24/01/2022 10:58:34"),], ["date"])

**Answer:** C

Explanation

This question is tricky. Two things are important to know here:

First, the syntax for `createDataFrame`: Here you need a list of tuples, like so: `[(1,), (2,)]`. To define a tuple in Python, if you just have a single item in it, it is important to put a comma after the item so that Python interprets it as a tuple and not just a normal parenthesis.

Second, you should understand the `to_timestamp` syntax. You can find out more about it in the documentation linked below.

For good measure, let's examine in detail why the incorrect options are wrong:

`dfDates = spark.createDataFrame([("23/01/2022 11:28:12",), ("24/01/2022 10:58:34",)], ["date"])` This code snippet does everything the question asks for - except that the data type of the date column is a string and not a timestamp. When no schema is specified, Spark sets the string data type as default.

`dfDates = spark.createDataFrame(["23/01/2022 11:28:12", "24/01/2022 10:58:34"], ["date"])` `dfDates = dfDates.withColumn("date", to_timestamp("dd/MM/yyyy HH:mm:ss", "date"))` In the first row of this command, Spark throws the following error: `TypeError: Can not infer schema for type: <class 'str'>`. This is because Spark expects to find row information, but instead finds strings. This is why you need to specify the data as tuples. Fortunately, the Spark documentation (linked below) shows a number of examples for creating DataFrames that should help you get on the right track here.

`dfDates = spark.createDataFrame([("23/01/2022 11:28:12",), ("24/01/2022 10:58:34",)], ["date"])`

`dfDates = dfDates.withColumnRenamed("date", to_timestamp("date", "yyyy-MM-dd HH:mm:ss"))`

The issue with this answer is that the operator `withColumnRenamed` is used. This operator simply renames a column, but it has no power to modify its actual content. This is why `withColumn` should be used instead. In addition, the date format `yyyy-MM-dd HH:mm:ss` does not reflect the format of the actual timestamp: `"23/01/2022 11:28:12"`.

`dfDates = spark.createDataFrame(["23/01/2022 11:28:12", "24/01/2022 10:58:34"], ["date"])` `dfDates = dfDates.withColumnRenamed("date", to_datetime("date", "yyyy-MM-dd HH:mm:ss"))` Here, `withColumnRenamed` is used instead of `withColumn` (see above). In addition, the rows are not expressed correctly - they should be written as tuples, using parentheses. Finally, even the date format is off here (see above).

More info: `pyspark.sql.functions.to_timestamp` - PySpark 3.1.2 documentation and

`pyspark.sql.Session.createDataFrame` - PySpark 3.1.1 documentation Static notebook | Dynamic notebook: See test 2

**NO.69** Which of the following statements about executors is correct, assuming that one can consider each of the JVMs working as executors as a pool of task execution slots?

- A. Slot is another name for executor.
- B. There must be less executors than tasks.
- C. An executor runs on a single core.
- D. There must be more slots than tasks.
- E. Tasks run in parallel via slots.

**Answer:** E

Explanation

Tasks run in parallel via slots.

Correct. Given the assumption, an executor then has one or more "slots", defined by the equation `spark.executor.cores / spark.task.cpus`. With the executor's resources divided into slots, each task takes up a slot and multiple tasks can be executed in parallel.

Slot is another name for executor.

No, a slot is part of an executor.

An executor runs on a single core.

No, an executor can occupy multiple cores. This is set by the `spark.executor.cores` option.

There must be more slots than tasks.

No. Slots just process tasks. One could imagine a scenario where there was just a single slot for multiple tasks, processing one task at a time. Granted - this is the opposite of what Spark should be used for, which is distributed data processing over multiple cores and machines, performing many tasks in parallel.

There must be less executors than tasks.

No, there is no such requirement.

More info: Spark Architecture | Distributed Systems Architecture (<https://bit.ly/3x4MZZt>)

**NO.70** Which of the following code blocks efficiently converts DataFrame transactionsDf from 12 into 24 partitions?

**A.** `transactionsDf.repartition(24, boost=True)`

**B.** `transactionsDf.repartition()`

**C.** `transactionsDf.repartition("itemId", 24)`

**D.** `transactionsDf.coalesce(24)`

**E.** `transactionsDf.repartition(24)`

**Answer:** E

Explanation

`transactionsDf.coalesce(24)`

No, the `coalesce()` method can only reduce, but not increase the number of partitions.

`transactionsDf.repartition()`

No, `repartition()` requires a `numPartitions` argument.

`transactionsDf.repartition("itemId", 24)`

No, here the `cols` and `numPartitions` argument have been mixed up. If the code block would be `transactionsDf.repartition(24, "itemId")`, this would be a valid solution.

`transactionsDf.repartition(24, boost=True)`

No, there is no `boost` argument in the `repartition()` method.

**NO.71** Which of the following code blocks returns a DataFrame that is an inner join of DataFrame itemsDf and DataFrame transactionsDf, on columns itemId and productId, respectively and in which every itemId just appears once?

**A.** `itemsDf.join(transactionsDf, "itemsDf.itemId==transactionsDf.productId").distinct("itemId")`

**B.** `itemsDf.join(transactionsDf, itemsDf.itemId==transactionsDf.productId).dropDuplicates(["itemId"])`

**C.** `itemsDf.join(transactionsDf, itemsDf.itemId==transactionsDf.productId).dropDuplicates("itemId")`

**D.** `itemsDf.join(transactionsDf, itemsDf.itemId==transactionsDf.productId, how="inner").distinct(["itemId"])`

**E.** `itemsDf.join(transactionsDf, "itemsDf.itemId==transactionsDf.productId", how="inner").dropDuplicates(["itemId"])`

**Answer:** B

Explanation

Filtering out distinct rows based on columns is achieved with the `dropDuplicates` method, not the

distinct method which does not take any arguments.

The second argument of the join() method only accepts strings if they are column names. The SQL-like statement "itemsDf.itemId==transactionsDf.productId" is therefore invalid.

In addition, it is not necessary to specify how="inner", since the default join type for the join command is already inner.

More info: [pyspark.sql.DataFrame.join](#) - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 2

**NO.72** Which of the following describes Spark's way of managing memory?

- A.** Spark uses a subset of the reserved system memory.
- B.** Storage memory is used for caching partitions derived from DataFrames.
- C.** As a general rule for garbage collection, Spark performs better on many small objects than few big objects.
- D.** Disabling serialization potentially greatly reduces the memory footprint of a Spark application.
- E.** Spark's memory usage can be divided into three categories: Execution, transaction, and storage.

**Answer:** B

Explanation

Spark's memory usage can be divided into three categories: Execution, transaction, and storage.

No, it is either execution or storage.

As a general rule for garbage collection, Spark performs better on many small objects than few big objects.

No, Spark's garbage collection runs faster on fewer big objects than many small objects.

Disabling serialization potentially greatly reduces the memory footprint of a Spark application.

The opposite is true - serialization reduces the memory footprint, but may impact performance in a negative way.

Spark uses a subset of the reserved system memory.

No, the reserved system memory is separate from Spark memory. Reserved memory stores Spark's internal objects.

More info: [Tuning - Spark 3.1.2 Documentation](#), [Spark Memory Management | Distributed Systems Architecture](#), [Learning Spark, 2nd Edition, Chapter 7](#)

**NO.73** The code block displayed below contains multiple errors. The code block should return a DataFrame that contains only columns transactionId, predError, value and storeId of DataFrame transactionsDf. Find the errors.

Code block:

```
transactionsDf.select([col(productId), col(f)])
```

Sample of transactionsDf:

```
1. +-----+-----+----+-----+-----+----+
2. | transactionId | predError | value | storeId | productId | f |
3. +-----+-----+----+-----+-----+----+
4. | 1 | 3 | 4 | 25 | 1 | null |
5. | 2 | 6 | 7 | 2 | 2 | null |
6. | 3 | 3 | null | 25 | 3 | null |
7. +-----+-----+----+-----+-----+----+
```

- A.** The column names should be listed directly as arguments to the operator and not as a list.

- B.** The select operator should be replaced by a drop operator, the column names should be listed directly as arguments to the operator and not as a list, and all column names should be expressed as strings without being wrapped in a col() operator.
- C.** The select operator should be replaced by a drop operator.
- D.** The column names should be listed directly as arguments to the operator and not as a list and following the pattern of how column names are expressed in the code block, columns productId and f should be replaced by transactionId, predError, value and storeId.
- E.** The select operator should be replaced by a drop operator, the column names should be listed directly as arguments to the operator and not as a list, and all col() operators should be removed.

**Answer:** B

Explanation

Correct code block: `transactionsDf.drop("productId", "f")`

This question requires a lot of thinking to get right. For solving it, you may take advantage of the digital notepad that is provided to you during the test. You have probably seen that the code block includes multiple errors. In the test, you are usually confronted with a code block that only contains a single error. However, since you are practicing here, this challenging multi-error question will make it easier for you to deal with single-error questions in the real exam.

The select operator should be replaced by a drop operator, the column names should be listed directly as arguments to the operator and not as a list, and all column names should be expressed as strings without being wrapped in a col() operator.

Correct! Here, you need to figure out the many, many things that are wrong with the initial code block. While the question can be solved by using a select statement, a drop statement, given the answer options, is the correct one. Then, you can read in the documentation that drop does not take a list as an argument, but just the column names that should be dropped. Finally, the column names should be expressed as strings and not as Python variable names as in the original code block.

The column names should be listed directly as arguments to the operator and not as a list.

Incorrect. While this is a good first step and part of the correct solution (see above), this modification is insufficient to solve the question.

The column names should be listed directly as arguments to the operator and not as a list and following the pattern of how column names are expressed in the code block, columns productId and f should be replaced by transactionId, predError, value and storeId.

Wrong. If you use the same pattern as in the original code block (`col(productId)`, `col(f)`), you are still making a mistake. `col(productId)` will trigger Python to search for the content of a variable named productId instead of telling Spark to use the column productId - for that, you need to express it as a string.

The select operator should be replaced by a drop operator, the column names should be listed directly as arguments to the operator and not as a list, and all col() operators should be removed.

No. This still leaves you with Python trying to interpret the column names as Python variables (see above).

The select operator should be replaced by a drop operator.

Wrong, this is not enough to solve the question. If you do this, you will still face problems since you are passing a Python list to drop and the column names are still interpreted as Python variables (see above).

More info: `pyspark.sql.DataFrame.drop` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.74** Which of the following describes a narrow transformation?

- A.** narrow transformation is an operation in which data is exchanged across partitions.
- B.** A narrow transformation is a process in which data from multiple RDDs is used.
- C.** A narrow transformation is a process in which 32-bit float variables are cast to smaller float variables, like 16-bit or 8-bit float variables.
- D.** A narrow transformation is an operation in which data is exchanged across the cluster.
- E.** A narrow transformation is an operation in which no data is exchanged across the cluster.

**Answer:** E

Explanation

A narrow transformation is an operation in which no data is exchanged across the cluster.

Correct! In narrow transformations, no data is exchanged across the cluster, since these transformations do not require any data from outside of the partition they are applied on. Typical narrow transformations include filter, drop, and coalesce.

A narrow transformation is an operation in which data is exchanged across partitions.

No, that would be one definition of a wide transformation, but not of a narrow transformation. Wide transformations typically cause a shuffle, in which data is exchanged across partitions, executors, and the cluster.

A narrow transformation is an operation in which data is exchanged across the cluster.

No, see explanation just above this one.

A narrow transformation is a process in which 32-bit float variables are cast to smaller float variables, like

16-bit or 8-bit float variables.

No, type conversion has nothing to do with narrow transformations in Spark.

A narrow transformation is a process in which data from multiple RDDs is used.

No. A resilient distributed dataset (RDD) can be described as a collection of partitions. In a narrow transformation, no data is exchanged between partitions. Thus, no data is exchanged between RDDs. One could say though that a narrow transformation and, in fact, any transformation results in a new RDD being created. This is because a transformation results in a change to an existing RDD (RDDs are the foundation of other Spark data structures, like DataFrames). But, since RDDs are immutable, a new RDD needs to be created to reflect the change caused by the transformation.

More info: [Spark Transformation and Action: A Deep Dive](#) | by Misbah Uddin | CodeX | Medium

**NO.75** Which of the following describes tasks?

- A.** A task is a command sent from the driver to the executors in response to a transformation.
- B.** Tasks transform jobs into DAGs.
- C.** A task is a collection of slots.
- D.** A task is a collection of rows.
- E.** Tasks get assigned to the executors by the driver.

**Answer:** E

Explanation

Tasks get assigned to the executors by the driver.

Correct! Or, in other words: Executors take the tasks that they were assigned to by the driver, run them over partitions, and report the their outcomes back to the driver.

Tasks transform jobs into DAGs.

No, this statement disrespects the order of elements in the Spark hierarchy. The Spark driver

transforms jobs into DAGs. Each job consists of one or more stages. Each stage contains one or more tasks.

A task is a collection of rows.

Wrong. A partition is a collection of rows. Tasks have little to do with a collection of rows. If anything, a task processes a specific partition.

A task is a command sent from the driver to the executors in response to a transformation.

Incorrect. The Spark driver does not send anything to the executors in response to a transformation, since transformations are evaluated lazily. So, the Spark driver would send tasks to executors only in response to actions.

A task is a collection of slots.

No. Executors have one or more slots to process tasks and each slot can be assigned a task.

**NO.76** Which of the following describes Spark actions?

- A.** Writing data to disk is the primary purpose of actions.
- B.** Actions are Spark's way of exchanging data between executors.
- C.** The driver receives data upon request by actions.
- D.** Stage boundaries are commonly established by actions.
- E.** Actions are Spark's way of modifying RDDs.

**Answer:** C

Explanation

The driver receives data upon request by actions.

Correct! Actions trigger the distributed execution of tasks on executors which, upon task completion, transfer result data back to the driver.

Actions are Spark's way of exchanging data between executors.

No. In Spark, data is exchanged between executors via shuffles.

Writing data to disk is the primary purpose of actions.

No. The primary purpose of actions is to access data that is stored in Spark's RDDs and return the data, often in aggregated form, back to the driver.

Actions are Spark's way of modifying RDDs.

Incorrect. Firstly, RDDs are immutable - they cannot be modified. Secondly, Spark generates new RDDs via transformations and not actions.

Stage boundaries are commonly established by actions.

Wrong. A stage boundary is commonly established by a shuffle, for example caused by a wide transformation.

**NO.77** Which of the following statements about Spark's configuration properties is incorrect?

- A.** The maximum number of tasks that an executor can process at the same time is controlled by the `spark.task.cpus` property.
- B.** The maximum number of tasks that an executor can process at the same time is controlled by the `spark.executor.cores` property.
- C.** The default value for `spark.sql.autoBroadcastJoinThreshold` is 10MB.
- D.** The default number of partitions to use when shuffling data for joins or aggregations is 300.
- E.** The default number of partitions returned from certain transformations can be controlled by the `spark.default.parallelism` property.

**Answer:** D

**Explanation**

The default number of partitions to use when shuffling data for joins or aggregations is 300.

No, the default value of the applicable property `spark.sql.shuffle.partitions` is 200.

The maximum number of tasks that an executor can process at the same time is controlled by the `spark.executor.cores` property.

Correct, see below.

The maximum number of tasks that an executor can process at the same time is controlled by the `spark.task.cpus` property.

Correct, the maximum number of tasks that an executor can process in parallel depends on both properties `spark.task.cpus` and `spark.executor.cores`. This is because the available number of slots is calculated by dividing the number of cores per executor by the number of cores per task. For more info specifically to this point, check out [Spark Architecture | Distributed Systems Architecture](#).

More info: [Configuration - Spark 3.1.2 Documentation](#)

**NO.78** The code block displayed below contains an error. The code block should combine data from DataFrames `itemsDf` and `transactionsDf`, showing all rows of DataFrame `itemsDf` that have a matching value in column `itemId` with a value in column `transactionId` of DataFrame `transactionsDf`. Find the error.

Code block:

```
itemsDf.join(itemsDf.itemId==transactionsDf.transactionId)
```

- A.** The join statement is incomplete.
- B.** The union method should be used instead of join.
- C.** The join method is inappropriate.
- D.** The merge method should be used instead of join.
- E.** The join expression is malformed.

**Answer:** A

**Explanation**

Correct code block:

```
itemsDf.join(transactionsDf, itemsDf.itemId==transactionsDf.transactionId)
```

The join statement is incomplete.

Correct! If you look at the documentation of `DataFrame.join()` (linked below), you see that the very first argument of join should be the DataFrame that should be joined with. This first argument is missing in the code block.

The join method is inappropriate.

No. By default, `DataFrame.join()` uses an inner join. This method is appropriate for the scenario described in the question.

The join expression is malformed.

Incorrect. The join expression `itemsDf.itemId==transactionsDf.transactionId` is correct syntax.

The merge method should be used instead of join.

False. There is no `DataFrame.merge()` method in PySpark.

The union method should be used instead of join.

Wrong. `DataFrame.union()` merges rows, but not columns as requested in the question.

More info: [pyspark.sql.DataFrame.join - PySpark 3.1.2 documentation](#), [pyspark.sql.DataFrame.union - PySpark 3.1.2 documentation](#) Static notebook | Dynamic notebook: See test 3

**NO.79** The code block shown below should return all rows of DataFrame `itemsDf` that have at least 3

items in column itemNameElements. Choose the answer that correctly fills the blanks in the code block to accomplish this.

Example of DataFrame itemsDf:

```
1. +-----+-----+-----+-----+
2. |itemId|itemName |supplier |itemNameElements |
3. +-----+-----+-----+-----+
4. |1 |Thick Coat for Walking in the Snow|Sports Company Inc. |[Thick, Coat, for, Walking, in, the, Snow]|
5. |2 |Elegant Outdoors Summer Dress |YetiX |[Elegant, Outdoors, Summer, Dress] |
6. |3 |Outdoors Backpack |Sports Company Inc. |[Outdoors, Backpack] |
```

7. +-----+-----+-----+-----+ Code block:

itemsDf. \_\_1\_\_ ( \_\_2\_\_ ( \_\_3\_\_ ) \_\_4\_\_ )

**A.** 1. select

2. count

3. col("itemNameElements")

4. >3

**B.** 1. filter

2. count

3. itemNameElements

4. >=3

**C.** 1. select

2. count

3. "itemNameElements"

4. >3

**D.** 1. filter

2. size

3. "itemNameElements"

4. >=3

(Correct)

**E.** 1. select

2. size

3. "itemNameElements"

4. >3

**Answer:** D

Explanation

Correct code block:

```
itemsDf.filter(size("itemNameElements")>3)
```

Output of code block:

```
+-----+-----+-----+-----+
|itemId|itemName |supplier |itemNameElements |
+-----+-----+-----+-----+
|1 |Thick Coat for Walking in the Snow|Sports Company Inc. |[Thick, Coat, for, Walking, in, the, Snow]|
|2 |Elegant Outdoors Summer Dress |YetiX |[Elegant, Outdoors, Summer, Dress] |
```

+-----+-----+-----+-----+ The big difficulty

with this question is in knowing the difference between count and size (refer to documentation below). size is the correct function to choose here since it returns the number of elements in an array on a per-row basis.

The other consideration for solving this question is the difference between select and filter. Since we want to return the rows in the original DataFrame, filter is the right choice. If we would use select, we would simply get a single-column DataFrame showing which rows match the criteria, like so:

```
+-----+
| (size(itemNameElements) > 3) |
+-----+
| true |
| true |
| false |
+-----+
```

More info:

Count documentation: [pyspark.sql.functions.count - PySpark 3.1.1 documentation](#) Size documentation: [pyspark.sql.functions.size - PySpark 3.1.1 documentation](#) Static notebook | Dynamic notebook: See test 1

**NO.80** Which of the following code blocks returns a DataFrame with an added column to DataFrame transactionsDf that shows the unix epoch timestamps in column transactionDate as strings in the format month/day/year in column transactionDateFormatted?

Excerpt of DataFrame transactionsDf:

- A.** transactionsDf.withColumn("transactionDateFormatted", from\_unixtime("transactionDate", format="dd/MM/yyyy"))
- B.** transactionsDf.withColumnRenamed("transactionDate", "transactionDateFormatted", from\_unixtime("transactionDateFormatted", format="MM/dd/yyyy"))
- C.** transactionsDf.apply(from\_unixtime(format="MM/dd/yyyy")).asColumnn("transactionDateFormatted")
- D.** transactionsDf.withColumn("transactionDateFormatted", from\_unixtime("transactionDate", format="MM/dd/yyyy"))
- E.** transactionsDf.withColumn("transactionDateFormatted", from\_unixtime("transactionDate"))

**Answer:** D

Explanation

transactionsDf.withColumn("transactionDateFormatted", from\_unixtime("transactionDate", format="MM/dd/yyyy")) Correct. This code block adds a new column with the name transactionDateFormatted to DataFrame transactionsDf, using Spark's from\_unixtime method to transform values in column transactionDate into strings, following the format requested in the question.

transactionsDf.withColumn("transactionDateFormatted", from\_unixtime("transactionDate", format="dd/MM/yyyy")) No. Although almost correct, this uses the wrong format for the timestamp to date conversion: day/month/year instead of month/day/year.

transactionsDf.withColumnRenamed("transactionDate", "transactionDateFormatted", from\_unixtime("transactionDateFormatted", format="MM/dd/yyyy")) Incorrect. This answer uses wrong syntax. The command DataFrame.withColumnRenamed() is for renaming an existing column

only has two string parameters, specifying the old and the new name of the column.

`transactionsDf.apply(from_unixtime(format="MM/dd/yyyy")).asColumn("transactionDateFormatted")` Wrong. Although this answer looks very tempting, it is actually incorrect Spark syntax. In Spark, there is no method `DataFrame.apply()`. Spark has an `apply()` method that can be used on grouped data - but this is irrelevant for this question, since we do not deal with grouped data here.

`transactionsDf.withColumn("transactionDateFormatted", from_unixtime("transactionDate"))` No. Although this is valid Spark syntax, the strings in column `transactionDateFormatted` would look like this:

2020-04-26 15:35:32, the default format specified in Spark for `from_unixtime` and not what is asked for in the question.

More info: `pyspark.sql.functions.from_unixtime` - PySpark 3.1.1 documentation and

`pyspark.sql.DataFrame.withColumnRenamed` - PySpark 3.1.1 documentation Static notebook |

Dynamic notebook: See test 1

**NO.81** Which of the following describes a way for resizing a DataFrame from 16 to 8 partitions in the most efficient way?

**A.** Use operation `DataFrame.repartition(8)` to shuffle the DataFrame and reduce the number of partitions.

**B.** Use operation `DataFrame.coalesce(8)` to fully shuffle the DataFrame and reduce the number of partitions.

**C.** Use a narrow transformation to reduce the number of partitions.

**D.** Use a wide transformation to reduce the number of partitions.

Use operation `DataFrame.coalesce(0.5)` to halve the number of partitions in the DataFrame.

**Answer:** C

Explanation

Use a narrow transformation to reduce the number of partitions.

Correct! `DataFrame.coalesce(n)` is a narrow transformation, and in fact the most efficient way to resize the DataFrame of all options listed. One would run `DataFrame.coalesce(8)` to resize the DataFrame.

Use operation `DataFrame.coalesce(8)` to fully shuffle the DataFrame and reduce the number of partitions.

Wrong. The `coalesce` operation avoids a full shuffle, but will shuffle data if needed. This answer is incorrect because it says "fully shuffle" - this is something the `coalesce` operation will not do. As a general rule, it will reduce the number of partitions with the very least movement of data possible.

More info:

distributed computing - Spark - `repartition()` vs `coalesce()` - Stack Overflow Use operation `DataFrame.coalesce(0.5)` to halve the number of partitions in the DataFrame.

Incorrect, since the `num_partitions` parameter needs to be an integer number defining the exact number of partitions desired after the operation. More info: `pyspark.sql.DataFrame.coalesce` - PySpark 3.1.2 documentation Use operation `DataFrame.repartition(8)` to shuffle the DataFrame and reduce the number of partitions.

No. The `repartition` operation will fully shuffle the DataFrame. This is not the most efficient way of reducing the number of partitions of all listed options.

Use a wide transformation to reduce the number of partitions.

No. While possible via the `DataFrame.repartition(n)` command, the resulting full shuffle is not the most efficient way of reducing the number of partitions.

**NO.82** Which of the following describes characteristics of the Spark UI?

- A.** Via the Spark UI, workloads can be manually distributed across executors.
- B.** Via the Spark UI, stage execution speed can be modified.
- C.** The Scheduler tab shows how jobs that are run in parallel by multiple users are distributed across the cluster.
- D.** There is a place in the Spark UI that shows the property `spark.executor.memory`.
- E.** Some of the tabs in the Spark UI are named Jobs, Stages, Storage, DAGs, Executors, and SQL.

**Answer:** D

Explanation

There is a place in the Spark UI that shows the property `spark.executor.memory`.

Correct, you can see Spark properties such as `spark.executor.memory` in the Environment tab.

Some of the tabs in the Spark UI are named Jobs, Stages, Storage, DAGs, Executors, and SQL.

Wrong - Jobs, Stages, Storage, Executors, and SQL are all tabs in the Spark UI. DAGs can be inspected in the

"Jobs" tab in the job details or in the Stages or SQL tab, but are not a separate tab.

Via the Spark UI, workloads can be manually distributed across distributors.

No, the Spark UI is meant for inspecting the inner workings of Spark which ultimately helps understand, debug, and optimize Spark transactions.

Via the Spark UI, stage execution speed can be modified.

No, see above.

The Scheduler tab shows how jobs that are run in parallel by multiple users are distributed across the cluster.

No, there is no Scheduler tab.

**NO.83** Which of the following DataFrame operators is never classified as a wide transformation?

- A.** `DataFrame.sort()`
- B.** `DataFrame.aggregate()`
- C.** `DataFrame.repartition()`
- D.** `DataFrame.select()`
- E.** `DataFrame.join()`

**Answer:** D

Explanation

As a general rule: After having gone through the practice tests you probably have a good feeling for what classifies as a wide and what classifies as a narrow transformation. If you are unsure, feel free to play around in Spark and display the explanation of the Spark execution plan via

`DataFrame.[operation, for example sort()].explain()`. If repartitioning is involved, it would count as a wide transformation.

`DataFrame.select()`

Correct! A wide transformation includes a shuffle, meaning that an input partition maps to one or more output partitions. This is expensive and causes traffic across the cluster. With the `select()` operation however, you pass commands to Spark that tell Spark to perform an operation on a specific slice of any partition. For this, Spark does not need to exchange data across partitions, each partition can be worked on independently. Thus, you do not cause a wide transformation.

`DataFrame.repartition()`

Incorrect. When you repartition a DataFrame, you redefine partition boundaries. Data will flow across your cluster and end up in different partitions after the repartitioning is completed. This is known as a shuffle and, in turn, is classified as a wide transformation.

`DataFrame.aggregate()`

No. When you aggregate, you may compare and summarize data across partitions. In the process, data are exchanged across the cluster, and newly formed output partitions depend on one or more input partitions. This is a typical characteristic of a shuffle, meaning that the aggregate operation may classify as a wide transformation.

`DataFrame.join()`

Wrong. Joining multiple DataFrames usually means that large amounts of data are exchanged across the cluster, as new partitions are formed. This is a shuffle and therefore `DataFrame.join()` counts as a wide transformation.

`DataFrame.sort()`

False. When sorting, Spark needs to compare many rows across all partitions to each other. This is an expensive operation, since data is exchanged across the cluster and new partitions are formed as data is reordered. This process classifies as a shuffle and, as a result, `DataFrame.sort()` counts as wide transformation.

More info: Understanding Apache Spark Shuffle | Philipp Brunenberg

**NO.84** Which of the following describes slots?

- A.** Slots are dynamically created and destroyed in accordance with an executor's workload.
- B.** To optimize I/O performance, Spark stores data on disk in multiple slots.
- C.** A Java Virtual Machine (JVM) working as an executor can be considered as a pool of slots for task execution.
- D.** A slot is always limited to a single core.

Slots are the communication interface for executors and are used for receiving commands and sending results to the driver.

**Answer:** C

Explanation

Slots are the communication interface for executors and are used for receiving commands and sending results to the driver.

Wrong, executors communicate with the driver directly.

Slots are dynamically created and destroyed in accordance with an executor's workload.

No, Spark does not actively create and destroy slots in accordance with the workload. Per executor, slots are made available in accordance with how many cores per executor (property `spark.executor.cores`) and how many CPUs per task (property `spark.task.cpus`) the Spark configuration calls for.

A slot is always limited to a single core.

No, a slot can span multiple cores. If a task would require multiple cores, it would have to be executed through a slot that spans multiple cores.

In Spark documentation, "core" is often used interchangeably with "thread", although "thread" is the more accurate word. A single physical core may be able to make multiple threads available. So, it is better to say that a slot can span multiple threads.

To optimize I/O performance, Spark stores data on disk in multiple slots.

No - Spark stores data on disk in multiple partitions, not slots.

More info: Spark Architecture | Distributed Systems Architecture

**NO.85** Which of the following code blocks returns a copy of DataFrame transactionsDf in which column productId has been renamed to productNumber?

- A. transactionsDf.withColumnRenamed("productId", "productNumber")
- B. transactionsDf.withColumn("productId", "productNumber")
- C. transactionsDf.withColumnRenamed("productNumber", "productId")
- D. transactionsDf.withColumnRenamed(col(productId), col(productNumber))
- E. transactionsDf.withColumnRenamed(productId, productNumber)

**Answer:** A

Explanation

More info: [pyspark.sql.DataFrame.withColumnRenamed](#) - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 2

**NO.86** In which order should the code blocks shown below be run in order to assign articlesDf a DataFrame that lists all items in column attributes ordered by the number of times these items occur, from most to least often?

Sample of DataFrame articlesDf:

```
1. +-----+-----+-----+
2. | itemId | attributes | supplier |
3. +-----+-----+-----+
4. | 1 | [blue, winter, cozy] | Sports Company Inc. |
5. | 2 | [red, summer, fresh, cooling] | YetiX |
6. | 3 | [green, summer, travel] | Sports Company Inc. |
7. +-----+-----+-----+
```

- A. 1. articlesDf = articlesDf.groupby("col")
- 2. articlesDf = articlesDf.select(explode(col("attributes")))
- 3. articlesDf = articlesDf.orderBy("count").select("col")
- 4. articlesDf = articlesDf.sort("count", ascending=False).select("col")
- 5. articlesDf = articlesDf.groupby("col").count()

- B. 4, 5
- C. 2, 5, 3
- D. 5, 2
- E. 2, 3, 4
- F. 2, 5, 4

**Answer:** E

Explanation

Correct code block:

```
articlesDf = articlesDf.select(explode(col('attributes')))
articlesDf = articlesDf.groupby('col').count()
articlesDf = articlesDf.sort('count', ascending=False).select('col')
```

Output of correct code block:

```
+-----+
| col |
+-----+
| summer |
```

```
| winter|
| blue|
| cozy|
| travel|
| fresh|
| red|
| cooling|
| green|
+-----+
```

Static notebook | Dynamic notebook: See test 2

**NO.87** Which of the following code blocks concatenates rows of DataFrames transactionsDf and transactionsNewDf, omitting any duplicates?

- A. transactionsDf.concat(transactionsNewDf).unique()
- B. transactionsDf.union(transactionsNewDf).distinct()
- C. spark.union(transactionsDf, transactionsNewDf).distinct()
- D. transactionsDf.join(transactionsNewDf, how="union").distinct()
- E. transactionsDf.union(transactionsNewDf).unique()

**Answer:** B

Explanation

DataFrame.unique() and DataFrame.concat() do not exist and union() is not a method of the SparkSession. In addition, there is no union option for the join method in the DataFrame.join() statement.

More info: [pyspark.sql.DataFrame.union - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 2

**NO.88** The code block displayed below contains an error. The code block is intended to write DataFrame transactionsDf to disk as a parquet file in location /FileStore/transactions\_split, using column storeId as key for partitioning. Find the error.

Code block:

```
transactionsDf.write.format("parquet").partitionOn("storeId").save("/FileStore/transactions_split")
```

- A. The format("parquet") expression is inappropriate to use here, "parquet" should be passed as first argument to the save() operator and "/FileStore/transactions\_split" as the second argument.
- B. Partitioning data by storeId is possible with the partitionBy expression, so partitionOn should be replaced by partitionBy.
- C. Partitioning data by storeId is possible with the bucketBy expression, so partitionOn should be replaced by bucketBy.
- D. partitionOn("storeId") should be called before the write operation.
- E. The format("parquet") expression should be removed and instead, the information should be added to the write expression like so: write("parquet").

**Answer:** B

Explanation

Correct code block:

```
transactionsDf.write.format("parquet").partitionBy("storeId").save("/FileStore/transactions_split")
```

More info: [partition by - Reading files which are written using PartitionBy or BucketBy in Spark - Stack Overflow](#)

k Overflow Static notebook | Dynamic notebook: See test 1

**NO.89** The code block displayed below contains an error. The code block should return a DataFrame in which column predErrorAdded contains the results of Python function add\_2\_if\_geq\_3 as applied to numeric and nullable column predError in DataFrame transactionsDf. Find the error.

Code block:

```
1.def add_2_if_geq_3(x):
2. if x is None:
3. return x
4. elif x >= 3:
5. return x+2
6. return x
7.
8.add_2_if_geq_3_udf = udf(add_2_if_geq_3)
9.
10.transactionsDf.withColumnRenamed("predErrorAdded", add_2_if_geq_3_udf(col("predError")))
```

- A.** The operator used to adding the column does not add column predErrorAdded to the DataFrame.
- B.** Instead of col("predError"), the actual DataFrame with the column needs to be passed, like so transactionsDf.predError.
- C.** The udf() method does not declare a return type.
- D.** UDFs are only available through the SQL API, but not in the Python API as shown in the code block.
- E.** The Python function is unable to handle null values, resulting in the code block crashing on execution.

**Answer:** A

Explanation

Correct code block:

```
def add_2_if_geq_3(x):
 if x is None:
 return x
 elif x >= 3:
 return x+2
 return x
add_2_if_geq_3_udf = udf(add_2_if_geq_3)
transactionsDf.withColumn("predErrorAdded", add_2_if_geq_3_udf(col("predError"))).show()
```

Instead of withColumnRenamed, you should use the withColumn operator.

The udf() method does not declare a return type.

It is fine that the udf() method does not declare a return type, this is not a required argument.

However, the default return type is StringType. This may not be the ideal return type for numeric, nullable data - but the code will run without specified return type nevertheless.

The Python function is unable to handle null values, resulting in the code block crashing on execution.

The Python function is able to handle null values, this is what the statement if x is None does.

UDFs are only available through the SQL API, but not in the Python API as shown in the code block.

No, they are available through the Python API. The code in the code block that concerns UDFs is correct.

Instead of col("predError"), the actual DataFrame with the column needs to be passed, like so

transactionsDf.predError.

You may choose to use the transactionsDf.predError syntax, but the col("predError") syntax is fine.

**NO.90** Which of the following code blocks returns a copy of DataFrame transactionsDf that only includes columns transactionId, storeId, productId and f?

Sample of DataFrame transactionsDf:

```
1. +-----+-----+----+-----+-----+----+
2. | transactionId | predError | value | storeId | productId | f |
3. +-----+-----+----+-----+-----+----+
4. | 1 | 3 | 4 | 25 | 1 | null |
5. | 2 | 6 | 7 | 2 | 2 | null |
6. | 3 | 3 | null | 25 | 3 | null |
7. +-----+-----+----+-----+-----+----+
```

- A. transactionsDf.drop(col("value"), col("predError"))
- B. transactionsDf.drop("predError", "value")
- C. transactionsDf.drop(value, predError)
- D. transactionsDf.drop(["predError", "value"])
- E. transactionsDf.drop([col("predError"), col("value")])

**Answer:** B

Explanation

Output of correct code block:

```
+-----+-----+-----+----+
| transactionId | storeId | productId | f |
+-----+-----+-----+----+
1	25	1	null
2	2	2	null
3	25	3	null
+-----+-----+-----+----+
```

To solve this question, you should be familiar with the drop() API. The order of column names does not matter

- in this question the order differs in some answers just to confuse you. Also, drop() does not take a list. The \*cols operator in the documentation means that all arguments passed to drop() are interpreted as column names.

More info: [pyspark.sql.DataFrame.drop - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 2

**NO.91** Which of the following code blocks returns a new DataFrame with only columns predError and values of every second row of DataFrame transactionsDf?

Entire DataFrame transactionsDf:

```
1. +-----+-----+----+-----+-----+----+
2. | transactionId | predError | value | storeId | productId | f |
3. +-----+-----+----+-----+-----+----+
4. | 1 | 3 | 4 | 25 | 1 | null |
5. | 2 | 6 | 7 | 2 | 2 | null |
6. | 3 | 3 | null | 25 | 3 | null |
7. | 4 | null | null | 3 | 2 | null |
```

8. | 5 | null | null | null | 2 | null |

9. | 6 | 3 | 2 | 25 | 2 | null |

10. +-----+-----+-----+-----+-----+-----+

- A. transactionsDf.filter(col("transactionId").isin([3,4,6])).select([predError, value])
- B. transactionsDf.select(col("transactionId").isin([3,4,6]), "predError", "value")
- C. transactionsDf.filter("transactionId" % 2 == 0).select("predError", "value")
- D. transactionsDf.filter(col("transactionId") % 2 == 0).select("predError", "value") (Correct)
- E. 1.transactionsDf.createOrReplaceTempView("transactionsDf")  
2.spark.sql("FROM transactionsDf SELECT predError, value WHERE transactionId % 2 = 2")
- F. transactionsDf.filter(col(transactionId).isin([3,4,6]))

**Answer:** D

Explanation

Output of correct code block:

```
+-----+-----+
|predError|value|
+-----+-----+
6	7
null	null
3	2
+-----+-----+
```

This is not an easy question to solve. You need to know that % stands for the module operator in Python. % 2 will return true for every second row. The statement using spark.sql gets it almost right (the modulo operator exists in SQL as well), but % 2 = 2 will never yield true, since modulo 2 is either 0 or 1.

Other answers are wrong since they are missing quotes around the column names and/or use filter or select incorrectly.

If you have any doubts about SparkSQL and answer options 3 and 4 in this question, check out the notebook I created as a response to a related student question.

Static notebook | Dynamic notebook: See test 1

**NO.92** Which of the following describes the difference between client and cluster execution modes?

- A. In cluster mode, the driver runs on the worker nodes, while the client mode runs the driver on the client machine.
- B. In cluster mode, the driver runs on the edge node, while the client mode runs the driver in a worker node.
- C. In cluster mode, each node will launch its own executor, while in client mode, executors will exclusively run on the client machine.
- D. In client mode, the cluster manager runs on the same host as the driver, while in cluster mode, the cluster manager runs on a separate node.
- E. In cluster mode, the driver runs on the master node, while in client mode, the driver runs on a virtual machine in the cloud.

**Answer:** A

Explanation

In cluster mode, the driver runs on the master node, while in client mode, the driver runs on a virtual machine in the cloud.

This is wrong, since execution modes do not specify whether workloads are run in the cloud or on-premise.

In cluster mode, each node will launch its own executor, while in client mode, executors will exclusively run on the client machine.

Wrong, since in both cases executors run on worker nodes.

In cluster mode, the driver runs on the edge node, while the client mode runs the driver in a worker node.

Wrong - in cluster mode, the driver runs on a worker node. In client mode, the driver runs on the client machine.

In client mode, the cluster manager runs on the same host as the driver, while in cluster mode, the cluster manager runs on a separate node.

No. In both modes, the cluster manager is typically on a separate node - not on the same host as the driver. It only runs on the same host as the driver in local execution mode.

More info: Learning Spark, 2nd Edition, Chapter 1, and Spark: The Definitive Guide, Chapter 15. ()

**NO.93** Which of the following code blocks returns all unique values across all values in columns value and productId in DataFrame transactionsDf in a one-column DataFrame?

- A.** `transactionsDf.select('value').join(transactionsDf.select('productId'), col('value')==col('productId'), 'outer')`
- B.** `transactionsDf.select(col('value'), col('productId')).agg({'*': 'count'})`
- C.** `transactionsDf.select('value', 'productId').distinct()`
- D.** `transactionsDf.select('value').union(transactionsDf.select('productId')).distinct()`
- E.** `transactionsDf.agg({'value': 'collect_set', 'productId': 'collect_set'})`

**Answer:** D

Explanation

`transactionsDf.select('value').union(transactionsDf.select('productId')).distinct()` Correct. This code block uses a common pattern for finding the unique values across multiple columns: union and distinct. In fact, it is so common that it is even mentioned in the Spark documentation for the union command (link below).

`transactionsDf.select('value', 'productId').distinct()`

Wrong. This code block returns unique rows, but not unique values.

`transactionsDf.agg({'value': 'collect_set', 'productId': 'collect_set'})` Incorrect. This code block will output a one-row, two-column DataFrame where each cell has an array of unique values in the respective column (even omitting any nulls).

`transactionsDf.select(col('value'), col('productId')).agg({'*': 'count'})` No. This command will count the number of rows, but will not return unique values.

`transactionsDf.select('value').join(transactionsDf.select('productId'), col('value')==col('productId'), 'outer')` Wrong. This command will perform an outer join of the value and productId columns. As such, it will return a two-column DataFrame. If you picked this answer, it might be a good idea for you to read up on the difference between union and join, a link is posted below.

More info: [pyspark.sql.DataFrame.union - PySpark 3.1.2 documentation](#), [sql - What is the difference between JOIN and UNION? - Stack Overflow Static notebook](#) | [Dynamic notebook](#): See test 3

**NO.94** Which of the following is a viable way to improve Spark's performance when dealing with large amounts of data, given that there is only a single application running on the cluster?

- A.** Increase values for the properties `spark.default.parallelism` and `spark.sql.shuffle.partitions`

- B. Decrease values for the properties `spark.default.parallelism` and `spark.sql.partitions`
- C. Increase values for the properties `spark.sql.parallelism` and `spark.sql.partitions`
- D. Increase values for the properties `spark.sql.parallelism` and `spark.sql.shuffle.partitions`
- E. Increase values for the properties `spark.dynamicAllocation.maxExecutors`, `spark.default.parallelism`, and `spark.sql.shuffle.partitions`

**Answer:** A

Explanation

Decrease values for the properties `spark.default.parallelism` and `spark.sql.partitions` No, these values need to be increased.

Increase values for the properties `spark.sql.parallelism` and `spark.sql.partitions` Wrong, there is no property `spark.sql.parallelism`.

Increase values for the properties `spark.sql.parallelism` and `spark.sql.shuffle.partitions` See above.

Increase values for the properties `spark.dynamicAllocation.maxExecutors`, `spark.default.parallelism`, and `spark.sql.shuffle.partitions` The property `spark.dynamicAllocation.maxExecutors` is only in effect if dynamic allocation is enabled, using the `spark.dynamicAllocation.enabled` property. It is disabled by default. Dynamic allocation can be useful when to run multiple applications on the same cluster in parallel. However, in this case there is only a single application running on the cluster, so enabling dynamic allocation would not yield a performance benefit.

More info: Practical Spark Tips For Data Scientists | Experfy.com and Basics of Apache Spark Configuration Settings | by Halil Ertan | Towards Data Science (<https://bit.ly/3gA0A6w> , <https://bit.ly/2QxhNTr>)

**NO.95** Which of the following code blocks performs an inner join between DataFrame `itemsDf` and DataFrame `transactionsDf`, using columns `itemId` and `transactionId` as join keys, respectively?

- A. `itemsDf.join(transactionsDf, "inner", itemsDf.itemId == transactionsDf.transactionId)`
- B. `itemsDf.join(transactionsDf, itemId == transactionId)`
- C. `itemsDf.join(transactionsDf, itemsDf.itemId == transactionsDf.transactionId, "inner")`
- D. `itemsDf.join(transactionsDf, "itemsDf.itemId == transactionsDf.transactionId", "inner")`
- E. `itemsDf.join(transactionsDf, col(itemsDf.itemId) == col(transactionsDf.transactionId))`

**Answer:** C

Explanation

More info: `pyspark.sql.DataFrame.join` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 2

**NO.96** The code block displayed below contains an error. The code block should save DataFrame `transactionsDf` at path `path` as a parquet file, appending to any existing parquet file. Find the error. Code block:

- A. `transactionsDf.format("parquet").option("mode", "append").save(path)`
- B. The code block is missing a reference to the `DataFrameWriter`.
- C. `save()` is evaluated lazily and needs to be followed by an action.
- D. The mode option should be omitted so that the command uses the default mode.
- E. The code block is missing a `bucketBy` command that takes care of partitions.
- F. Given that the DataFrame should be saved as parquet file, `path` is being passed to the wrong method.

**Answer: B**

Explanation

Correct code block:

```
transactionsDf.write.format("parquet").option("mode", "append").save(path)
```

**NO.97** In which order should the code blocks shown below be run in order to create a table of all values in column attributes next to the respective values in column supplier in DataFrame itemsDf?

1. itemsDf.createOrReplaceView("itemsDf")
2. spark.sql("FROM itemsDf SELECT 'supplier', explode('Attributes')")
3. spark.sql("FROM itemsDf SELECT supplier, explode(attributes)")
4. itemsDf.createOrReplaceTempView("itemsDf")

**A.** 4, 3**B.** 1, 3**C.** 2**D.** 4, 2**E.** 1, 2**Answer: A**

Explanation

Static notebook | Dynamic notebook: See test 1

**NO.98** In which order should the code blocks shown below be run in order to create a DataFrame that shows the mean of column predError of DataFrame transactionsDf per column storeId and productId, where productId should be either 2 or 3 and the returned DataFrame should be sorted in ascending order by column storeId, leaving out any nulls in that column?

DataFrame transactionsDf:

1. +-----+-----+----+-----+-----+----+
2. | transactionId | predError | value | storeId | productId | f |
3. +-----+-----+----+-----+-----+----+
4. | 1 | 3 | 4 | 25 | 1 | null |
5. | 2 | 6 | 7 | 2 | 2 | null |
6. | 3 | 3 | null | 25 | 3 | null |
7. | 4 | null | null | 3 | 2 | null |
8. | 5 | null | null | null | 2 | null |
9. | 6 | 3 | 2 | 25 | 2 | null |
10. +-----+-----+----+-----+-----+----+
1. .mean("predError")
2. .groupBy("storeId")
3. .orderBy("storeId")
4. transactionsDf.filter(transactionsDf.storeId.isNotNull())
5. .pivot("productId", [2, 3])

**A.** 4, 5, 2, 3, 1**B.** 4, 2, 1**C.** 4, 1, 5, 2, 3**D.** 4, 2, 5, 1, 3**E.** 4, 3, 2, 5, 1

**Answer: D**

Explanation

Correct code block:

```
transactionsDf.filter(transactionsDf.storeId.isNotNull()).groupBy("storeId").pivot("productId", [2, 3]).mean("predError").orderBy("storeId")
```

Output of correct code block:

```
+-----+-----+
|storeId| 2| 3|
+-----+-----+
2	6.0	null
3	null	null
25	3.0	3.0
+-----+-----+
```

This question is quite convoluted and requires you to think hard about the correct order of operations.

The pivot method also makes an appearance - a method that you may not know all that much about (yet).

At the first position in all answers is code block 4, so the question is essentially just about the ordering of the remaining 4 code blocks.

The question states that the returned DataFrame should be sorted by column storeId. So, it should make sense to have code block 3 which includes the orderBy operator at the very end of the code block. This leaves you with only two answer options.

Now, it is useful to know more about the context of pivot in PySpark. A common pattern is groupBy, pivot, and then another aggregating function, like mean. In the documentation linked below you can see that pivot is a method of pyspark.sql.GroupedData - meaning that before pivoting, you have to use groupBy. The only answer option matching this requirement is the one in which code block 2 (which includes groupBy) is stated before code block 5 (which includes pivot).

More info: [pyspark.sql.GroupedData.pivot - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 3

**NO.99** The code block displayed below contains an error. The code block should arrange the rows of DataFrame transactionsDf using information from two columns in an ordered fashion, arranging first by column value, showing smaller numbers at the top and greater numbers at the bottom, and then by column predError, for which all values should be arranged in the inverse way of the order of items in column value. Find the error.

Code block:

```
transactionsDf.orderBy('value', asc_nulls_first(col('predError')))
```

- A.** Two orderBy statements with calls to the individual columns should be chained, instead of having both columns in one orderBy statement.
- B.** Column value should be wrapped by the col() operator.
- C.** Column predError should be sorted in a descending way, putting nulls last.
- D.** Column predError should be sorted by desc\_nulls\_first() instead.
- E.** Instead of orderBy, sort should be used.

**Answer: C**

Explanation

Correct code block:

```
transactionsDf.orderBy('value', desc_nulls_last('predError'))
```

Column predError should be sorted in a descending way, putting nulls last.

Correct! By default, Spark sorts ascending, putting nulls first. So, the inverse sort of the default sort is indeed desc\_nulls\_last.

Instead of orderBy, sort should be used.

No. DataFrame.sort() orders data per partition, it does not guarantee a global order. This is why orderBy is the more appropriate operator here.

Column value should be wrapped by the col() operator.

Incorrect. DataFrame.sort() accepts both string and Column objects.

Column predError should be sorted by desc\_nulls\_first() instead.

Wrong. Since Spark's default sort order matches asc\_nulls\_first(), nulls would have to come last when inverted.

Two orderBy statements with calls to the individual columns should be chained, instead of having both columns in one orderBy statement.

No, this would just sort the DataFrame by the very last column, but would not take information from both columns into account, as noted in the question.

More info: pyspark.sql.DataFrame.orderBy - PySpark 3.1.2 documentation,

pyspark.sql.functions.desc\_nulls\_last - PySpark 3.1.2 documentation, sort() vs orderBy() in Spark |

Towards Data Science Static notebook | Dynamic notebook: See test 3

**NO.100** Which of the following code blocks creates a new 6-column DataFrame by appending the rows of the

6-column DataFrame yesterdayTransactionsDf to the rows of the 6-column DataFrame todayTransactionsDf, ignoring that both DataFrames have different column names?

- A. union(todayTransactionsDf, yesterdayTransactionsDf)
- B. todayTransactionsDf.unionByName(yesterdayTransactionsDf, allowMissingColumns=True)
- C. todayTransactionsDf.unionByName(yesterdayTransactionsDf)
- D. todayTransactionsDf.concat(yesterdayTransactionsDf)
- E. todayTransactionsDf.union(yesterdayTransactionsDf)

**Answer:** E

Explanation

```
todayTransactionsDf.union(yesterdayTransactionsDf)
```

Correct. The union command appends rows of yesterdayTransactionsDf to the rows of todayTransactionsDf, ignoring that both DataFrames have different column names. The resulting DataFrame will have the column names of DataFrame todayTransactionsDf.

```
todayTransactionsDf.unionByName(yesterdayTransactionsDf)
```

No. unionByName specifically tries to match columns in the two DataFrames by name and only appends values in columns with identical names across the two DataFrames. In the form presented above, the command is a great fit for joining DataFrames that have exactly the same columns, but in a different order. In this case though, the command will fail because the two DataFrames have different columns.

todayTransactionsDf.unionByName(yesterdayTransactionsDf, allowMissingColumns=True) No. The unionByName command is described in the previous explanation. However, with the allowMissingColumns argument set to True, it is no longer an issue that the two DataFrames have different column names. Any columns that do not have a match in the other DataFrame will be filled

with null where there is no value. In the case at hand, the resulting DataFrame will have 7 or more columns though, so it this command is not the right answer.

`union(todayTransactionsDf, yesterdayTransactionsDf)`

No, there is no union method in `pyspark.sql.functions`.

`todayTransactionsDf.concat(yesterdayTransactionsDf)`

Wrong, the DataFrame class does not have a concat method.

More info: `pyspark.sql.DataFrame.union` - PySpark 3.1.2 documentation,

`pyspark.sql.DataFrame.unionByName` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.101** The code block displayed below contains an error. The code block should use Python method `find_most_freq_letter` to find the letter present most in column `itemName` of DataFrame `itemsDf` and return it in a new column `most_frequent_letter`. Find the error.

Code block:

1. `find_most_freq_letter_udf = udf(find_most_freq_letter)`

2. `itemsDf.withColumn("most_frequent_letter", find_most_freq_letter("itemName"))`

**A.** Spark is not using the UDF method correctly.

**B.** The UDF method is not registered correctly, since the return type is missing.

**C.** The "itemName" expression should be wrapped in `col()`.

**D.** UDFs do not exist in PySpark.

**E.** Spark is not adding a column.

**Answer:** A

Explanation

Correct code block:

`find_most_freq_letter_udf = udf(find_most_freq_letter)`

`itemsDf.withColumn("most_frequent_letter", find_most_freq_letter_udf("itemName"))` Spark should use the previously registered `find_most_freq_letter_udf` method here - but it is not doing that in the original codeblock. There, it just uses the non-UDF version of the Python method.

Note that typically, we would have to specify a return type for `udf()`. Except in this case, since the default return type for `udf()` is a string which is what we are expecting here. If we wanted to return an integer variable instead, we would have to register the Python function as UDF using `find_most_freq_letter_udf = udf(find_most_freq_letter, IntegerType())`.

More info: `pyspark.sql.functions.udf` - PySpark 3.1.1 documentation

**NO.102** The code block shown below should return a column that indicates through boolean variables whether rows in DataFrame `transactionsDf` have values greater or equal to 20 and smaller or equal to

30 in column `storeId` and have the value 2 in column `productId`. Choose the answer that correctly fills the blanks in the code block to accomplish this.

`transactionsDf._1__((__2__._3__)_4__(__5__))`

**A.** 1. select

2. `col("storeId")`

3. `between(20, 30)`

4. and

5. `col("productId")==2`

**B.** 1. where  
 2. col("storeId")  
 3. geq(20).leq(30)  
 4. &  
 5. col("productId")==2

**C.** 1. select  
 2. "storeId"  
 3. between(20, 30)  
 4. &&  
 5. col("productId")==2

**D.** 1. select  
 2. col("storeId")  
 3. between(20, 30)  
 4. &&  
 5. col("productId")=2

**E.** 1. select  
 2. col("storeId")  
 3. between(20, 30)  
 4. &  
 5. col("productId")==2

**Answer:** D

Explanation

Correct code block:

transactionsDf.select((col("storeId").between(20, 30)) & (col("productId")==2)) Although this question may make you think that it asks for a filter or where statement, it does not. It asks explicitly to return a column with booleans - this should point you to the select statement.

Another trick here is the rarely used between() method. It exists and resolves to ((storeId >= 20) AND (storeId <= 30)) in SQL. geq() and leq() do not exist.

Another riddle here is how to chain the two conditions. The only valid answer here is &. Operators like && or and are not valid. Other boolean operators that would be valid in Spark are | and.

Static notebook | Dynamic notebook: See test 1

**NO.103** The code block displayed below contains an error. The code block should return all rows of DataFrame transactionsDf, but including only columns storeId and predError. Find the error.

Code block:

```
spark.collect(transactionsDf.select("storeId", "predError"))
```

- A.** Instead of select, DataFrame transactionsDf needs to be filtered using the filter operator.
- B.** Columns storeId and predError need to be represented as a Python list, so they need to be wrapped in brackets ([]).
- C.** The take method should be used instead of the collect method.
- D.** Instead of collect, collectAsRows needs to be called.
- E.** The collect method is not a method of the SparkSession object.

**Answer:** E

Explanation

Correct code block:

```
transactionsDf.select("storeId", "predError").collect()
```

collect() is a method of the DataFrame object.

More info: [pyspark.sql.DataFrame.collect - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 2

**NO.104** Which of the following code blocks returns a DataFrame that matches the multi-column DataFrame itemsDf, except that integer column itemId has been converted into a string column?

- A. itemsDf.withColumn("itemId", convert("itemId", "string"))
- B. itemsDf.withColumn("itemId", col("itemId").cast("string"))
- (Correct)
- C. itemsDf.select(cast("itemId", "string"))
- D. itemsDf.withColumn("itemId", col("itemId").convert("string"))
- E. spark.cast(itemsDf, "itemId", "string")

**Answer:** B

Explanation

```
itemsDf.withColumn("itemId", col("itemId").cast("string"))
```

Correct. You can convert the data type of a column using the cast method of the Column class. Also note that you will have to use the withColumn method on itemsDf for replacing the existing itemId column with the new version that contains strings.

```
itemsDf.withColumn("itemId", col("itemId").convert("string"))
```

Incorrect. The Column object that col("itemId") returns does not have a convert method.

```
itemsDf.withColumn("itemId", convert("itemId", "string"))
```

Wrong. Spark's spark.sql.functions module does not have a convert method. The question is trying to mislead you by using the word "converted". Type conversion is also called "type casting". This may help you remember to look for a cast method instead of a convert method (see correct answer).

```
itemsDf.select(astype("itemId", "string"))
```

False. While astype is a method of Column (and an alias of Column.cast), it is not a method of pyspark.sql.functions (what the code block implies). In addition, the question asks to return a full DataFrame that matches the multi-column DataFrame itemsDf. Selecting just one column from itemsDf as in the code block would just return a single-column DataFrame.

```
spark.cast(itemsDf, "itemId", "string")
```

No, the Spark session (called by spark) does not have a cast method. You can find a list of all methods available for the Spark session linked in the documentation below.

More info:

- [pyspark.sql.Column.cast - PySpark 3.1.2 documentation](#)
- [pyspark.sql.Column.astype - PySpark 3.1.2 documentation](#)
- [pyspark.sql.SparkSession - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 3

**NO.105** Which of the following statements about executors is correct?

- A. Executors are launched by the driver.
- B. Executors stop upon application completion by default.
- C. Each node hosts a single executor.
- D. Executors store data in memory only.

**E.** An executor can serve multiple applications.

**Answer:** B

Explanation

Executors stop upon application completion by default.

Correct. Executors only persist during the lifetime of an application.

A notable exception to that is when Dynamic Resource Allocation is enabled (which it is not by default). With Dynamic Resource Allocation enabled, executors are terminated when they are idle, independent of whether the application has been completed or not.

An executor can serve multiple applications.

Wrong. An executor is always specific to the application. It is terminated when the application completes (exception see above).

Each node hosts a single executor.

No. Each node can host one or more executors.

Executors store data in memory only.

No. Executors can store data in memory or on disk.

Executors are launched by the driver.

Incorrect. Executors are launched by the cluster manager on behalf of the driver.

More info: Job Scheduling - Spark 3.1.2 Documentation, How Applications are Executed on a Spark Cluster | Anatomy of a Spark Application | InformIT, and Spark Jargon for Starters. This blog is to clear some of the... | by Mageswaran D | Medium

**NO.106** The code block displayed below contains an error. The code block should produce a DataFrame with color as the only column and three rows with color values of red, blue, and green, respectively.

Find the error.

Code block:

```
1.spark.createDataFrame([("red",), ("blue",), ("green",)], "color")
```

Instead of calling spark.createDataFrame, just DataFrame should be called.

**A.** The commas in the tuples with the colors should be eliminated.

**B.** The colors red, blue, and green should be expressed as a simple Python list, and not a list of tuples

.

**C.** Instead of color, a data type should be specified.

**D.** The "color" expression needs to be wrapped in brackets, so it reads ["color"].

**Answer:** D

Explanation

Correct code block:

```
spark.createDataFrame([("red",), ("blue",), ("green",)], ["color"])
```

The createDataFrame syntax is not exactly straightforward, but luckily the documentation (linked below) provides several examples on how to use it. It also shows an example very similar to the code block presented here which should help you answer this question correctly.

More info: pyspark.sql.Session.createDataFrame - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 2

**NO.107** Which of the following code blocks returns a DataFrame where columns predError and productId are removed from DataFrame transactionsDf?

Sample of DataFrame transactionsDf:

```
1.+-----+-----+----+-----+-----+----+
2.|transactionId|predError|value|storeId|productId|f|
3.+-----+-----+----+-----+-----+----+
4.|1|3|4|25|1|null|
5.|2|6|7|2|2|null|
6.|3|3|null|25|3|null|
7.+-----+-----+----+-----+-----+----+
```

- A. transactionsDf.withColumnRemoved("predError", "productId")
- B. transactionsDf.drop(["predError", "productId", "associateId"])
- C. transactionsDf.drop("predError", "productId", "associateId")
- D. transactionsDf.dropColumns("predError", "productId", "associateId")
- E. transactionsDf.drop(col("predError", "productId"))

**Answer:** D

Explanation

The key here is to understand that columns that are passed to DataFrame.drop() are ignored if they do not exist in the DataFrame. So, passing column name associateId to transactionsDf.drop() does not have any effect.

Passing a list to transactionsDf.drop() is not valid. The documentation (link below) shows the call structure as DataFrame.drop(\*cols). The \* means that all arguments that are passed to DataFrame.drop() are read as columns. However, since a list of columns, for example ["predError", "productId", "associateId"] is not a column, Spark will run into an error.

More info: [pyspark.sql.DataFrame.drop - PySpark 3.1.1 documentation](#)

Static notebook | Dynamic notebook: See test 1

**NO.108** Which of the following code blocks performs a join in which the small DataFrame transactionsDf is sent to all executors where it is joined with DataFrame itemsDf on columns storeId and itemId, respectively?

- A. itemsDf.join(transactionsDf, itemsDf.itemId == transactionsDf.storeId, "right\_outer")
- B. itemsDf.join(transactionsDf, itemsDf.itemId == transactionsDf.storeId, "broadcast")
- C. itemsDf.merge(transactionsDf, "itemsDf.itemId == transactionsDf.storeId", "broadcast")
- D. itemsDf.join(broadcast(transactionsDf), itemsDf.itemId == transactionsDf.storeId)
- E. itemsDf.join(transactionsDf, broadcast(itemsDf.itemId == transactionsDf.storeId))

**Answer:** D

Explanation

The issue with all answers that have "broadcast" as very last argument is that "broadcast" is not a valid join type. While the entry with "right\_outer" is a valid statement, it is not a broadcast join. The item where broadcast() is wrapped around the equality condition is not valid code in Spark. broadcast() needs to be wrapped around the name of the small DataFrame that should be broadcast.

More info: Learning Spark, 2nd Edition, Chapter 7

Static notebook | Dynamic notebook: See test 1

tion and explanation?

**NO.109** Which of the following code blocks returns a new DataFrame with the same columns as DataFrame transactionsDf, except for columns predError and value which should be removed?

- A. `transactionsDf.drop(["predError", "value"])`
- B. `transactionsDf.drop("predError", "value")`
- C. `transactionsDf.drop(col("predError"), col("value"))`
- D. `transactionsDf.drop(predError, value)`
- E. `transactionsDf.drop("predError & value")`

**Answer:** B

Explanation

More info: [pyspark.sql.DataFrame.drop - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 2

**NO.110** Which of the following code blocks uses a schema fileSchema to read a parquet file at location filePath into a DataFrame?

- A. `spark.read.schema(fileSchema).format("parquet").load(filePath)`
- B. `spark.read.schema("fileSchema").format("parquet").load(filePath)`
- C. `spark.read().schema(fileSchema).parquet(filePath)`
- D. `spark.read().schema(fileSchema).format(parquet).load(filePath)`
- E. `spark.read.schema(fileSchema).open(filePath)`

**Answer:** A

Explanation

Pay attention here to which variables are quoted. fileSchema is a variable and thus should not be in quotes.

parquet is not a variable and therefore should be in quotes.

SparkSession.read (here referenced as spark.read) returns a DataFrameReader which all subsequent calls reference - the DataFrameReader is not callable, so you should not use parentheses here.

Finally, there is no open method in PySpark. The method name is load.

Static notebook | Dynamic notebook: See test 1

**NO.111** The code block displayed below contains an error. The code block should return the average of rows in column value grouped by unique storeId. Find the error.

Code block:

```
transactionsDf.agg("storeId").avg("value")
```

- A. Instead of `avg("value")`, `avg(col("value"))` should be used.
- B. The `avg("value")` should be specified as a second argument to `agg()` instead of being appended to it.
- C. All column names should be wrapped in `col()` operators.
- D. `agg` should be replaced by `groupBy`.
- E. "storeId" and "value" should be swapped.

**Answer:** D

Explanation

Static notebook | Dynamic notebook: See test 1

([https://flrs.github.io/spark\\_practice\\_tests\\_code/#1/30.html](https://flrs.github.io/spark_practice_tests_code/#1/30.html) ,

[https://bit.ly/sparkpracticeexams\\_import\\_instructions](https://bit.ly/sparkpracticeexams_import_instructions))

**NO.112** The code block shown below should write DataFrame transactionsDf to disk at path csvPath

as a single CSV file, using tabs (\t characters) as separators between columns, expressing missing values as string n/a, and omitting a header row with column names. Choose the answer that correctly fills the blanks in the code block to accomplish this.

transactionsDf.\_\_1\_\_.write.\_\_2\_\_(\_\_3\_\_, " ").\_\_4\_\_.\_\_5\_\_(csvPath)

**A.** 1. coalesce(1)

2. option

3. "sep"

4. option("header", True)

5. path

**B.** 1. coalesce(1)

2. option

3. "colsep"

4. option("nullValue", "n/a")

5. path

**C.** 1. repartition(1)

2. option

3. "sep"

4. option("nullValue", "n/a")

5. csv

(Correct)

**D.** 1. csv

2. option

3. "sep"

4. option("emptyValue", "n/a")

5. path

\* 1. repartition(1)

2. mode

3. "sep"

4. mode("nullValue", "n/a")

5. csv

**Answer:** C

Explanation

Correct code block:

transactionsDf.repartition(1).write.option("sep", "\t").option("nullValue", "n/a").csv(csvPath) It is important here to understand that the question specifically asks for writing the DataFrame as a single CSV file. This should trigger you to think about partitions. By default, every partition is written as a separate file, so you need to include repartition(1) into your call. coalesce(1) works here, too! Secondly, the question is very much an invitation to search through the parameters in the Spark documentation that work with DataFrameWriter.csv (link below). You will also need to know that you need an option() statement to apply these parameters.

The final concern is about the general call structure. Once you have called accessed write of your DataFrame, options follow and then you write the DataFrame with csv. Instead of csv(csvPath), you could also use save(csvPath, format='csv') here.

More info: [pyspark.sql.DataFrameWriter.csv - PySpark 3.1.1 documentation](#) Static notebook |

Dynamic notebook: See test 1

**NO.113** Which of the following code blocks reads the parquet file stored at filePath into DataFrame itemsDf, using a valid schema for the sample of itemsDf shown below?

Sample of itemsDf:

```
1. +-----+-----+-----+
2. |itemId|attributes|supplier|
3. +-----+-----+-----+
4. | 1 | [blue, winter, cozy] | Sports Company Inc. |
5. | 2 | [red, summer, fresh, cooling] | YetiX |
6. | 3 | [green, summer, travel] | Sports Company Inc. |
7. +-----+-----+-----+
```

- A.** 1.itemsDfSchema = StructType([  
2. StructField("itemId", IntegerType()),  
3. StructField("attributes", StringType()),  
4. StructField("supplier", StringType())])  
5.  
6.itemsDf = spark.read.schema(itemsDfSchema).parquet(filePath)
- B.** 1.itemsDfSchema = StructType([  
2. StructField("itemId", IntegerType),  
3. StructField("attributes", ArrayType(StringType)),  
4. StructField("supplier", StringType)])  
5.  
6.itemsDf = spark.read.schema(itemsDfSchema).parquet(filePath)
- C.** 1.itemsDf = spark.read.schema('itemId integer, attributes <string>, supplier string').parquet(filePath)
- D.** 1.itemsDfSchema = StructType([  
2. StructField("itemId", IntegerType()),  
3. StructField("attributes", ArrayType(StringType())),  
4. StructField("supplier", StringType())])  
5.  
6.itemsDf = spark.read.schema(itemsDfSchema).parquet(filePath)
- E.** 1.itemsDfSchema = StructType([  
2. StructField("itemId", IntegerType()),  
3. StructField("attributes", ArrayType([StringType()])),  
4. StructField("supplier", StringType())])  
5.  
6.itemsDf = spark.read(schema=itemsDfSchema).parquet(filePath)

**Answer: D**

Explanation

The challenge in this question comes from there being an array variable in the schema. In addition, you should know how to pass a schema to the DataFrameReader that is invoked by spark.read. The correct way to define an array of strings in a schema is through ArrayType(StringType()). A schema can be passed to the DataFrameReader by simply appending schema(structType) to the read() operator. Alternatively, you can also define a schema as a string. For example, for the schema of itemsDf, the following string would make sense: itemId integer, attributes array<string>, supplier string.

A thing to keep in mind is that in schema definitions, you always need to instantiate the types, like so: `StringType()`. Just using `StringType` does not work in pySpark and will fail.

Another concern with schemas is whether columns should be nullable, so allowed to have null values. In the case at hand, this is not a concern however, since the question just asks for a "valid"

schema. Both non-nullable and nullable column schemas would be valid here, since no null value appears in the DataFrame sample.

More info: Learning Spark, 2nd Edition, Chapter 3

Static notebook | Dynamic notebook: See test 3

**NO.114** Which of the elements in the labeled panels represent the operation performed for broadcast variables?

Larger image

**A.** 2, 5

**B.** 3

**C.** 2, 3

**D.** 1, 2

**E.** 1, 3, 4

**Answer:** C

Explanation

2,3

Correct! Both panels 2 and 3 represent the operation performed for broadcast variables. While a broadcast operation may look like panel 3, with the driver being the bottleneck, it most probably looks like panel 2.

This is because the torrent protocol sits behind Spark's broadcast implementation. In the torrent protocol, each executor will try to fetch missing broadcast variables from the driver or other nodes, preventing the driver from being the bottleneck.

1,2

Wrong. While panel 2 may represent broadcasting, panel 1 shows bi-directional communication which does not occur in broadcast operations.

3

No. While broadcasting may materialize like shown in panel 3, its use of the torrent protocol also enables communication as shown in panel 2 (see first explanation).

1,3,4

No. While panel 2 shows broadcasting, panel 1 shows bi-directional communication - not a characteristic of broadcasting. Panel 4 shows uni-directional communication, but in the wrong direction.

Panel 4 resembles more an accumulator variable than a broadcast variable.

2,5

Incorrect. While panel 2 shows broadcasting, panel 5 includes bi-directional communication - not a characteristic of broadcasting.

More info: Broadcast Join with Spark - [henning.kropponline.de](http://henning.kropponline.de)

**NO.115** The code block shown below should return a DataFrame with only columns from DataFrame `transactionsDf` for which there is a corresponding `transactionId` in DataFrame `itemsDf`. DataFrame `itemsDf` is very small and much smaller than DataFrame `transactionsDf`. The query should be

executed in an optimized way. Choose the answer that correctly fills the blanks in the code block to accomplish this.

\_\_1\_\_\_. \_\_2\_\_ ( \_\_3\_\_ , \_\_4\_\_ , \_\_5\_\_ )

**A.** 1. transactionsDf

2. join

3. broadcast(itemsDf)

4. transactionsDf.transactionId==itemsDf.transactionId

5. "outer"

**B.** 1. transactionsDf

2. join

3. itemsDf

4. transactionsDf.transactionId==itemsDf.transactionId

5. "anti"

**C.** 1. transactionsDf

2. join

3. broadcast(itemsDf)

4. "transactionId"

5. "left\_semi"

**D.** 1. itemsDf

2. broadcast

3. transactionsDf

4. "transactionId"

5. "left\_semi"

**E.** 1. itemsDf

2. join

3. broadcast(transactionsDf)

4. "transactionId"

5. "left\_semi"

**Answer:** C

Explanation

Correct code block:

```
transactionsDf.join(broadcast(itemsDf), "transactionId", "left_semi")
```

This question is extremely difficult and exceeds the difficulty of questions in the exam by far.

A first indication of what is asked from you here is the remark that "the query should be executed in an optimized way". You also have qualitative information about the size of itemsDf and transactionsDf. Given that itemsDf is "very small" and that the execution should be optimized, you should consider instructing Spark to perform a broadcast join, broadcasting the "very small" DataFrame itemsDf to all executors. You can explicitly suggest this to Spark via wrapping itemsDf into a broadcast() operator. One answer option does not include this operator, so you can disregard it. Another answer option wraps the broadcast() operator around transactionsDf - the bigger of the two DataFrames. This answer option does not make sense in the optimization context and can likewise be disregarded.

When thinking about the broadcast() operator, you may also remember that it is a method of pyspark.sql.functions. One answer option, however, resolves to itemsDf.broadcast([...]). The DataFrame class has no broadcast() method, so this answer option can be eliminated as well.

All two remaining answer options resolve to `transactionsDf.join(...)` in the first 2 gaps, so you will have to figure out the details of the join now. You can pick between an outer and a left semi join. An outer join would include columns from both DataFrames, where a left semi join only includes columns from the "left" table, here `transactionsDf`, just as asked for by the question. So, the correct answer is the one that uses the `left_semi` join.

**NO.116** The code block displayed below contains an error. When the code block below has executed, it should have divided DataFrame `transactionsDf` into 14 parts, based on columns `storeId` and `transactionDate` (in this order). Find the error.

Code block:

```
transactionsDf.coalesce(14, ("storeId", "transactionDate"))
```

- A.** The parentheses around the column names need to be removed and `.select()` needs to be appended to the code block.
- B.** Operator `coalesce` needs to be replaced by `repartition`, the parentheses around the column names need to be removed, and `.count()` needs to be appended to the code block.  
(Correct)
- C.** Operator `coalesce` needs to be replaced by `repartition`, the parentheses around the column names need to be removed, and `.select()` needs to be appended to the code block.
- D.** Operator `coalesce` needs to be replaced by `repartition` and the parentheses around the column names need to be replaced by square brackets.
- E.** Operator `coalesce` needs to be replaced by `repartition`.

**Answer:** B

Explanation

Correct code block:

```
transactionsDf.repartition(14, "storeId", "transactionDate").count()
```

Since we do not know how many partitions DataFrame `transactionsDf` has, we cannot safely use `coalesce`, since it would not make any change if the current number of partitions is smaller than 14. So, we need to use `repartition`.

In the Spark documentation, the call structure for `repartition` is shown like this:

`DataFrame.repartition(numPartitions, *cols)`. The `*` operator means that any argument after `numPartitions` will be interpreted as column. Therefore, the brackets need to be removed.

Finally, the question specifies that after the execution the DataFrame should be divided. So, indirectly this question is asking us to append an action to the code block. Since `.select()` is a transformation, the only possible choice here is `.count()`.

More info: [pyspark.sql.DataFrame.repartition - PySpark 3.1.1 documentation](#) Static notebook | Dynamic notebook: See test 1

**NO.117** The code block displayed below contains an error. The code block should trigger Spark to cache DataFrame `transactionsDf` in executor memory where available, writing to disk where insufficient executor memory is available, in a fault-tolerant way. Find the error.

Code block:

```
transactionsDf.persist(StorageLevel.MEMORY_AND_DISK)
```

- A.** Caching is not supported in Spark, data are always recomputed.
- B.** Data caching capabilities can be accessed through the spark object, but not through the DataFrame API.

- C.** The storage level is inappropriate for fault-tolerant storage.
- D.** The code block uses the wrong operator for caching.
- E.** The DataFrameWriter needs to be invoked.

**Answer:** C

Explanation

The storage level is inappropriate for fault-tolerant storage.

Correct. Typically, when thinking about fault tolerance and storage levels, you would want to store redundant copies of the dataset. This can be achieved by using a storage level such as `StorageLevel.MEMORY_AND_DISK_2`.

The code block uses the wrong command for caching.

Wrong. In this case, `DataFrame.persist()` needs to be used, since this operator supports passing a storage level.

`DataFrame.cache()` does not support passing a storage level.

Caching is not supported in Spark, data are always recomputed.

Incorrect. Caching is an important component of Spark, since it can help to accelerate Spark programs to great extent. Caching is often a good idea for datasets that need to be accessed repeatedly.

Data caching capabilities can be accessed through the spark object, but not through the DataFrame API.

No. Caching is either accessed through `DataFrame.cache()` or `DataFrame.persist()`.

The DataFrameWriter needs to be invoked.

Wrong. The DataFrameWriter can be accessed via `DataFrame.write` and is used to write data to external data stores, mostly on disk. Here, we find keywords such as "cache" and "executor memory" that point us away from using external data stores. We aim to save data to memory to accelerate the reading process, since reading from disk is comparatively slower. The DataFrameWriter does not write to memory, so we cannot use it here.

More info: [Best practices for caching in Spark SQL](#) | by David Vrba | Towards Data Science

**NO.118** Which of the following code blocks reads in the parquet file stored at location `filePath`, given that all columns in the parquet file contain only whole numbers and are stored in the most appropriate format for this kind of data?

**A.** 1. `spark.read.schema(`  
 2. `StructType(`  
 3. `StructField("transactionId", IntegerType(), True),`  
 4. `StructField("predError", IntegerType(), True)`  
 5. `).load(filePath)`

**B.** 1. `spark.read.schema([`  
 2. `StructField("transactionId", NumberType(), True),`  
 3. `StructField("predError", IntegerType(), True)`  
 4. `]).load(filePath)`

**C.** 1. `spark.read.schema(`  
 2. `StructType([`  
 3. `StructField("transactionId", StringType(), True),`  
 4. `StructField("predError", IntegerType(), True)]`  
 5. `).parquet(filePath)`

**D.** 1.spark.read.schema(  
 2. StructType([  
 3. StructField("transactionId", IntegerType(), True),  
 4. StructField("predError", IntegerType(), True)]  
 5. )).format("parquet").load(filePath)

**E.** 1.spark.read.schema([  
 2. StructField("transactionId", IntegerType(), True),  
 3. StructField("predError", IntegerType(), True)  
 4. ]).load(filePath, format="parquet")

**Answer:** D

Explanation

The schema passed into schema should be of type StructType or a string, so all entries in which a list is passed are incorrect.

In addition, since all numbers are whole numbers, the IntegerType() data type is the correct option here.

NumberType() is not a valid data type and StringType() would fail, since the parquet file is stored in the "most appropriate format for this kind of data", meaning that it is most likely an IntegerType, and Spark does not convert data types if a schema is provided.

Also note that StructType accepts only a single argument (a list of StructFields). So, passing multiple arguments is invalid.

Finally, Spark needs to know which format the file is in. However, all of the options listed are valid here, since Spark assumes parquet as a default when no file format is specifically passed.

More info: [pyspark.sql.DataFrameReader.schema](#) - PySpark 3.1.2 documentation and [StructType](#) - PySpark 3.1.2 documentation

**NO.119** Which of the following code blocks generally causes a great amount of network traffic?

- A.** DataFrame.select()
- B.** DataFrame.coalesce()
- C.** DataFrame.collect()
- D.** DataFrame.rdd.map()
- E.** DataFrame.count()

**Answer:** C

Explanation

DataFrame.collect() sends all data in a DataFrame from executors to the driver, so this generally causes a great amount of network traffic in comparison to the other options listed.

DataFrame.coalesce() just reduces the number of partitions and generally aims to reduce network traffic in comparison to a full shuffle.

DataFrame.select() is evaluated lazily and, unless followed by an action, does not cause significant network traffic.

DataFrame.rdd.map() is evaluated lazily, it does therefore not cause great amounts of network traffic.

DataFrame.count() is an action. While it does cause some network traffic, for the same DataFrame, collecting all data in the driver would generally be considered to cause a greater amount of network traffic.

**NO.120** Which of the following code blocks creates a new DataFrame with 3 columns, productId, highest, and lowest, that shows the biggest and smallest values of column value per value in column productId from DataFrame transactionsDf?

Sample of DataFrame transactionsDf:

```
1.+-----+-----+----+-----+-----+----+
2.| transactionId | predError | value | storeId | productId | f |
3.+-----+-----+----+-----+-----+----+
4.| 1 | 3 | 4 | 25 | 1 | null |
5.| 2 | 6 | 7 | 2 | 2 | null |
6.| 3 | 3 | null | 25 | 3 | null |
7.| 4 | null | null | 3 | 2 | null |
8.| 5 | null | null | null | 2 | null |
9.| 6 | 3 | 2 | 25 | 2 | null |
10.+-----+-----+----+-----+-----+----+
```

- A. transactionsDf.max('value').min('value')
- B. transactionsDf.agg(max('value').alias('highest'), min('value').alias('lowest'))
- C. transactionsDf.groupby(col(productId)).agg(max(col(value)).alias("highest"), min(col(value)).alias("lowest"))
- D. transactionsDf.groupby('productId').agg(max('value').alias('highest'), min('value').alias('lowest'))
- E. transactionsDf.groupby("productId").agg({"highest": max("value"), "lowest": min("value")})

**Answer:** D

Explanation

transactionsDf.groupby('productId').agg(max('value').alias('highest'), min('value').alias('lowest'))

Correct. groupby and aggregate is a common pattern to investigate aggregated values of groups.

transactionsDf.groupby("productId").agg({"highest": max("value"), "lowest": min("value")}) Wrong.

While DataFrame.agg() accepts dictionaries, the syntax of the dictionary in this code block is wrong.

If you use a dictionary, the syntax should be like {"value": "max"}, so using the column name as the key and the aggregating function as value.

transactionsDf.agg(max('value').alias('highest'), min('value').alias('lowest')) Incorrect. While this is valid Spark syntax, it does not achieve what the question asks for. The question specifically asks for values to be aggregated per value in column productId - this column is not considered here. Instead, the max() and min() values are calculated as if the entire DataFrame was a group.

transactionsDf.max('value').min('value')

Wrong. There is no DataFrame.max() method in Spark, so this command will fail.

transactionsDf.groupby(col(productId)).agg(max(col(value)).alias("highest"),

min(col(value)).alias("lowest")) No. While this may work if the column names are expressed as strings, this will not work as is. Python will interpret the column names as variables and, as a result, pySpark will not understand which columns you want to aggregate.

More info: [pyspark.sql.DataFrame.agg - PySpark 3.1.2 documentation](#)

Static notebook | Dynamic notebook: See test 3

**NO.121** Which of the following describes the conversion of a computational query into an execution plan in Spark?

- A. Spark uses the catalog to resolve the optimized logical plan.
- B. The catalog assigns specific resources to the optimized memory plan.

- C. The executed physical plan depends on a cost optimization from a previous stage.
- D. Depending on whether DataFrame API or SQL API are used, the physical plan may differ.
- E. The catalog assigns specific resources to the physical plan.

**Answer:** C

Explanation

The executed physical plan depends on a cost optimization from a previous stage.

Correct! Spark considers multiple physical plans on which it performs a cost analysis and selects the final physical plan in accordance with the lowest-cost outcome of that analysis. That final physical plan is then executed by Spark.

Spark uses the catalog to resolve the optimized logical plan.

No. Spark uses the catalog to resolve the unresolved logical plan, but not the optimized logical plan.

Once the unresolved logical plan is resolved, it is then optimized using the Catalyst Optimizer.

The optimized logical plan is the input for physical planning.

The catalog assigns specific resources to the physical plan.

No. The catalog stores metadata, such as a list of names of columns, data types, functions, and databases.

Spark consults the catalog for resolving the references in a logical plan at the beginning of the conversion of the query into an execution plan. The result is then an optimized logical plan.

Depending on whether DataFrame API or SQL API are used, the physical plan may differ.

Wrong - the physical plan is independent of which API was used. And this is one of the great strengths of Spark!

The catalog assigns specific resources to the optimized memory plan.

There is no specific "memory plan" on the journey of a Spark computation.

More info: Spark's Logical and Physical plans ... When, Why, How and Beyond. | by Laurent Leturgez | datalex | Medium

**NO.122** Which of the elements that are labeled with a circle and a number contain an error or are misrepresented?

- A. 1, 10
- B. 1, 8
- C. 10
- D. 7, 9, 10
- E. 1, 4, 6, 9

**Answer:** B

Explanation

1: Correct - This should just read "API" or "DataFrame API". The DataFrame is not part of the SQL API. To make a DataFrame accessible via SQL, you first need to create a DataFrame view. That view can then be accessed via SQL.

4: Although "K\_38\_INU" looks odd, it is a completely valid name for a DataFrame column.

6: No, StringType is a correct type.

7: Although a StringType may not be the most efficient way to store a phone number, there is nothing fundamentally wrong with using this type here.

8: Correct - TreeType is not a type that Spark supports.

9: No, Spark DataFrames support ArrayType variables. In this case, the variable would represent a sequence of elements with type LongType, which is also a valid type for Spark DataFrames.

10: There is nothing wrong with this row.

More info: Data Types - Spark 3.1.1 Documentation (<https://bit.ly/3aAPKJT>)

**NO.123** The code block displayed below contains an error. The code block should display the schema of DataFrame transactionsDf. Find the error.

Code block:

```
transactionsDf.rdd.printSchema
```

- A.** There is no way to print a schema directly in Spark, since the schema can be printed easily through using `print(transactionsDf.columns)`, so that should be used instead.
- B.** The code block should be wrapped into a `print()` operation.
- C.** `printSchema` is only accessible through the spark session, so the code block should be rewritten as `spark.printSchema(transactionsDf)`.
- D.** `printSchema` is a method and should be written as `printSchema()`. It is also not callable through `transactionsDf.rdd`, but should be called directly from `transactionsDf`.

(Correct)

- E.** `printSchema` is not a method of `transactionsDf.rdd`. Instead, the schema should be printed via `transactionsDf.print_schema()`.

**Answer:** D

Explanation

Correct code block:

```
transactionsDf.printSchema()
```

This is more of a knowledge question that you should just memorize or look up in the provided documentation during the exam. You can get more info about `DataFrame.printSchema()` in the documentation (link below). However - it is a plain simple method without any arguments.

One answer points to an alternative of printing the schema: You could also use `print(transactionsDf.schema)`.

This will give you readable, but not nicely formatted, description of the schema.

More info: `pyspark.sql.DataFrame.printSchema` - PySpark 3.1.1 documentation Static notebook |

Dynamic notebook: See test 1

**NO.124** Which of the following is a problem with using accumulators?

- A.** Only unnamed accumulators can be inspected in the Spark UI.
- B.** Only numeric values can be used in accumulators.
- C.** Accumulator values can only be read by the driver, but not by executors.
- D.** Accumulators do not obey lazy evaluation.
- E.** Accumulators are difficult to use for debugging because they will only be updated once, independent if a task has to be re-run due to hardware failure.

**Answer:** C

Explanation

Accumulator values can only be read by the driver, but not by executors.

Correct. So, for example, you cannot use an accumulator variable for coordinating workloads between executors. The typical, canonical, use case of an accumulator value is to report data, for example for debugging purposes, back to the driver. For example, if you wanted to count values that match a specific condition in a UDF for debugging purposes, an accumulator provides a good way to do that.

Only numeric values can be used in accumulators.

No. While pySpark's Accumulator only supports numeric values (think int and float), you can define accumulators for custom types via the AccumulatorParam interface (documentation linked below).

Accumulators do not obey lazy evaluation.

Incorrect - accumulators do obey lazy evaluation. This has implications in practice: When an accumulator is encapsulated in a transformation, that accumulator will not be modified until a subsequent action is run.

Accumulators are difficult to use for debugging because they will only be updated once, independent if a task has to be re-run due to hardware failure.

Wrong. A concern with accumulators is in fact that under certain conditions they can run for each task more than once. For example, if a hardware failure occurs during a task after an accumulator variable has been increased but before a task has finished and Spark launches the task on a different worker in response to the failure, already executed accumulator variable increases will be repeated.

Only unnamed accumulators can be inspected in the Spark UI.

No. Currently, in PySpark, no accumulators can be inspected in the Spark UI. In the Scala interface of Spark, only named accumulators can be inspected in the Spark UI.

More info: Aggregating Results with Spark Accumulators | Sparkour, RDD Programming Guide - Spark 3.1.2 Documentation, pyspark.Accumulator - PySpark 3.1.2 documentation, and pyspark.AccumulatorParam - PySpark 3.1.2 documentation

**NO.125** Which of the following code blocks returns a new DataFrame in which column attributes of DataFrame itemsDf is renamed to feature0 and column supplier to feature1?

**A.** itemsDf.withColumnRenamed(attributes, feature0).withColumnRenamed(supplier, feature1)

**B.** 1.itemsDf.withColumnRenamed("attributes", "feature0")

2.itemsDf.withColumnRenamed("supplier", "feature1")

**C.** itemsDf.withColumnRenamed(col("attributes"), col("feature0"), col("supplier"), col("feature1"))

**D.** itemsDf.withColumnRenamed("attributes", "feature0").withColumnRenamed("supplier", "feature1")

**E.** itemsDf.withColumn("attributes", "feature0").withColumn("supplier", "feature1")

**Answer: D**

Explanation

itemsDf.withColumnRenamed("attributes", "feature0").withColumnRenamed("supplier", "feature1")

Correct! Spark's DataFrame.withColumnRenamed syntax makes it relatively easy to change the name of a column.

itemsDf.withColumnRenamed(attributes, feature0).withColumnRenamed(supplier, feature1)

Incorrect. In this code block, the Python interpreter will try to use attributes and the other column names as variables. Needless to say, they are undefined, and as a result the block will not run.

itemsDf.withColumnRenamed(col("attributes"), col("feature0"), col("supplier"), col("feature1"))

Wrong. The DataFrame.withColumnRenamed() operator takes exactly two string arguments. So, in this answer both using col() and using four arguments is wrong.

itemsDf.withColumnRenamed("attributes", "feature0")

itemsDf.withColumnRenamed("supplier", "feature1")

No. In this answer, the returned DataFrame will only have column supplier be renamed, since the result of the first line is not written back to itemsDf.

itemsDf.withColumn("attributes", "feature0").withColumn("supplier", "feature1") Incorrect. While withColumn works for adding and naming new columns, you cannot use it to rename existing

columns.

More info: `pyspark.sql.DataFrame.withColumnRenamed` - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 3

**NO.126** The code block displayed below contains at least one error. The code block should return a DataFrame with only one column, result. That column should include all values in column value from DataFrame transactionsDf raised to the power of 5, and a null value for rows in which there is no value in column value. Find the error(s).

Code block:

```
1.from pyspark.sql.functions import udf
2.from pyspark.sql import types as T
3.
4.transactionsDf.createOrReplaceTempView('transactions')
5.
6.def pow_5(x):
7. return x**5
8.
9.spark.udf.register(pow_5, 'power_5_udf', T.LongType())
10.spark.sql('SELECT power_5_udf(value) FROM transactions')
```

**A.** The pow\_5 method is unable to handle empty values in column value and the name of the column in the returned DataFrame is not result.

**B.** The returned DataFrame includes multiple columns instead of just one column.

**C.** The pow\_5 method is unable to handle empty values in column value, the name of the column in the returned DataFrame is not result, and the SparkSession cannot access the transactionsDf DataFrame.

**D.** The pow\_5 method is unable to handle empty values in column value, the name of the column in the returned DataFrame is not result, and Spark driver does not call the UDF function appropriately.

**E.** The pow\_5 method is unable to handle empty values in column value, the UDF function is not registered properly with the Spark driver, and the name of the column in the returned DataFrame is not result.

**Answer:** D

Explanation

Correct code block:

```
from pyspark.sql.functions import udf
from pyspark.sql import types as T
transactionsDf.createOrReplaceTempView('transactions')
def pow_5(x):
 if x:
 return x**5
 return x
spark.udf.register('power_5_udf', pow_5, T.LongType())
spark.sql('SELECT power_5_udf(value) AS result FROM transactions')
```

Here it is important to understand how the pow\_5 method handles empty values. In the wrong code block above, the pow\_5 method is unable to handle empty values and will throw an error, since Python's \*\* operator cannot deal with any null value Spark passes into method pow\_5.

The order of arguments for registering the UDF function with Spark via `spark.udf.register` matters. In the code snippet in the question, the arguments for the SQL method name and the actual Python function are switched. You can read more about the arguments of `spark.udf.register` and see some examples of its usage in the documentation (link below).

Finally, you should recognize that in the original code block, an expression to rename column created through the UDF function is missing. The renaming is done by SQL's AS result argument.

Omitting that argument, you end up with the column name `power_5_udf(value)` and not result.

More info: `pyspark.sql.functions.udf` - PySpark 3.1.1 documentation

**NO.127** Which of the following code blocks reads in the JSON file stored at `filePath` as a DataFrame?

- A. `spark.read.json(filePath)`
- B. `spark.read.path(filePath, source="json")`
- C. `spark.read().path(filePath)`
- D. `spark.read().json(filePath)`
- E. `spark.read.path(filePath)`

**Answer:** A

Explanation

`spark.read.json(filePath)`

Correct. `spark.read` accesses Spark's `DataFrameReader`. Then, Spark identifies the file type to be read as JSON type by passing `filePath` into the `DataFrameReader.json()` method.

`spark.read.path(filePath)`

Incorrect. Spark's `DataFrameReader` does not have a `path` method. A universal way to read in files is provided by the `DataFrameReader.load()` method (link below).

`spark.read.path(filePath, source="json")`

Wrong. A `DataFrameReader.path()` method does not exist (see above).

`spark.read().json(filePath)`

Incorrect. `spark.read` is a way to access Spark's `DataFrameReader`. However, the `DataFrameReader` is not callable, so calling it via `spark.read()` will fail.

`spark.read().path(filePath)`

No, Spark's `DataFrameReader` is not callable (see above).

More info: `pyspark.sql.DataFrameReader.json` - PySpark 3.1.2 documentation,

`pyspark.sql.DataFrameReader.load` - PySpark 3.1.2 documentation Static notebook | Dynamic

notebook: See test 3

**NO.128** Which of the following describes characteristics of the Spark driver?

- A. The Spark driver requests the transformation of operations into DAG computations from the worker nodes.
- B. If set in the Spark configuration, Spark scales the Spark driver horizontally to improve parallel processing performance.
- C. The Spark driver processes partitions in an optimized, distributed fashion.
- D. In a non-interactive Spark application, the Spark driver automatically creates the `SparkSession` object.
- E. The Spark driver's responsibility includes scheduling queries for execution on worker nodes.

**Answer:** D

Explanation

The Spark driver requests the transformation of operations into DAG computations from the worker nodes.

No, the Spark driver transforms operations into DAG computations itself.

If set in the Spark configuration, Spark scales the Spark driver horizontally to improve parallel processing performance.

No. There is always a single driver per application, but one or more executors.

The Spark driver processes partitions in an optimized, distributed fashion.

No, this is what executors do.

In a non-interactive Spark application, the Spark driver automatically creates the SparkSession object.

Wrong. In a non-interactive Spark application, you need to create the SparkSession object. In an interactive Spark shell, the Spark driver instantiates the object for you.

**NO.129** Which of the following code blocks saves DataFrame transactionsDf in location /FileStore/transactions.csv as a CSV file and throws an error if a file already exists in the location?

- A. transactionsDf.write.save("/FileStore/transactions.csv")
- B. transactionsDf.write.format("csv").mode("error").path("/FileStore/transactions.csv")
- C. transactionsDf.write.format("csv").mode("ignore").path("/FileStore/transactions.csv")
- D. transactionsDf.write("csv").mode("error").save("/FileStore/transactions.csv")
- E. transactionsDf.write.format("csv").mode("error").save("/FileStore/transactions.csv")

**Answer:** E

Explanation

Static notebook | Dynamic notebook: See test 1

([https://flrs.github.io/spark\\_practice\\_tests\\_code/#1/28.html](https://flrs.github.io/spark_practice_tests_code/#1/28.html) ,  
[https://bit.ly/sparkpracticeexams\\_import\\_instructions](https://bit.ly/sparkpracticeexams_import_instructions))

**NO.130** The code block displayed below contains an error. The code block should return a DataFrame where all entries in column supplier contain the letter combination et in this order. Find the error.

Code block:

```
itemsDf.filter(Column('supplier').isin('et'))
```

- A. The Column operator should be replaced by the col operator and instead of isin, contains should be used.
- B. The expression inside the filter parenthesis is malformed and should be replaced by isin('et', 'supplier').
- C. Instead of isin, it should be checked whether column supplier contains the letters et, so isin should be replaced with contains. In addition, the column should be accessed using col['supplier'].
- D. The expression only returns a single column and filter should be replaced by select.

**Answer:** B

Explanation

Correct code block:

```
itemsDf.filter(col('supplier').contains('et'))
```

A mixup can easily happen here between isin and contains. Since we want to check whether a column

"contains" the values et, this is the operator we should use here. Note that both methods are methods of Spark's Column object. See below for documentation links.

A specific Column object can be accessed through the `col()` method and not the `Column()` method or through `col[]`, which is an essential thing to know here. In PySpark, Column references a generic column object. To use it for queries, you need to link the generic column object to a specific DataFrame. This can be achieved, for example, through the `col()` method.

More info:

- isin documentation: `pyspark.sql.Column.isin` - PySpark 3.1.1 documentation
- contains documentation: `pyspark.sql.Column.contains` - PySpark 3.1.1 documentation Static notebook | Dynamic notebook: See test 1

**NO.131** The code block displayed below contains an error. The code block should write DataFrame `transactionsDf` as a parquet file to location `filePath` after partitioning it on column `storeId`. Find the error.

Code block:

```
transactionsDf.write.partitionOn("storeId").parquet(filePath)
```

- A.** The partitioning column as well as the file path should be passed to the `write()` method of DataFrame `transactionsDf` directly and not as appended commands as in the code block.
- B.** The `partitionOn` method should be called before the write method.
- C.** The operator should use the `mode()` option to configure the `DataFrameWriter` so that it replaces any existing files at location `filePath`.
- D.** Column `storeId` should be wrapped in a `col()` operator.
- E.** No method `partitionOn()` exists for the DataFrame class, `partitionBy()` should be used instead.

**Answer:** E

Explanation

No method `partitionOn()` exists for the DataFrame class, `partitionBy()` should be used instead.

Correct! Find out more about `partitionBy()` in the documentation (linked below).

The operator should use the `mode()` option to configure the `DataFrameWriter` so that it replaces any existing files at location `filePath`.

No. There is no information about whether files should be overwritten in the question.

The partitioning column as well as the file path should be passed to the `write()` method of DataFrame `transactionsDf` directly and not as appended commands as in the code block.

Incorrect. To write a DataFrame to disk, you need to work with a `DataFrameWriter` object which you get access to through the `DataFrame.writer` property - no parentheses involved.

Column `storeId` should be wrapped in a `col()` operator.

No, this is not necessary - the problem is in the `partitionOn` command (see above).

The `partitionOn` method should be called before the write method.

Wrong. First of all `partitionOn` is not a valid method of DataFrame. However, even assuming `partitionOn` would be replaced by `partitionBy` (which is a valid method), this method is a method of `DataFrameWriter` and not of DataFrame. So, you would always have to first call `DataFrame.write` to get access to the `DataFrameWriter` object and afterwards call `partitionBy`.

More info: `pyspark.sql.DataFrameWriter.partitionBy` - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 3

**NO.132** The code block shown below should convert up to 5 rows in DataFrame `transactionsDf` that have the value 25 in column `storeId` into a Python list. Choose the answer that correctly fills the blanks in the code block to accomplish this.

Code block:

transactionsDf.\_\_1\_\_(\_\_2\_\_).\_\_3\_\_(\_\_4\_\_)

**A.** 1. filter

2. "storeId"==25

3. collect

4. 5

**B.** 1. filter

2. col("storeId")==25

3. toLocalIterator

4. 5

**C.** 1. select

2. storeId==25

3. head

4. 5

**D.** 1. filter

2. col("storeId")==25

3. take

4. 5

**E.** 1. filter

2. col("storeId")==25

3. collect

4. 5

**Answer:** D

Explanation

The correct code block is:

```
transactionsDf.filter(col("storeId")==25).take(5)
```

Any of the options with collect will not work because collect does not take any arguments, and in both cases the argument 5 is given.

The option with toLocalIterator will not work because the only argument to toLocalIterator is prefetchPartitions which is a boolean, so passing 5 here does not make sense.

The option using head will not work because the expression passed to select is not proper syntax. It would work if the expression would be col("storeId")==25.

Static notebook | Dynamic notebook: See test 1

([https://flrs.github.io/spark\\_practice\\_tests\\_code/#1/24.html](https://flrs.github.io/spark_practice_tests_code/#1/24.html) ,

[https://bit.ly/sparkpracticeexams\\_import\\_instructions](https://bit.ly/sparkpracticeexams_import_instructions))

**NO.133** Which of the following code blocks returns a one-column DataFrame of all values in column supplier of DataFrame itemsDf that do not contain the letter X? In the DataFrame, every value should only be listed once.

Sample of DataFrame itemsDf:

1. +-----+-----+-----+-----+-----+

2. | itemId | itemName | attributes | supplier |

3. +-----+-----+-----+-----+-----+

4. | 1 | Thick Coat for Wa... | [blue, winter, cozy] | Sports Company Inc. |

5. | 2 | Elegant Outdoors ... | [red, summer, fre... | YetiX |

6. | 3 | Outdoors Backpack | [green, summer, t... | Sports Company Inc. |

7. +-----+-----+-----+-----+-----+

- A. `itemsDf.filter(col('supplier').not_contains('X')).select(supplier).distinct()`
- B. `itemsDf.select(~col('supplier').contains('X')).distinct()`
- C. `itemsDf.filter(not(col('supplier').contains('X'))).select('supplier').unique()`
- D. `itemsDf.filter(~col('supplier').contains('X')).select('supplier').distinct()`
- E. `itemsDf.filter(!col('supplier').contains('X')).select(col('supplier')).unique()`

**Answer:** D

Explanation

Output of correct code block:

```
+-----+
| supplier|
+-----+
| Sports Company Inc. |
+-----+
```

Key to managing this question is understand which operator to use to do the opposite of an operation

- the ~ (not) operator. In addition, you should know that there is no `unique()` method.

Static notebook | Dynamic notebook: See test 1

**NO.134** Which of the following is one of the big performance advantages that Spark has over Hadoop?

- A. Spark achieves great performance by storing data in the DAG format, whereas Hadoop can only use parquet files.
- B. Spark achieves higher resiliency for queries since, different from Hadoop, it can be deployed on Kubernetes.
- C. Spark achieves great performance by storing data and performing computation in memory, whereas large jobs in Hadoop require a large amount of relatively slow disk I/O operations.
- D. Spark achieves great performance by storing data in the HDFS format, whereas Hadoop can only use parquet files.
- E. Spark achieves performance gains for developers by extending Hadoop's DataFrames with a user-friendly API.

**Answer:** C

Explanation

Spark achieves great performance by storing data in the DAG format, whereas Hadoop can only use parquet files.

Wrong, there is no "DAG format". DAG stands for "directed acyclic graph". The DAG is a means of representing computational steps in Spark. However, it is true that Hadoop does not use a DAG.

The introduction of the DAG in Spark was a result of the limitation of Hadoop's map reduce framework in which data had to be written to and read from disk continuously.

Graph DAG in Apache Spark - DataFlair

Spark achieves great performance by storing data in the HDFS format, whereas Hadoop can only use parquet files.

No. Spark can certainly store data in HDFS (as well as other formats), but this is not a key performance advantage over Hadoop. Hadoop can use multiple file formats, not only parquet.

Spark achieves higher resiliency for queries since, different from Hadoop, it can be deployed on Kubernetes.

No, resiliency is not asked for in the question. The question is about performance improvements.

Both Hadoop and Spark can be deployed on Kubernetes.

Spark achieves performance gains for developers by extending Hadoop's DataFrames with a user-friendly API.

No. DataFrames are a concept in Spark, but not in Hadoop.

**NO.135** The code block shown below should return a two-column DataFrame with columns transactionId and supplier, with combined information from DataFrames itemsDf and transactionsDf. The code block should merge rows in which column productId of DataFrame transactionsDf matches the value of column itemId in DataFrame itemsDf, but only where column storeId of DataFrame transactionsDf does not match column itemId of DataFrame itemsDf. Choose the answer that correctly fills the blanks in the code block to accomplish this.

Code block:

```
transactionsDf.__1__(itemsDf, __2__).__3__(__4__)
```

**A.** 1. join

2. transactionsDf.productId==itemsDf.itemId, how="inner"

3. select

4. "transactionId", "supplier"

**B.** 1. select

2. "transactionId", "supplier"

3. join

4. [transactionsDf.storeId!=itemsDf.itemId, transactionsDf.productId==itemsDf.itemId]

**C.** 1. join

2. [transactionsDf.productId==itemsDf.itemId, transactionsDf.storeId!=itemsDf.itemId]

3. select

4. "transactionId", "supplier"

**D.** 1. filter

2. "transactionId", "supplier"

3. join

4. "transactionsDf.storeId!=itemsDf.itemId, transactionsDf.productId==itemsDf.itemId"

**E.** 1. join

2. transactionsDf.productId==itemsDf.itemId, transactionsDf.storeId!=itemsDf.itemId

3. filter

4. "transactionId", "supplier"

**Answer:** C

Explanation

This question is pretty complex and, in its complexity, is probably above what you would encounter in the exam. However, reading the question carefully, you can use your logic skills to weed out the wrong answers here.

First, you should examine the join statement which is common to all answers. The first argument of the join() operator (documentation linked below) is the DataFrame to be joined with. Where join is in gap 3, the first argument of gap 4 should therefore be another DataFrame. For none of the questions where join is in the third gap, this is the case. So you can immediately discard two answers.

For all other answers, join is in gap 1, followed by .(itemsDf, according to the code block. Given how the join() operator is called, there are now three remaining candidates.

Looking further at the `join()` statement, the second argument (`on=`) expects "a string for the join column name, a list of column names, a join expression (Column), or a list of Columns", according to the documentation. As one answer option includes a list of join expressions (`transactionsDf.productId==itemsDf.itemId, transactionsDf.storeId!=itemsDf.itemId`) which is unsupported according to the documentation, we can discard that answer, leaving us with two remaining candidates.

Both candidates have valid syntax, but only one of them fulfills the condition in the question "only where column `storeId` of DataFrame `transactionsDf` does not match column `itemId` of DataFrame `itemsDf`". So, this one remaining answer option has to be the correct one!

As you can see, although sometimes overwhelming at first, even more complex questions can be figured out by rigorously applying the knowledge you can gain from the documentation during the exam.

More info: `pyspark.sql.DataFrame.join` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.136** Which of the following code blocks returns a single-row DataFrame that only has a column `corr` which shows the Pearson correlation coefficient between columns `predError` and `value` in DataFrame `transactionsDf`?

- A. `transactionsDf.select(corr(["predError", "value"]).alias("corr")).first()`
- B. `transactionsDf.select(corr(col("predError"), col("value")).alias("corr")).first()`
- C. `transactionsDf.select(corr(predError, value).alias("corr"))`
- D. `transactionsDf.select(corr(col("predError"), col("value")).alias("corr"))` (Correct)
- E. `transactionsDf.select(corr("predError", "value"))`

**Answer:** D

Explanation

In difficulty, this question is above what you can expect from the exam. What this question NO: wants to teach you, however, is to pay attention to the useful details included in the documentation. `pyspark.sql.corr` is not a very common method, but it deals with Spark's data structure in an interesting way.

The command takes two columns over multiple rows and returns a single row - similar to an aggregation function. When examining the documentation (linked below), you will find this code example:

```
a = range(20)
b = [2 * x for x in range(20)]
df = spark.createDataFrame(zip(a, b), ["a", "b"])
df.agg(corr("a", "b").alias('c')).collect()
[Row(c=1.0)]
```

See how `corr` just returns a single row? Once you understand this, you should be suspicious about answers that include `first()`, since there is no need to just select a single row. A reason to eliminate those answers is that `DataFrame.first()` returns an object of type `Row`, but not `DataFrame`, as requested in the question.

`transactionsDf.select(corr(col("predError"), col("value")).alias("corr"))` Correct! After calculating the Pearson correlation coefficient, the resulting column is correctly renamed to `corr`.

`transactionsDf.select(corr(predError, value).alias("corr"))`

No. In this answer, Python will interpret column names `predError` and `value` as variable names.

`transactionsDf.select(corr(col("predError"), col("value")).alias("corr")).first()` Incorrect. `first()` returns a

row, not a DataFrame (see above and linked documentation below).

```
transactionsDf.select(corr("predError", "value"))
```

Wrong. While this statement returns a DataFrame in the desired shape, the column will have the name `corr(predError, value)` and not `corr`.

```
transactionsDf.select(corr(["predError", "value"]).alias("corr")).first()
```

False. In addition to `first()` returning a row, this code block also uses the wrong call structure for command `corr` which takes two arguments (the two columns to correlate).

More info:

- `pyspark.sql.functions.corr` - PySpark 3.1.2 documentation

- `pyspark.sql.DataFrame.first` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.137** Which of the following code blocks reads in parquet file `/FileStore/imports.parquet` as a DataFrame?

- A. `spark.mode("parquet").read("/FileStore/imports.parquet")`
- B. `spark.read.path("/FileStore/imports.parquet", source="parquet")`
- C. `spark.read().parquet("/FileStore/imports.parquet")`
- D. `spark.read.parquet("/FileStore/imports.parquet")`
- E. `spark.read().format('parquet').open("/FileStore/imports.parquet")`

**Answer:** D

Explanation

Static notebook | Dynamic notebook: See test 1

([https://flrs.github.io/spark\\_practice\\_tests\\_code/#1/23.html](https://flrs.github.io/spark_practice_tests_code/#1/23.html) ,

[https://bit.ly/sparkpracticeexams\\_import\\_instructions](https://bit.ly/sparkpracticeexams_import_instructions))

**NO.138** The code block shown below should return a copy of DataFrame `transactionsDf` with an added column `cos`.

This column should have the values in column `value` converted to degrees and having the cosine of those converted values taken, rounded to two decimals. Choose the answer that correctly fills the blanks in the code block to accomplish this.

Code block:

```
transactionsDf.__1__(__2__, round(__3__(__4__(__5__)),2))
```

- A. 1. `withColumn`  
2. `col("cos")`  
3. `cos`  
4. `degrees`  
5. `transactionsDf.value`
- B. 1. `withColumnRenamed`  
2. `"cos"`  
3. `cos`  
4. `degrees`  
5. `"transactionsDf.value"`
- C. 1. `withColumn`  
2. `"cos"`  
3. `cos`

- 4. degrees
- 5. transactionsDf.value

**D.** 1. withColumn

- 2. col("cos")
- 3. cos
- 4. degrees
- 5. col("value")

E

- . 1. withColumn
- 2. "cos"
- 3. degrees
- 4. cos
- 5. col("value")

**Answer:** C

Explanation

Correct code block:

`transactionsDf.withColumn("cos", round(cos(degrees(transactionsDf.value)),2))` This question is especially confusing because `col`, `"cos"` are so similar. Similar-looking answer options can also appear in the exam and, just like in this question, you need to pay attention to the details to identify what the correct answer option is.

The first answer option to throw out is the one that starts with `withColumnRenamed`: The question NO:

speaks specifically of adding a column. The `withColumnRenamed` operator only renames an existing column, however, so you cannot use it here.

Next, you will have to decide what should be in gap 2, the first argument of `transactionsDf.withColumn()`.

Looking at the documentation (linked below), you can find out that the first argument of `withColumn` actually needs to be a string with the name of the column to be added. So, any answer that includes `col("cos")` as the option for gap 2 can be disregarded.

This leaves you with two possible answers. The real difference between these two answers is where the `cos` and `degree` methods are, either in gaps 3 and 4, or vice-versa. From the question you can find out that the new column should have "the values in column value converted to degrees and having the cosine of those converted values taken". This prescribes you a clear order of operations: First, you convert values from column value to degrees and then you take the cosine of those values. So, the inner parenthesis (gap 4) should contain the degree method and then, logically, gap 3 holds the `cos` method. This leaves you with just one possible correct answer.

More info: `pyspark.sql.DataFrame.withColumn` - PySpark 3.1.2 documentation Static notebook |

Dynamic notebook: See test 3

**NO.139** The code block displayed below contains an error. The code block should configure Spark to split data in 20 parts when exchanging data between executors for joins or aggregations. Find the error.

Code block:

```
spark.conf.set(spark.sql.shuffle.partitions, 20)
```

**A.** The code block uses the wrong command for setting an option.

**B.** The code block sets the wrong option.

- C.** The code block expresses the option incorrectly.
- D.** The code block sets the incorrect number of parts.
- E.** The code block is missing a parameter.

**Answer:** C

Explanation

Correct code block:

```
spark.conf.set("spark.sql.shuffle.partitions", 20)
```

The code block expresses the option incorrectly.

Correct! The option should be expressed as a string.

The code block sets the wrong option.

No, `spark.sql.shuffle.partitions` is the correct option for the use case in the question.

The code block sets the incorrect number of parts.

Wrong, the code block correctly states 20 parts.

The code block uses the wrong command for setting an option.

No, in PySpark `spark.conf.set()` is the correct command for setting an option.

The code block is missing a parameter.

Incorrect, `spark.conf.set()` takes two parameters.

More info: [Configuration - Spark 3.1.2 Documentation](#)

**NO.140** Which of the following code blocks reads all CSV files in directory `filePath` into a single `DataFrame`, with column names defined in the CSV file headers?

Content of directory `filePath`:

1. `_SUCCESS`
2. `_committed_2754546451699747124`
3. `_started_2754546451699747124`
4. `part-00000-tid-2754546451699747124-10eb85bf-8d91-4dd0-b60b-2f3c02eeecaa-298-1-c000.csv.gz`
5. `part-00001-tid-2754546451699747124-10eb85bf-8d91-4dd0-b60b-2f3c02eeecaa-299-1-c000.csv.gz`
6. `part-00002-tid-2754546451699747124-10eb85bf-8d91-4dd0-b60b-2f3c02eeecaa-300-1-c000.csv.gz`
7. `part-00003-tid-2754546451699747124-10eb85bf-8d91-4dd0-b60b-2f3c02eeecaa-301-1-c000.csv.gz`

**A.** `spark.read.format("csv").option("header",True).option("compression","zip").load(filePath)`

**B.** `spark.read().option("header",True).load(filePath)`

**C.** `spark.read.format("csv").option("header",True).load(filePath)`

**D.** `spark.read.load(filePath)`

**Answer:** C

Explanation

The files in directory `filePath` are partitions of a `DataFrame` that have been exported using gzip compression.

Spark automatically recognizes this situation and imports the CSV files as separate partitions into a single `DataFrame`. It is, however, necessary to specify that Spark should load the file headers in the CSV with the `header` option, which is set to `False` by default.

**NO.141** Which of the following is the deepest level in Spark's execution hierarchy?

- A. Job
- B. Task
- C. Executor
- D. Slot
- E. Stage

**Answer:** B

Explanation

The hierarchy is, from top to bottom: Job, Stage, Task.

Executors and slots facilitate the execution of tasks, but they are not directly part of the hierarchy. Executors are launched by the driver on worker nodes for the purpose of running a specific Spark application. Slots help Spark parallelize work. An executor can have multiple slots which enable it to process multiple tasks in parallel.

**NO.142** The code block shown below should return a DataFrame with columns transactionId, predError, value, and f from DataFrame transactionsDf. Choose the answer that correctly fills the blanks in the code block to accomplish this.

transactionsDf.\_\_1\_\_(\_\_2\_\_)

- A. 1. filter  
2. "transactionId", "predError", "value", "f"
- B. 1. select  
2. "transactionId, predError, value, f"
- C. 1. select  
2. ["transactionId", "predError", "value", "f"]
- D. 1. where  
2. col("transactionId"), col("predError"), col("value"), col("f")
- E. 1. select  
2. col(["transactionId", "predError", "value", "f"])

**Answer:** C

Explanation

Correct code block:

```
transactionsDf.select(["transactionId", "predError", "value", "f"])
```

The DataFrame.select returns specific columns from the DataFrame and accepts a list as its only argument.

Thus, this is the correct choice here. The option using col(["transactionId", "predError", "value", "f"]) is invalid, since inside col(), one can only pass a single column name, not a list. Likewise, all columns being specified in a single string like "transactionId, predError, value, f" is not valid syntax.

filter and where filter rows based on conditions, they do not control which columns to return.

Static notebook | Dynamic notebook: See test 2

**NO.143** The code block shown below should read all files with the file ending .png in directory path into Spark.

Choose the answer that correctly fills the blanks in the code block to accomplish this.

spark.\_\_1\_\_.\_\_2\_\_(\_\_3\_\_).option(\_\_4\_\_, "\*.png").\_\_5\_\_(path)

- A.** 1. read()  
2. format  
3. "binaryFile"  
4. "recursiveFileLookup"  
5. load
- B.** 1. read  
2. format  
3. "binaryFile"  
4. "pathGlobFilter"  
5. load
- C.** 1. read  
2. format  
3. binaryFile  
4. pathGlobFilter  
5. load
- D.** 1. open  
2. format  
3. "image"  
4. "fileType"  
5. open
- E.** 1. open  
2. as  
3. "binaryFile"  
4. "pathGlobFilter"  
5. load

**Answer:** B

Explanation

Correct code block:

`spark.read.format("binaryFile").option("recursiveFileLookup", "*.png").load(path)` Spark can deal with binary files, like images. Using the `binaryFile` format specification in the `SparkSession`'s `read` API is the way to read in those files. Remember that, to access the `read` API, you need to start the command with `spark.read`. The `pathGlobFilter` option is a great way to filter files by name (and ending). Finally, the path can be specified using the load operator - the open operator shown in one of the answers does not exist.

**NO.144** Which of the following code blocks applies the Python function `to_limit` on column `predError` in table `transactionsDf`, returning a `DataFrame` with columns `transactionId` and `result`?

- A.** 1.`spark.udf.register("LIMIT_FCN", to_limit)`  
2.`spark.sql("SELECT transactionId, LIMIT_FCN(predError) AS result FROM transactionsDf")` (Correct)
- B.** 1.`spark.udf.register("LIMIT_FCN", to_limit)`  
2.`spark.sql("SELECT transactionId, LIMIT_FCN(predError) FROM transactionsDf AS result")`
- C.** 1.`spark.udf.register("LIMIT_FCN", to_limit)`  
2.`spark.sql("SELECT transactionId, to_limit(predError) AS result FROM transactionsDf")`  
`spark.sql("SELECT transactionId, udf(to_limit(predError)) AS result FROM transactionsDf")`
- D.** 1.`spark.udf.register(to_limit, "LIMIT_FCN")`

2.spark.sql("SELECT transactionId, LIMIT\_FCN(predError) AS result FROM transactionsDf")

**Answer:** A

Explanation

spark.udf.register("LIMIT\_FCN", to\_limit)

spark.sql("SELECT transactionId, LIMIT\_FCN(predError) AS result FROM transactionsDf") Correct!

First, you have to register to\_limit as UDF to use it in a sql statement. Then, you can use it under the LIMIT\_FCN name, correctly naming the resulting column result.

spark.udf.register(to\_limit, "LIMIT\_FCN")

spark.sql("SELECT transactionId, LIMIT\_FCN(predError) AS result FROM transactionsDf") No. In this answer, the arguments to spark.udf.register are flipped.

spark.udf.register("LIMIT\_FCN", to\_limit)

spark.sql("SELECT transactionId, to\_limit(predError) AS result FROM transactionsDf") Wrong, this answer does not use the registered LIMIT\_FCN in the sql statement, but tries to access the to\_limit method directly. This will fail, since Spark cannot access it.

spark.sql("SELECT transactionId, udf(to\_limit(predError)) AS result FROM transactionsDf") Incorrect, there is no udf method in Spark's SQL.

spark.udf.register("LIMIT\_FCN", to\_limit)

spark.sql("SELECT transactionId, LIMIT\_FCN(predError) FROM transactionsDf AS result") False. In this answer, the column that results from applying the UDF is not correctly renamed to result.

Static notebook | Dynamic notebook: See test 3

**NO.145** Which of the following describes the role of tasks in the Spark execution hierarchy?

- A.** Tasks are the smallest element in the execution hierarchy.
- B.** Within one task, the slots are the unit of work done for each partition of the data.
- C.** Tasks are the second-smallest element in the execution hierarchy.
- D.** Stages with narrow dependencies can be grouped into one task.
- E.** Tasks with wide dependencies can be grouped into one stage.

**Answer:** A

Explanation

Stages with narrow dependencies can be grouped into one task.

Wrong, tasks with narrow dependencies can be grouped into one stage.

Tasks with wide dependencies can be grouped into one stage.

Wrong, since a wide transformation causes a shuffle which always marks the boundary of a stage. So, you cannot bundle multiple tasks that have wide dependencies into a stage.

Tasks are the second-smallest element in the execution hierarchy.

No, they are the smallest element in the execution hierarchy.

Within one task, the slots are the unit of work done for each partition of the data.

No, tasks are the unit of work done per partition. Slots help Spark parallelize work. An executor can have multiple slots which enable it to process multiple tasks in parallel.

**NO.146** Which of the following code blocks returns a DataFrame that has all columns of DataFrame transactionsDf and an additional column predErrorSquared which is the squared value of column predError in DataFrame transactionsDf?

- A.** transactionsDf.withColumn("predError", pow(col("predErrorSquared"), 2))
- B.** transactionsDf.withColumnRenamed("predErrorSquared", pow(predError, 2))

- C. `transactionsDf.withColumn("predErrorSquared", pow(col("predError"), lit(2)))`
- D. `transactionsDf.withColumn("predErrorSquared", pow(predError, lit(2)))`
- E. `transactionsDf.withColumn("predErrorSquared", "predError"**2)`

**Answer:** C

Explanation

While only one of these code blocks works, the DataFrame API is pretty flexible when it comes to accepting columns into the `pow()` method. The following code blocks would also work:

`transactionsDf.withColumn("predErrorSquared", pow("predError", 2))`

`transactionsDf.withColumn("predErrorSquared", pow("predError", lit(2)))` Static notebook | Dynamic notebook: See test 1 ([https://flrs.github.io/spark\\_practice\\_tests\\_code/#1/26.html](https://flrs.github.io/spark_practice_tests_code/#1/26.html) , [https://bit.ly/sparkpracticeexams\\_import\\_instructions](https://bit.ly/sparkpracticeexams_import_instructions))

**NO.147** Which of the following code blocks can be used to save DataFrame `transactionsDf` to memory only, recalculating partitions that do not fit in memory when they are needed?

- A. `from pyspark import StorageLevel`  
`transactionsDf.cache(StorageLevel.MEMORY_ONLY)`
- B. `transactionsDf.cache()`
- C. `transactionsDf.storage_level('MEMORY_ONLY')`
- D. `transactionsDf.persist()`
- E. `transactionsDf.clear_persist()`
- F. `from pyspark import StorageLevel`  
`transactionsDf.persist(StorageLevel.MEMORY_ONLY)`

**Answer:** F

Explanation

`from pyspark import StorageLevel` `transactionsDf.persist(StorageLevel.MEMORY_ONLY)` Correct. Note that the storage level `MEMORY_ONLY` means that all partitions that do not fit into memory will be recomputed when they are needed.

`transactionsDf.cache()`

This is wrong because the default storage level of `DataFrame.cache()` is `MEMORY_AND_DISK`, meaning that partitions that do not fit into memory are stored on disk.

`transactionsDf.persist()`

This is wrong because the default storage level of `DataFrame.persist()` is `MEMORY_AND_DISK`.

`transactionsDf.clear_persist()`

Incorrect, since `clear_persist()` is not a method of DataFrame.

`transactionsDf.storage_level('MEMORY_ONLY')`

Wrong. `storage_level` is not a method of DataFrame.

More info: RDD Programming Guide - Spark 3.0.0 Documentation, `pyspark.sql.DataFrame.persist` - PySpark

3.0.0 documentation (<https://bit.ly/3sxHLVC> , <https://bit.ly/3j2N6B9>)

**NO.148** The code block shown below should add column `transactionDateForm` to DataFrame `transactionsDf`. The column should express the unix-format timestamps in column `transactionDate` as string type like Apr 26 (Sunday). Choose the answer that correctly fills the blanks in the code block to accomplish this.

`transactionsDf._1__(__2__, from_unixtime(__3__, __4__))`

- A.** 1. withColumn
  - 2. "transactionDateForm"
  - 3. "MMM d (EEEE)"
  - 4. "transactionDate"
- B.** 1. select
  - 2. "transactionDate"
  - 3. "transactionDateForm"
  - 4. "MMM d (EEEE)"
- C.** 1. withColumn
  - 2. "transactionDateForm"
  - 3. "transactionDate"
  - 4. "MMM d (EEEE)"
- D.** 1. withColumn
  - 2. "transactionDateForm"
  - 3. "transactionDate"
  - 4. "MM d (EEE)"
- E.** 1. withColumnRenamed
  - 2. "transactionDate"
  - 3. "transactionDateForm"
  - 4. "MM d (EEE)"

**Answer:** C

Explanation

Correct code block:

`transactionsDf.withColumn("transactionDateForm", from_unixtime("transactionDate", "MMM d (EEEE)"))` The question specifically asks about "adding" a column. In the context of all presented answers, `DataFrame.withColumn()` is the correct command for this. In theory, `DataFrame.select()` could also be used for this purpose, if all existing columns are selected and a new one is added. `DataFrame.withColumnRenamed()` is not the appropriate command, since it can only rename existing columns, but cannot add a new column or change the value of a column.

Once `DataFrame.withColumn()` is chosen, you can read in the documentation (see below) that the first input argument to the method should be the column name of the new column.

The final difficulty is the date format. The question indicates that the date format Apr 26 (Sunday) is desired. The answers give "MMM d (EEEE)" and "MM d (EEE)" as options. It can be hard to know the details of the date format that is used in Spark. Specifically, knowing the differences between MMM and MM is probably not something you deal with every day. But, there is an easy way to remember the difference: M (one letter) is usually the shortest form: 4 for April. MM includes padding: 04 for April. MMM (three letters) is the three-letter month abbreviation: Apr for April. And MMMM is the longest possible form: April. Knowing this four-letter sequence helps you select the correct option here.

More info: `pyspark.sql.DataFrame.withColumn` - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 3

**NO.149** Which of the following code blocks creates a new DataFrame with two columns season and wind\_speed\_ms where column season is of data type string and column wind\_speed\_ms is of data type double?

- A. `spark.DataFrame({"season": ["winter", "summer"], "wind_speed_ms": [4.5, 7.5]})`
- B. `spark.createDataFrame([("summer", 4.5), ("winter", 7.5)], ["season", "wind_speed_ms"])`
- C. 1. `from pyspark.sql import types as T`  
2. `spark.createDataFrame([("summer", 4.5), ("winter", 7.5)], T.StructType([T.StructField("season", T.CharType()), T.StructField("season", T.DoubleType())]))`
- D. `spark.newDataFrame([("summer", 4.5), ("winter", 7.5)], ["season", "wind_speed_ms"])`
- E. `spark.createDataFrame({"season": ["winter", "summer"], "wind_speed_ms": [4.5, 7.5]})`

**Answer: B**

Explanation

`spark.createDataFrame([("summer", 4.5), ("winter", 7.5)], ["season", "wind_speed_ms"])` Correct. This command uses the Spark Session's `createDataFrame` method to create a new `DataFrame`. Notice how rows, columns, and column names are passed in here: The rows are specified as a Python list. Every entry in the list is a new row. Columns are specified as Python tuples (for example `("summer", 4.5)`). Every column is one entry in the tuple.

The column names are specified as the second argument to `createDataFrame()`. The documentation (link below) shows that "when schema is a list of column names, the type of each column will be inferred from data" (the first argument). Since values 4.5 and 7.5 are both float variables, Spark will correctly infer the double type for column `wind_speed_ms`. Given that all values in column "season" contain only strings, Spark will cast the column appropriately as string.

Find out more about `SparkSession.createDataFrame()` via the link below.

`spark.newDataFrame([("summer", 4.5), ("winter", 7.5)], ["season", "wind_speed_ms"])` No, the `SparkSession` does not have a `newDataFrame` method.

`from pyspark.sql import types as T`

`spark.createDataFrame([("summer", 4.5), ("winter", 7.5)], T.StructType([T.StructField("season", T.CharType()), T.StructField("season", T.DoubleType())]))`

No. `pyspark.sql.types` does not have a `CharType` type. See link below for available data types in Spark. `spark.createDataFrame({"season": ["winter", "summer"], "wind_speed_ms": [4.5, 7.5]})` No, this is not correct Spark syntax. If you have considered this option to be correct, you may have some experience with Python's `pandas` package, in which this would be correct syntax. To create a Spark `DataFrame` from a `Pandas DataFrame`, you can simply use `spark.createDataFrame(pandasDf)` where `pandasDf` is the `Pandas DataFrame`.

Find out more about Spark syntax options using the examples in the documentation for `SparkSession.createDataFrame` linked below.

`spark.DataFrame({"season": ["winter", "summer"], "wind_speed_ms": [4.5, 7.5]})` No, the `Spark Session` (indicated by `spark` in the code above) does not have a `DataFrame` method.

More info: `pyspark.sql.SparkSession.createDataFrame` - PySpark 3.1.1 documentation and Data Types - Spark 3.1.2 Documentation Static notebook | Dynamic notebook: See test 1

**NO.150** Which of the following describes Spark's standalone deployment mode?

- A. Standalone mode uses a single JVM to run Spark driver and executor processes.
- B. Standalone mode means that the cluster does not contain the driver.
- C. Standalone mode is how Spark runs on YARN and Mesos clusters.
- D. Standalone mode uses only a single executor per worker per application.
- E. Standalone mode is a viable solution for clusters that run multiple frameworks, not only Spark.

**Answer: D**

**Explanation**

Standalone mode uses only a single executor per worker per application.

This is correct and a limitation of Spark's standalone mode.

Standalone mode is a viable solution for clusters that run multiple frameworks.

Incorrect. A limitation of standalone mode is that Apache Spark must be the only framework running on the cluster. If you would want to run multiple frameworks on the same cluster in parallel, for example Apache Spark and Apache Flink, you would consider the YARN deployment mode.

Standalone mode uses a single JVM to run Spark driver and executor processes.

No, this is what local mode does.

Standalone mode is how Spark runs on YARN and Mesos clusters.

No. YARN and Mesos modes are two deployment modes that are different from standalone mode.

These modes allow Spark to run alongside other frameworks on a cluster. When Spark is run in standalone mode, only the Spark framework can run on the cluster.

Standalone mode means that the cluster does not contain the driver.

Incorrect, the cluster does not contain the driver in client mode, but in standalone mode the driver runs on a node in the cluster.

More info: Learning Spark, 2nd Edition, Chapter 1

**NO.151** Which of the following describes a shuffle?

- A.** A shuffle is a process that is executed during a broadcast hash join.
- B.** A shuffle is a process that compares data across executors.
- C.** A shuffle is a process that compares data across partitions.
- D.** A shuffle is a Spark operation that results from `DataFrame.coalesce()`.
- E.** A shuffle is a process that allocates partitions to executors.

**Answer:** C

**Explanation**

A shuffle is a Spark operation that results from `DataFrame.coalesce()`.

No. `DataFrame.coalesce()` does not result in a shuffle.

A shuffle is a process that allocates partitions to executors.

This is incorrect.

A shuffle is a process that is executed during a broadcast hash join.

No, broadcast hash joins avoid shuffles and yield performance benefits if at least one of the two tables is small in size ( $\leq 10$  MB by default). Broadcast hash joins can avoid shuffles because instead of exchanging partitions between executors, they broadcast a small table to all executors that then perform the rest of the join operation locally.

A shuffle is a process that compares data across executors.

No, in a shuffle, data is compared across partitions, and not executors.

More info: Spark Repartition & Coalesce - Explained (<https://bit.ly/32KF7zS>)

**NO.152** Which of the following code blocks reads JSON file `imports.json` into a `DataFrame`?

- A.** `spark.read().mode("json").path("/FileStore/imports.json")`
- B.** `spark.read.format("json").path("/FileStore/imports.json")`
- C.** `spark.read("json", "/FileStore/imports.json")`
- D.** `spark.read.json("/FileStore/imports.json")`
- E.** `spark.read().json("/FileStore/imports.json")`

**Answer: D**

Explanation

Static notebook | Dynamic notebook: See test 1

([https://flrs.github.io/spark\\_practice\\_tests\\_code/#1/25.html](https://flrs.github.io/spark_practice_tests_code/#1/25.html) ,  
[https://bit.ly/sparkpracticeexams\\_import\\_instructions](https://bit.ly/sparkpracticeexams_import_instructions))

**NO.153** The code block displayed below contains an error. The code block is intended to return all columns of DataFrame transactionsDf except for columns predError, productId, and value. Find the error.

Excerpt of DataFrame transactionsDf:

```
transactionsDf.select(~col("predError"), ~col("productId"), ~col("value"))
```

- A.** The select operator should be replaced by the drop operator and the arguments to the drop operator should be column names predError, productId and value wrapped in the col operator so they should be expressed like drop(col(predError), col(productId), col(value)).
- B.** The select operator should be replaced with the deselect operator.
- C.** The column names in the select operator should not be strings and wrapped in the col operator, so they should be expressed like select(~col(predError), ~col(productId), ~col(value)).
- D.** The select operator should be replaced by the drop operator.
- E.** The select operator should be replaced by the drop operator and the arguments to the drop operator should be column names predError, productId and value as strings.

(Correct)

**Answer: E**

Explanation

Correct code block:

```
transactionsDf.drop("predError", "productId", "value")
```

Static notebook | Dynamic notebook: See test 1

**NO.154** The code block displayed below contains an error. The code block should return a new DataFrame that only contains rows from DataFrame transactionsDf in which the value in column predError is at least 5. Find the error.

Code block:

```
transactionsDf.where("col(predError) >= 5")
```

- A.** The argument to the where method should be "predError >= 5".
- B.** Instead of where(), filter() should be used.
- C.** The expression returns the original DataFrame transactionsDf and not a new DataFrame. To avoid this, the code block should be transactionsDf.toNewDataFrame().where("col(predError) >= 5").
- D.** The argument to the where method cannot be a string.
- E.** Instead of >=, the SQL operator GEQ should be used.

**Answer: A**

Explanation

The argument to the where method cannot be a string.

It can be a string, no problem here.

Instead of where(), filter() should be used.

No, that does not matter. In PySpark, where() and filter() are equivalent.

Instead of &gt;=, the SQL operator GEQ should be used.

Incorrect.

The expression returns the original DataFrame transactionsDf and not a new DataFrame. To avoid this, the code block should be `transactionsDf.toNewDataFrame().where("col(predError) >= 5")`.

No, Spark returns a new DataFrame.

Static notebook | Dynamic notebook: See test 1

([https://flrs.github.io/spark\\_practice\\_tests\\_code/#1/27.html](https://flrs.github.io/spark_practice_tests_code/#1/27.html) ,  
[https://bit.ly/sparkpracticeexams\\_import\\_instructions](https://bit.ly/sparkpracticeexams_import_instructions))

**NO.155** Which of the following statements about reducing out-of-memory errors is incorrect?

- A.** Concatenating multiple string columns into a single column may guard against out-of-memory errors.
- B.** Reducing partition size can help against out-of-memory errors.
- C.** Limiting the amount of data being automatically broadcast in joins can help against out-of-memory errors.
- D.** Setting a limit on the maximum size of serialized data returned to the driver may help prevent out-of-memory errors.
- E.** Decreasing the number of cores available to each executor can help against out-of-memory errors.

**Answer:** A

Explanation

Concatenating multiple string columns into a single column may guard against out-of-memory errors. Exactly, this is an incorrect answer! Concatenating any string columns does not reduce the size of the data, it just structures it a different way. This does little to how Spark processes the data and definitely does not reduce out-of-memory errors.

Reducing partition size can help against out-of-memory errors.

No, this is not incorrect. Reducing partition size is a viable way to aid against out-of-memory errors, since executors need to load partitions into memory before processing them. If the executor does not have enough memory available to do that, it will throw an out-of-memory error. Decreasing partition size can therefore be very helpful for preventing that.

Decreasing the number of cores available to each executor can help against out-of-memory errors.

No, this is not incorrect. To process a partition, this partition needs to be loaded into the memory of an executor. If you imagine that every core in every executor processes a partition, potentially in parallel with other executors, you can imagine that memory on the machine hosting the executors fills up quite quickly. So, memory usage of executors is a concern, especially when multiple partitions are processed at the same time. To strike a balance between performance and memory usage, decreasing the number of cores may help against out-of-memory errors.

Setting a limit on the maximum size of serialized data returned to the driver may help prevent out-of-memory errors.

No, this is not incorrect. When using commands like `collect()` that trigger the transmission of potentially large amounts of data from the cluster to the driver, the driver may experience out-of-memory errors. One strategy to avoid this is to be careful about using commands like `collect()` that send back large amounts of data to the driver. Another strategy is setting the parameter `spark.driver.maxResultSize`. If data to be transmitted to the driver exceeds the threshold specified by the parameter, Spark will abort the job and therefore prevent an out-of-memory error.

Limiting the amount of data being automatically broadcast in joins can help against out-of-memory errors.

Wrong, this is not incorrect. As part of Spark's internal optimization, Spark may choose to speed up

operations by broadcasting (usually relatively small) tables to executors. This broadcast is happening from the driver, so all the broadcast tables are loaded into the driver first. If these tables are relatively big, or multiple mid-size tables are being broadcast, this may lead to an out-of-memory error. The maximum table size for which Spark will consider broadcasting is set by the `spark.sql.autoBroadcastJoinThreshold` parameter.

More info: Configuration - Spark 3.1.2 Documentation and Spark OOM Error - Closeup. Does the following look familiar when... | by Amit Singh Rathore | The Startup | Medium

**NO.156** The code block displayed below contains an error. The code block below is intended to add a column `itemNameElements` to DataFrame `itemsDf` that includes an array of all words in column `itemName`. Find the error.

Sample of DataFrame `itemsDf`:

```
1. +-----+-----+-----+
2. |itemId|itemName |supplier |
3. +-----+-----+-----+
4. | 1 |Thick Coat for Walking in the Snow|Sports Company Inc. |
5. | 2 |Elegant Outdoors Summer Dress |YetiX |
6. | 3 |Outdoors Backpack |Sports Company Inc. |
7. +-----+-----+-----+
```

Code block:

```
itemsDf.withColumnRenamed("itemNameElements", split("itemName"))
itemsDf.withColumnRenamed("itemNameElements", split("itemName"))
```

- A.** All column names need to be wrapped in the `col()` operator.
- B.** Operator `withColumnRenamed` needs to be replaced with operator `withColumn` and a second argument `","` needs to be passed to the `split` method.
- C.** Operator `withColumnRenamed` needs to be replaced with operator `withColumn` and the `split` method needs to be replaced by the `splitString` method.
- D.** Operator `withColumnRenamed` needs to be replaced with operator `withColumn` and a second argument `" "` needs to be passed to the `split` method.
- E.** The expressions `"itemNameElements"` and `split("itemName")` need to be swapped.

**Answer:** D

Explanation

Correct code block:

```
itemsDf.withColumn("itemNameElements", split("itemName", " "))
```

Output of code block:

```
+-----+-----+-----+-----+
|itemId|itemName |supplier |itemNameElements |
+-----+-----+-----+-----+
1	Thick Coat for Walking in the Snow	Sports Company Inc.	[Thick, Coat, for, Walking, in, the, Snow]
2	Elegant Outdoors Summer Dress	YetiX	[Elegant, Outdoors, Summer, Dress]
3	Outdoors Backpack	Sports Company Inc.	[Outdoors, Backpack]
+-----+-----+-----+-----+ The key to solving
```

this question is that the split method definitely needs a second argument here (also look at the link to the documentation below). Given the values in column itemName in DataFrame itemsDf, this should be a space character " ". This is the character we need to split the words in the column.

More info: `pyspark.sql.functions.split` - PySpark 3.1.1 documentation

Static notebook | Dynamic notebook: See test 1

**NO.157** Which of the following describes how Spark achieves fault tolerance?

- A.** Spark helps fast recovery of data in case of a worker fault by providing the MEMORY\_AND\_DISK storage level option.
- B.** If an executor on a worker node fails while calculating an RDD, that RDD can be recomputed by another executor using the lineage.
- C.** Spark builds a fault-tolerant layer on top of the legacy RDD data system, which by itself is not fault tolerant.
- D.** Due to the mutability of DataFrames after transformations, Spark reproduces them using observed lineage in case of worker node failure.
- E.** Spark is only fault-tolerant if this feature is specifically enabled via the `spark.fault_recovery.enabled` property.

**Answer:** B

Explanation

Due to the mutability of DataFrames after transformations, Spark reproduces them using observed lineage in case of worker node failure.

Wrong - Between transformations, DataFrames are immutable. Given that Spark also records the lineage, Spark can reproduce any DataFrame in case of failure. These two aspects are the key to understanding fault tolerance in Spark.

Spark builds a fault-tolerant layer on top of the legacy RDD data system, which by itself is not fault tolerant.

Wrong. RDD stands for Resilient Distributed Dataset and it is at the core of Spark and not a "legacy system".

It is fault-tolerant by design.

Spark helps fast recovery of data in case of a worker fault by providing the MEMORY\_AND\_DISK storage level option.

This is not true. For supporting recovery in case of worker failures, Spark provides "\_2", "\_3", and so on, storage level options, for example MEMORY\_AND\_DISK\_2. These storage levels are specifically designed to keep duplicates of the data on multiple nodes. This saves time in case of a worker fault, since a copy of the data can be used immediately, vs. having to recompute it first.

Spark is only fault-tolerant if this feature is specifically enabled via the `spark.fault_recovery.enabled` property.

No, Spark is fault-tolerant by design.

**NO.158** The code block shown below should return a new 2-column DataFrame that shows one attribute from column attributes per row next to the associated itemName, for all suppliers in column supplier whose name includes Sports. Choose the answer that correctly fills the blanks in the code block to accomplish this.

Sample of DataFrame itemsDf:

```
1.+-----+-----+-----+-----+
```

```

2. |itemId|itemName |attributes |supplier |
3. +-----+-----+-----+-----+
4. | 1 | Thick Coat for Walking in the Snow |[blue, winter, cozy] | Sports Company Inc. |
5. | 2 | Elegant Outdoors Summer Dress |[red, summer, fresh, cooling]| YetiX |
6. | 3 | Outdoors Backpack |[green, summer, travel] | Sports Company Inc. |
7. +-----+-----+-----+-----+ Code block:
itemsDf.__1__(__2__).select(__3__, __4__)

```

**A.** 1. filter

```

2. col("supplier").isin("Sports")
3. "itemName"
4. explode(col("attributes"))

```

**B.** 1. where

```

2. col("supplier").contains("Sports")
3. "itemName"
4. "attributes"

```

**C.** 1. where

```

2. col(supplier).contains("Sports")
3. explode(attributes)
4. itemName

```

**D.** 1. where

```

2. "Sports".isin(col("Supplier"))
3. "itemName"
4. array_explode("attributes")

```

**E.** 1. filter

```

2. col("supplier").contains("Sports")
3. "itemName"
4. explode("attributes")

```

**Answer:** E

Explanation

Output of correct code block:

```

+-----+-----+
|itemName |col |
+-----+-----+
Thick Coat for Walking in the Snow	blue
Thick Coat for Walking in the Snow	winter
Thick Coat for Walking in the Snow	cozy
Outdoors Backpack	green
Outdoors Backpack	summer
Outdoors Backpack	travel
+-----+-----+

```

The key to solving this question is knowing about Spark's explode operator. Using this operator, you can extract values from arrays into single rows. The following guidance steps through the answers systematically from the first to the last gap. Note that there are many ways to solving the gap questions and filtering out wrong answers, you do not always have to start filtering out from the first gap, but can also exclude some answers based on obvious problems you see with them.

The answers to the first gap present you with two options: filter and where. These two are actually

synonyms in PySpark, so using either of those is fine. The answer options to this gap therefore do not help us in selecting the right answer.

The second gap is more interesting. One answer option includes `"Sports".isin(col("Supplier"))`. This construct does not work, since Python's string does not have an `isin` method. Another option contains `col(supplier)`. Here, Python will try to interpret `supplier` as a variable. We have not set this variable, so this is not a viable answer. Then, you are left with answers options that include `col("supplier").contains("Sports")` and `col("supplier").isin("Sports")`. The question states that we are looking for suppliers whose name includes Sports, so we have to go for the `contains` operator here. We would use the `isin` operator if we wanted to filter out for supplier names that match any entries in a list of supplier names.

Finally, we are left with two answers that fill the third gap both with `"itemName"` and the fourth gap either with `explode("attributes")` or `"attributes"`. While both are correct Spark syntax, only `explode("attributes")` will help us achieve our goal. Specifically, the question asks for one attribute from column attributes per row - this is what the `explode()` operator does.

One answer option also includes `array_explode()` which is not a valid operator in PySpark.

More info: [pyspark.sql.functions.explode](#) - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.159** Which of the following DataFrame methods is classified as a transformation?

- A. `DataFrame.count()`
- B. `DataFrame.show()`
- C. `DataFrame.select()`
- D. `DataFrame.foreach()`
- E. `DataFrame.first()`

**Answer:** C

Explanation

`DataFrame.select()`

Correct, `DataFrame.select()` is a transformation. When the command is executed, it is evaluated lazily and returns an RDD when it is triggered by an action.

`DataFrame.foreach()`

Incorrect, `DataFrame.foreach()` is not a transformation, but an action. The intention of `foreach()` is to apply code to each element of a DataFrame to update accumulator variables or write the elements to external storage. The process does not return an RDD - it is an action!

`DataFrame.first()`

Wrong. As an action, `DataFrame.first()` executed immediately and returns the first row of a DataFrame.

`DataFrame.count()`

Incorrect. `DataFrame.count()` is an action and returns the number of rows in a DataFrame.

`DataFrame.show()`

No, `DataFrame.show()` is an action and displays the DataFrame upon execution of the command.

**NO.160** Which of the following statements about RDDs is incorrect?

- A. An RDD consists of a single partition.
- B. The high-level DataFrame API is built on top of the low-level RDD API.
- C. RDDs are immutable.

- D.** RDD stands for Resilient Distributed Dataset.
- E.** RDDs are great for precisely instructing Spark on how to do a query.

**Answer:** A

Explanation

An RDD consists of a single partition.

Quite the opposite: Spark partitions RDDs and distributes the partitions across multiple nodes.

**NO.161** Which of the following describes properties of a shuffle?

- A.** Operations involving shuffles are never evaluated lazily.
- B.** Shuffles involve only single partitions.
- C.** Shuffles belong to a class known as "full transformations".
- D.** A shuffle is one of many actions in Spark.
- E.** In a shuffle, Spark writes data to disk.

**Answer:** E

Explanation

In a shuffle, Spark writes data to disk.

Correct! Spark's architecture dictates that intermediate results during a shuffle are written to disk.

A shuffle is one of many actions in Spark.

Incorrect. A shuffle is a transformation, but not an action.

Shuffles involve only single partitions.

No, shuffles involve multiple partitions. During a shuffle, Spark generates output partitions from multiple input partitions.

Operations involving shuffles are never evaluated lazily.

Wrong. A shuffle is a costly operation and Spark will evaluate it as lazily as other transformations.

This is, until a subsequent action triggers its evaluation.

Shuffles belong to a class known as "full transformations".

Not quite. Shuffles belong to a class known as "wide transformations". "Full transformation" is not a relevant term in Spark.

More info: Spark - The Definitive Guide, Chapter 2 and Spark: disk I/O on stage boundaries explanation - Stack Overflow

**NO.162** Which of the following statements about Spark's DataFrames is incorrect?

- A.** Spark's DataFrames are immutable.
- B.** Spark's DataFrames are equal to Python's DataFrames.
- C.** Data in DataFrames is organized into named columns.
- D.** RDDs are at the core of DataFrames.
- E.** The data in DataFrames may be split into multiple chunks.

**Answer:** B

Explanation

Spark's DataFrames are equal to Python's or R's DataFrames.

No, they are not equal. They are only similar. A major difference between Spark and Python is that Spark's DataFrames are distributed, whereby Python's are not.

**NO.163** Which of the following code blocks returns approximately 1000 rows, some of them potentially being duplicates, from the 2000-row DataFrame transactionsDf that only has unique

rows?

- A. `transactionsDf.sample(True, 0.5)`
- B. `transactionsDf.take(1000).distinct()`
- C. `transactionsDf.sample(False, 0.5)`
- D. `transactionsDf.take(1000)`
- E. `transactionsDf.sample(True, 0.5, force=True)`

**Answer:** A

Explanation

To solve this question, you need to know that `DataFrame.sample()` is not guaranteed to return the exact fraction of the number of rows specified as an argument. Furthermore, since duplicates may be returned, you should understand that the operator's `withReplacement` argument should be set to `True`. A `force=` argument for the operator does not exist.

While the `take` argument returns an exact number of rows, it will just take the first specified number of rows (1000 in this question) from the `DataFrame`. Since the `DataFrame` does not include duplicate rows, there is no potential of any of those returned rows being duplicates when using `take()`, so the correct answer cannot involve `take()`.

More info: `pyspark.sql.DataFrame.sample` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 2

**NO.164** The code block displayed below contains an error. The code block should read the csv file located at path `data/transactions.csv` into `DataFrame transactionsDf`, using the first row as column header and casting the columns in the most appropriate type. Find the error.

First 3 rows of `transactions.csv`:

```
1.transactionId;storeId;productId;name
2.1;23;12;green grass
3.2;35;31;yellow sun
4.3;23;12;green grass
```

Code block:

```
transactionsDf = spark.read.load("data/transactions.csv", sep=";", format="csv", header=True)
```

- A. The `DataFrameReader` is not accessed correctly.
- B. The transaction is evaluated lazily, so no file will be read.
- C. Spark is unable to understand the file type.
- D. The code block is unable to capture all columns.
- E. The resulting `DataFrame` will not have the appropriate schema.

**Answer:** E

Explanation

Correct code block:

```
transactionsDf = spark.read.load("data/transactions.csv", sep=";", format="csv", header=True,
inferSchema=True)
```

By default, Spark does not infer the schema of the CSV (since this usually takes some time). So, you need to add the `inferSchema=True` option to the code block.

More info: `pyspark.sql.DataFrameReader.csv` - PySpark 3.1.2 documentation

**NO.165** Which of the following code blocks displays the 10 rows with the smallest values of column value in `DataFrame transactionsDf` in a nicely formatted way?

- A. `transactionsDf.sort(asc(value)).show(10)`

- B. `transactionsDf.sort(col("value")).show(10)`
- C. `transactionsDf.sort(col("value").desc()).head()`
- D. `transactionsDf.sort(col("value").asc()).print(10)`
- E. `transactionsDf.orderBy("value").asc().show(10)`

**Answer:** B

Explanation

`show()` is the correct method to look for here, since the question specifically asks for displaying the rows in a nicely formatted way. Here is the output of `show` (only a few rows shown):

```
+-----+-----+-----+-----+-----+-----+
| transactionId | predError | value | storeId | productId | f | transactionDate |
+-----+-----+-----+-----+-----+-----+
3	3	1	25	3	null	1585824821
5	null	2	null	2	null	1575285427
4	null	3	3	2	null	1583244275
+-----+-----+-----+-----+-----+-----+
```

With regards to the sorting, specifically in ascending order since the smallest values should be shown first, the following expressions are valid:

- `transactionsDf.sort(col("value"))` ("ascending" is the default sort direction in the sort method)
- `transactionsDf.sort(asc(col("value")))`
- `transactionsDf.sort(asc("value"))`
- `transactionsDf.sort(transactionsDf.value.asc())`
- `transactionsDf.sort(transactionsDf.value)`

Also, `orderBy` is just an alias of `sort`, so all of these expressions work equally well using `orderBy`.

Static notebook | Dynamic notebook: See test 1

**NO.166** Which is the highest level in Spark's execution hierarchy?

- A. Task
- B. Executor
- C. Slot
- D. Job
- E. Stage

**Answer:** D

**NO.167** The code block displayed below contains an error. The code block should configure Spark so that DataFrames up to a size of 20 MB will be broadcast to all worker nodes when performing a join. Find the error.

Code block:

- A. `spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 20)`
- B. Spark will only broadcast DataFrames that are much smaller than the default value.
- C. The correct option to write configurations is through `spark.config` and not `spark.conf`.
- D. Spark will only apply the limit to threshold joins and not to other joins.
- E. The passed limit has the wrong variable type.
- F. The command is evaluated lazily and needs to be followed by an action.

**Answer:** B

**Explanation**

This is question is hard. Let's assess the different answers one-by-one.

Spark will only broadcast DataFrames that are much smaller than the default value.

This is correct. The default value is 10 MB (10485760 bytes). Since the configuration for `spark.sql.autoBroadcastJoinThreshold` expects a number in bytes (and not megabytes), the code block sets the limits to merely 20 bytes, instead of the requested  $20 * 1024 * 1024 (= 20971520)$  bytes.

The command is evaluated lazily and needs to be followed by an action.

No, this command is evaluated right away!

Spark will only apply the limit to threshold joins and not to other joins.

There are no "threshold joins", so this option does not make any sense.

The correct option to write configurations is through `spark.config` and not `spark.conf`.

No, it is indeed `spark.conf`!

The passed limit has the wrong variable type.

The configuration expects the number of bytes, a number, as an input. So, the 20 provided in the code block is fine.

**NO.168** Which of the following code blocks returns a DataFrame with approximately 1,000 rows from the 10,000-row DataFrame `itemsDf`, without any duplicates, returning the same rows even if the code block is run twice?

- A. `itemsDf.sampleBy("row", fractions={0: 0.1}, seed=82371)`
- B. `itemsDf.sample(fraction=0.1, seed=87238)`
- C. `itemsDf.sample(fraction=1000, seed=98263)`
- D. `itemsDf.sample(withReplacement=True, fraction=0.1, seed=23536)`
- E. `itemsDf.sample(fraction=0.1)`

**Answer:** B

**Explanation**

`itemsDf.sample(fraction=0.1, seed=87238)`

Correct. If `itemsDf` has 10,000 rows, this code block returns about 1,000, since `DataFrame.sample()` is never guaranteed to return an exact amount of rows. To ensure you are not returning duplicates, you should leave the `withReplacement` parameter at `False`, which is the default. Since the question specifies that the same rows should be returned even if the code block is run twice, you need to specify a seed. The number passed in the seed does not matter as long as it is an integer.

`itemsDf.sample(withReplacement=True, fraction=0.1, seed=23536)`

Incorrect. While this code block fulfills almost all requirements, it may return duplicates. This is because `withReplacement` is set to `True`.

Here is how to understand what replacement means: Imagine you have a bucket of 10,000 numbered balls and you need to take 1,000 balls at random from the bucket (similar to the problem in the question). Now, if you would take those balls with replacement, you would take a ball, note its number, and put it back into the bucket, meaning the next time you take a ball from the bucket there would be a chance you could take the exact same ball again. If you took the balls without replacement, you would leave the ball outside the bucket and not put it back in as you take the next 999 balls.

`itemsDf.sample(fraction=1000, seed=98263)`

Wrong. The `fraction` parameter needs to have a value between 0 and 1. In this case, it should be 0.1, since

$1,000/10,000 = 0.1$ .

```
itemsDf.sampleBy("row", fractions={0: 0.1}, seed=82371)
```

No, `DataFrame.sampleBy()` is meant for stratified sampling. This means that based on the values in a column in a `DataFrame`, you can draw a certain fraction of rows containing those values from the `DataFrame` (more details linked below). In the scenario at hand, `sampleBy` is not the right operator to use because you do not have any information about any column that the sampling should depend on.

```
itemsDf.sample(fraction=0.1)
```

Incorrect. This code block checks all the boxes except that it does not ensure that when you run it a second time, the exact same rows will be returned. In order to achieve this, you would have to specify a seed.

More info:

- `pyspark.sql.DataFrame.sample` - PySpark 3.1.2 documentation
- `pyspark.sql.DataFrame.sampleBy` - PySpark 3.1.2 documentation
- Types of Samplings in PySpark 3. The explanations of the sampling... | by Pinar Ersoy | Towards Data Science

**NO.169** Which of the following statements about DAGs is correct?

- A.** DAGs help direct how Spark executors process tasks, but are a limitation to the proper execution of a query when an executor fails.
- B.** DAG stands for "Directing Acyclic Graph".
- C.** Spark strategically hides DAGs from developers, since the high degree of automation in Spark means that developers never need to consider DAG layouts.
- D.** In contrast to transformations, DAGs are never lazily executed.
- E.** DAGs can be decomposed into tasks that are executed in parallel.

**Answer:** E

Explanation

DAG stands for "Directing Acyclic Graph".

No, DAG stands for "Directed Acyclic Graph".

Spark strategically hides DAGs from developers, since the high degree of automation in Spark means that developers never need to consider DAG layouts.

No, quite the opposite. You can access DAGs through the Spark UI and they can be of great help when optimizing queries manually.

In contrast to transformations, DAGs are never lazily executed.

DAGs represent the execution plan in Spark and as such are lazily executed when the driver requests the data processed in the DAG.

**NO.170** The code block displayed below contains an error. The code block should count the number of rows that have a `predError` of either 3 or 6. Find the error.

Code block:

```
transactionsDf.filter(col('predError').in([3, 6])).count()
```

- A.** The number of rows cannot be determined with the `count()` operator.
- B.** Instead of `filter`, the `select` method should be used.
- C.** The method used on column `predError` is incorrect.
- D.** Instead of a list, the values need to be passed as single arguments to the `in` operator.
- E.** Numbers 3 and 6 need to be passed as string variables.

**Answer: C**

Explanation

Correct code block:

```
transactionsDf.filter(col('predError').isin([3, 6])).count()
```

The isin method is the correct one to use here - the in method does not exist for the Column object.

More info: [pyspark.sql.Column.isin](#) - PySpark 3.1.2 documentation

**NO.171** Which of the following code blocks shuffles DataFrame transactionsDf, which has 8 partitions, so that it has 10 partitions?

**A.** transactionsDf.repartition(transactionsDf.getNumPartitions()+2)

**B.** transactionsDf.repartition(transactionsDf.rdd.getNumPartitions()+2)

**C.** transactionsDf.coalesce(10)

**D.** transactionsDf.coalesce(transactionsDf.getNumPartitions()+2)

**E.** transactionsDf.repartition(transactionsDf.\_partitions+2)

**Answer: B**

Explanation

```
transactionsDf.repartition(transactionsDf.rdd.getNumPartitions()+2)
```

Correct. The repartition operator is the correct one for increasing the number of partitions. calling getNumPartitions() on DataFrame.rdd returns the current number of partitions.

```
transactionsDf.coalesce(10)
```

No, after this command transactionsDf will continue to only have 8 partitions. This is because coalesce() can only decrease the amount of partitions, but not increase it.

```
transactionsDf.repartition(transactionsDf.getNumPartitions()+2)
```

Incorrect, there is no getNumPartitions() method for the DataFrame class.

```
transactionsDf.coalesce(transactionsDf.getNumPartitions()+2)
```

Wrong, coalesce() can only be used for reducing the number of partitions and there is no getNumPartitions() method for the DataFrame class.

```
transactionsDf.repartition(transactionsDf._partitions+2)
```

No, DataFrame has no \_partitions attribute. You can find out the current number of partitions of a DataFrame with the DataFrame.rdd.getNumPartitions() method.

More info: [pyspark.sql.DataFrame.repartition](#) - PySpark 3.1.2 documentation,

[pyspark.RDD.getNumPartitions](#) - PySpark 3.1.2 documentation Static notebook | Dynamic notebook:

See test 3

**NO.172** Which of the following code blocks selects all rows from DataFrame transactionsDf in which column productId is zero or smaller or equal to 3?

**A.** transactionsDf.filter(productId==3 or productId<1)

**B.** transactionsDf.filter((col("productId")==3) or (col("productId")<1))

**C.** transactionsDf.filter(col("productId")==3 | col("productId")<1)

**D.** transactionsDf.where("productId=3).or("productId"<1))

**E.** transactionsDf.filter((col("productId")==3) | (col("productId")<1))

**Answer: E**

Explanation

This question targets your knowledge about how to chain filtering conditions. Each filtering condition

should be in parentheses. The correct operator for "or" is the pipe character (|) and not the word or. Another operator of concern is the equality operator. For the purpose of comparison, equality is expressed as two equal signs (==).

Static notebook | Dynamic notebook: See test 2

**NO.173** Which of the following code blocks silently writes DataFrame itemsDf in avro format to location fileLocation if a file does not yet exist at that location?

- A. `itemsDf.write.avro(fileLocation)`
- B. `itemsDf.write.format("avro").mode("ignore").save(fileLocation)`
- C. `itemsDf.write.format("avro").mode("errorifexists").save(fileLocation)`
- D. `itemsDf.save.format("avro").mode("ignore").write(fileLocation)`
- E. `spark.DataFrameWriter(itemsDf).format("avro").write(fileLocation)`

**Answer:** A

Explanation

The trick in this question is knowing the "modes" of the DataFrameWriter. Mode ignore will ignore if a file already exists and not replace that file, but also not throw an error. Mode errorifexists will throw an error, and is the default mode of the DataFrameWriter. The question NO:

explicitly calls for the DataFrame to be "silently" written if it does not exist, so you need to specify mode("ignore") here to avoid having Spark communicate any error to you if the file already exists. The 'overwrite' mode would not be right here, since, although it would be silent, it would overwrite the already-existing file. This is not what the question asks for.

It is worth noting that the option starting with `spark.DataFrameWriter(itemsDf)` cannot work, since spark references the SparkSession object, but that object does not provide the DataFrameWriter. As you can see in the documentation (below), DataFrameWriter is part of PySpark's SQL API, but not of its SparkSession API.

More info:

DataFrameWriter: `pyspark.sql.DataFrameWriter.save` - PySpark 3.1.1 documentation  
 SparkSession API: Spark SQL - PySpark 3.1.1 documentation  
 Static notebook | Dynamic notebook: See test 1

**NO.174** Which of the following describes the role of the cluster manager?

- A. The cluster manager schedules tasks on the cluster in client mode.
- B. The cluster manager schedules tasks on the cluster in local mode.
- C. The cluster manager allocates resources to Spark applications and maintains the executor processes in client mode.
- D. The cluster manager allocates resources to Spark applications and maintains the executor processes in remote mode.
- E. The cluster manager allocates resources to the DataFrame manager.

**Answer:** C

Explanation

The cluster manager allocates resources to Spark applications and maintains the executor processes in client mode.

Correct. In cluster mode, the cluster manager is located on a node other than the client machine.

From there it starts and ends executor processes on the cluster nodes as required by the Spark application running on the Spark driver.

The cluster manager allocates resources to Spark applications and maintains the executor processes

in remote mode.

Wrong, there is no "remote" execution mode in Spark. Available execution modes are local, client, and cluster.

The cluster manager allocates resources to the DataFrame manager

Wrong, there is no "DataFrame manager" in Spark.

The cluster manager schedules tasks on the cluster in client mode.

No, in client mode, the Spark driver schedules tasks on the cluster - not the cluster manager.

The cluster manager schedules tasks on the cluster in local mode.

Wrong: In local mode, there is no "cluster". The Spark application is running on a single machine, not on a cluster of machines.

**NO.175** Which of the following code blocks writes DataFrame itemsDf to disk at storage location filePath, making sure to substitute any existing data at that location?

- A. itemsDf.write.mode("overwrite").parquet(filePath)
- B. itemsDf.write.option("parquet").mode("overwrite").path(filePath)
- C. itemsDf.write(filePath, mode="overwrite")
- D. itemsDf.write.mode("overwrite").path(filePath)
- E. itemsDf.write().parquet(filePath, mode="overwrite")

**Answer:** A

Explanation

itemsDf.write.mode("overwrite").parquet(filePath)

Correct! itemsDf.write returns a pyspark.sql.DataFrameWriter instance whose overwriting behavior can be modified via the mode setting or by passing mode="overwrite" to the parquet() command. Although the parquet format is not prescribed for solving this question, parquet() is a valid operator to initiate Spark to write the data to disk.

itemsDf.write.mode("overwrite").path(filePath)

No. A pyspark.sql.DataFrameWriter instance does not have a path() method.

itemsDf.write.option("parquet").mode("overwrite").path(filePath)

Incorrect, see above. In addition, a file format cannot be passed via the option() method.

itemsDf.write(filePath, mode="overwrite")

Wrong. Unfortunately, this is too simple. You need to obtain access to a DataFrameWriter for the DataFrame through calling itemsDf.write upon which you can apply further methods to control how Spark data should be written to disk. You cannot, however, pass arguments to itemsDf.write directly.

itemsDf.write().parquet(filePath, mode="overwrite")

False. See above.

More info: pyspark.sql.DataFrameWriter.parquet - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 3

**NO.176** The code block shown below should add a column itemNameBetweenSeparators to DataFrame itemsDf. The column should contain arrays of maximum 4 strings. The arrays should be composed of the values in column itemsDf which are separated at - or whitespace characters. Choose the answer that correctly fills the blanks in the code block to accomplish this.

Sample of DataFrame itemsDf:

1. +-----+-----+-----+-----+
2. |itemId|itemName|supplier|
3. +-----+-----+-----+-----+

4. | 1 | Thick Coat for Walking in the Snow | Sports Company Inc. |

5. | 2 | Elegant Outdoors Summer Dress | YetiX |

6. | 3 | Outdoors Backpack | Sports Company Inc. |

7. +-----+-----+-----+-----+

Code block:

```
itemsDf.__1__(__2__, __3__(__4__, "[\\s\\-]", __5__))
```

**A.** 1. withColumn

2. "itemNameBetweenSeparators"

3. split

4. "itemName"

5. 4

(Correct)

**B.** 1. withColumnRenamed

2. "itemNameBetweenSeparators"

3. split

4. "itemName"

5. 4

**C.** 1. withColumnRenamed

2. "itemName"

3. split

4. "itemNameBetweenSeparators"

5. 4

**D.** 1. withColumn

2. "itemNameBetweenSeparators"

3. split

4. "itemName"

5. 5

**E.** 1. withColumn

2. itemNameBetweenSeparators

3. str\_split

4. "itemName"

5. 5

**Answer:** A

Explanation

This question deals with the parameters of Spark's split operator for strings.

To solve this question, you first need to understand the difference between `DataFrame.withColumn()` and `DataFrame.withColumnRenamed()`. The correct option here is `DataFrame.withColumn()` since, according to the question, we want to add a column and not rename an existing column. This leaves you with only 3 answers to consider.

The second gap should be filled with the name of the new column to be added to the DataFrame.

One of the remaining answers states the column name as `itemNameBetweenSeparators`, while the other two state it as `"itemNameBetweenSeparators"`. The correct option here is

`"itemNameBetweenSeparators"`, since the other option would let Python try to interpret `itemNameBetweenSeparators` as the name of a variable, which we have not defined. This leaves you with 2 answers to consider.

The decision boils down to how to fill gap 5. Either with 4 or with 5. The question asks for arrays of

maximum four strings. The code in gap 5 relates to the limit parameter of Spark's split operator (see documentation linked below). The documentation states that "the resulting array's length will not be more than limit", meaning that we should pick the answer option with 4 as the code in the fifth gap here.

On a side note: One answer option includes a function `str_split`. This function does not exist in pySpark.

More info: `pyspark.sql.functions.split` - PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3

**NO.177** Which of the following are valid execution modes?

- A. Kubernetes, Local, Client
- B. Client, Cluster, Local
- C. Server, Standalone, Client
- D. Cluster, Server, Local
- E. Standalone, Client, Cluster

**Answer:** B

Explanation

This is a tricky question to get right, since it is easy to confuse execution modes and deployment modes. Even in literature, both terms are sometimes used interchangeably.

There are only 3 valid execution modes in Spark: Client, cluster, and local execution modes. Execution modes do not refer to specific frameworks, but to where infrastructure is located with respect to each other.

In client mode, the driver sits on a machine outside the cluster. In cluster mode, the driver sits on a machine inside the cluster. Finally, in local mode, all Spark infrastructure is started in a single JVM (Java Virtual Machine) in a single computer which then also includes the driver.

Deployment modes often refer to ways that Spark can be deployed in cluster mode and how it uses specific frameworks outside Spark. Valid deployment modes are standalone, Apache YARN, Apache Mesos and Kubernetes.

Client, Cluster, Local

Correct, all of these are the valid execution modes in Spark.

Standalone, Client, Cluster

No, standalone is not a valid execution mode. It is a valid deployment mode, though.

Kubernetes, Local, Client

No, Kubernetes is a deployment mode, but not an execution mode.

Cluster, Server, Local

No, Server is not an execution mode.

Server, Standalone, Client

No, standalone and server are not execution modes.

More info: Apache Spark Internals - Learning Journal

**NO.178** Which of the following code blocks shows the structure of a DataFrame in a tree-like way, containing both column names and types?

- A. `1.print(itemsDf.columns)`  
`2.print(itemsDf.types)`
- B. `itemsDf.printSchema()`

- C. `spark.schema(itemsDf)`
- D. `itemsDf.rdd.printSchema()`
- E. `itemsDf.print.schema()`

**Answer:** B

Explanation

`itemsDf.printSchema()`

Correct! Here is an example of what `itemsDf.printSchema()` shows, you can see the tree-like structure containing both column names and types:

root

```
|-- itemId: integer (nullable = true)
|-- attributes: array (nullable = true)
| |-- element: string (containsNull = true)
|-- supplier: string (nullable = true)
```

`itemsDf.rdd.printSchema()`

No, the DataFrame's underlying RDD does not have a `printSchema()` method.

`spark.schema(itemsDf)`

Incorrect, there is no `spark.schema` command.

`print(itemsDf.columns)`

`print(itemsDf.dtypes)`

Wrong. While the output of this code blocks contains both column names and column types, the information is not arranged in a tree-like way.

`itemsDf.print.schema()`

No, DataFrame does not have a `print` method.

Static notebook | Dynamic notebook: See test 3

**NO.179** Which of the following code blocks returns all unique values of column stored in DataFrame `transactionsDf`?

- A. `transactionsDf["storeId"].distinct()`
- B. `transactionsDf.select("storeId").distinct()`  
(Correct)
- C. `transactionsDf.filter("storeId").distinct()`
- D. `transactionsDf.select(col("storeId").distinct())`
- E. `transactionsDf.distinct("storeId")`

**Answer:** B

Explanation

`distinct()` is a method of a DataFrame. Knowing this, or recognizing this from the documentation, is the key to solving this question.

More info: `pyspark.sql.DataFrame.distinct` - PySpark 3.1.2 documentation Static notebook | Dynamic notebook: See test 2